



CiteGraph-NLP: A Confidence-Aware System for Citation Lineage Analysis and Study-Scale Evidence Extraction

A thesis submitted in partial fulfilment of the requirements for the degree of
Bachelor of Science in Artificial Intelligence

Department of Computer Science
National University of Technology

Authors

Name	Registration Number
M. Ahad Imran	F23607034
Syed Zain-ul-Abidin	F23607031
Anas Zafar	F22607024

Supervisor: Dr. Amna Ikram

Session: 2023–2027

Declaration

We declare that this thesis and the system it describes are our own work, except where explicitly attributed. All external data is retrieved from public scholarly APIs under their published terms of use. No paywalled full text was scraped or redistributed. Every quantitative result reported in Chapter 6 was produced by a script committed to the project repository and can be recomputed by a reader with network access; the commands are given in Appendix C.

Provenance of the work. An initial prototype of this system was developed in the sixth semester. The project has been carried forward into the Final Year Project in the seventh semester, and the team composition has changed since the prototype phase. The work presented here substantially extends that prototype. The evaluation reported in Chapter 6 was designed and executed during the FYP phase, and the corrections catalogued in Chapter 5, including the traversal, weighting

and extraction defects, were identified and resolved in this phase. Where a result or component predates the FYP, this is stated at the point of use rather than presented as new work.

Where the delivered system falls short of the original project proposal, this thesis states the shortfall explicitly rather than describing the intended behaviour as though it had been achieved. Section 6.8 and Chapter 7 catalogue these divergences.

Abstract

Determining which earlier papers a body of research actually rests on is a manual and error-prone task. A reader who wants to trace a claim back to its origin must follow reference lists by hand, and the citation counts that bibliometric tools report treat every citation as equally informative. A citation to a 43,548-participant randomised trial and a citation to a three-patient case report are indistinguishable in a raw citation graph.

This thesis presents **CiteGraph-NLP**, a system that constructs a citation graph from a single seed paper and weights its edges by the *scale of the evidence* each cited paper reports, rather than by citation count alone. The system resolves a user-supplied identifier (DOI, PMID, PMCID, title or URL) against three public scholarly APIs, performs a depth-limited breadth-first traversal over references and citing works, extracts reported study population sizes from abstracts using a pattern-based information-extraction pipeline, weights each citation edge by a normalised population score scaled by extraction confidence, and ranks probable foundational papers using PageRank over the weighted graph.

The contribution is not a new ranking algorithm. PageRank on citation networks is long established [chen2007gems; walker2007citerank]. It is a working, measured pipeline that makes the *uncertainty* of automated evidence extraction visible to the user: every extracted population carries a confidence score and a status of *resolved*, *ambiguous* or *missing*, and edges whose evidence could not be established fall back to a uniform structural weight rather than silently disappearing.

The system was evaluated against a 20-paper gold standard annotated from abstract text, spanning randomised trials, cohort studies, case series, epidemiological and modelling designs, together with deliberately hard negatives whose abstracts contain large numbers that are *not* study populations. Population detection achieved precision 0.917, recall 1.000 and F1 0.957; exact sample-size values were correct for 10 of 11 positive cases; semantic type classification was correct for 6 of 10. Metadata resolution succeeded for 20 of 20 papers. Three live citation traversals produced graphs with zero dangling edges and zero isolated nodes.

The evaluation also exposes the system's limits honestly. Semantic type accuracy of 60% shows that distinguishing *randomised* from *enrolled* from *analysed* is not solved by surface patterns. A systematic review produced a confident false positive. Twenty-eight per cent of papers in a typical graph carry no abstract in the primary metadata source at all, and a second provider had to be queried to recover them. Full-text parsing, specified in the project proposal, was not implemented; the system reads abstracts only.

Keywords: citation analysis, knowledge graphs, information extraction, biomedical NLP, PageRank, evidence synthesis, scholarly APIs

Acknowledgements

We are grateful to **Sir Rooshan Saleem**, who teaches Natural Language Processing, for the idea this project started from and for telling us to build it. The suggestion that citation graphs could be weighted by the evidence the cited papers actually report, rather than by how often they are cited, is the premise the whole system rests on. Without that push there would have been no prototype to carry into the Final Year Project.

We thank our supervisor, **Dr. Amna Ikram**, for guiding us through every stage of this FYP. Her direction shaped the scope, the evaluation and the decision to report the system's failures as carefully as its successes. Chapter 6 states where the system falls short of the original proposal, and that it does so at all is a result of her insistence that measured results matter more than claimed ones.

We also thank the maintainers of OpenAlex, Crossref and Europe PMC, whose open APIs made this work possible at no cost, and the reviewers of our interim demonstrations for questions that sent us back to the measurements.

Table of Contents

1. **Introduction:** motivation, problem statement, objectives, scope, contributions
2. **Literature Review:** citation analysis, scholarly infrastructure, biomedical information extraction, evidence appraisal
3. **Requirements and Methodology:** requirements capture, development process, evaluation strategy
4. **System Design and Architecture:** layered architecture, data model, algorithms
5. **Implementation:** pipeline stages, provider integration, engineering defects and their resolution
6. **Evaluation and Results:** gold standard, measured results, performance, failure analysis
7. **Discussion:** interpretation, threats to validity, divergence from the proposal
8. **Conclusion and Future Work**

Appendices

- A. API Reference
 - B. Configuration Reference
 - C. Reproducing the Results
 - D. Gold Standard Annotations
 - E. Requirements Traceability
 - F. Test Inventory
 - G. Source Listings
 - H. Full-Scale Generated Diagrams
-

List of Figures

Every structural figure is generated from the source tree by `scripts/generate_diagrams.py`; Appendix H records how, and carries the full-scale plates the chapter figures are reduced from.

Figure Subject

- 4.1 Inter-package dependencies
 - 4.2 Object composition among related classes
 - 4.3 Data model, query to population
 - 4.4 Data model, population to run result
 - 5.1 Calls made by the pipeline orchestrator
 - H.1 Combined data model, all seven entities
 - H.2 Complete class diagram
 - H.3 Complete call graph
-

List of Abbreviations

Term	Expansion
------	-----------

API	Application Programming Interface
BFS	Breadth-First Search
CORS	Cross-Origin Resource Sharing
DOI	Digital Object Identifier
FYP	Final Year Project
IE	Information Extraction
JATS	Journal Article Tag Suite
N_eff	Effective study population size
NLP	Natural Language Processing
PICO	Population, Intervention, Comparison, Outcome
PMCID	PubMed Central Identifier
PMID	PubMed Identifier
RCT	Randomised Controlled Trial
SPA	Single-Page Application
SSRF	Server-Side Request Forgery
TLS	Transport Layer Security

CiteGraph-NLP

- [Chapter 1: Introduction](#)
 - [1.1 Background and Motivation](#)
 - [1.2 Problem Statement](#)
 - [1.3 Research Questions](#)
 - [1.4 Aim and Objectives](#)
 - [1.5 Scope and Delimitations](#)
 - [1.6 Contributions](#)
 - [1.7 Thesis Structure](#)
- [Chapter 2: Literature Review](#)
 - [2.1 Citation indexing and bibliometric indicators](#)
 - [2.1.1 The citation index](#)
 - [2.1.2 Count-based indicators and their known confounds](#)
 - [2.2 Structure and dynamics of citation networks](#)
 - [2.2.1 Empirical distribution of citations](#)
 - [2.2.2 Network models of scientific collaboration and citation](#)
 - [2.2.3 PageRank applied to citation networks](#)
 - [2.2.4 Co-citation analysis and visualisation](#)
 - [2.3 Scholarly data infrastructure](#)
 - [2.4 Biomedical information extraction](#)
 - [2.4.1 Annotated corpora](#)
 - [2.4.2 PICO extraction](#)
 - [2.4.3 Domain-adapted language models](#)
 - [2.4.4 Automated evidence appraisal](#)
 - [2.5 Evidence appraisal and the role of study size](#)
 - [2.6 Synthesis and research gap](#)
- [Chapter 3: Requirements and Methodology](#)
 - [3.1 Requirements Elicitation](#)
 - [3.2 Functional Requirements](#)
 - [3.2.1 Input handling](#)
 - [3.2.2 Metadata resolution](#)
 - [3.2.3 Citation traversal](#)
 - [3.2.4 Population extraction](#)
 - [3.2.5 Weighting, ranking and output](#)
 - [3.3 Non-functional requirements](#)
 - [3.4 Development Methodology](#)
 - [3.4.1 Process](#)
 - [3.4.2 Version control and quality gates](#)
 - [3.5 Evaluation Strategy](#)
 - [3.5.1 Evaluation design](#)
 - [3.5.2 Gold standard construction](#)
 - [3.5.3 Statistical treatment](#)
 - [3.5.4 Acknowledged limitations of the evaluation](#)
 - [3.5.5 What is not evaluated](#)
- [Chapter 3 \(continued\): Project Management](#)
 - [3.6 Team Organisation](#)
 - [3.7 Development Timeline](#)
 - [3.8 Risk Management](#)
 - [3.9 Tools and Infrastructure](#)
 - [3.10 Lessons for Process](#)

- [Chapter 4: System Design and Architecture](#)
 - [4.1 Architectural Overview](#)
 - [4.1.1 Design principles](#)
 - [4.2 Data Model](#)
 - [4.2.1 Paper](#)
 - [4.2.2 PopulationCandidate and PopulationResolution](#)
 - [4.2.3 Semantic types](#)
 - [4.2.4 CitationEdge](#)
 - [4.3 Metadata Resolution](#)
 - [4.3.1 Parallel query and merge](#)
 - [4.3.2 Identifier canonicalisation](#)
 - [4.4 Citation Traversal](#)
 - [4.4.1 Level-synchronous breadth-first search](#)
 - [4.4.2 Identity aliasing](#)
 - [4.4.3 Abstract backfill](#)
 - [4.4.4 Budget allocation](#)
 - [4.5 Edge Weighting](#)
 - [4.5.1 Formulation](#)
 - [4.5.2 Why confidence scales only the evidential term](#)
 - [4.5.3 Acknowledged simplification](#)
 - [4.6 Graph Analytics](#)
 - [4.6.1 Foundational paper ranking](#)
 - [4.6.2 Citation path ranking](#)
 - [4.7 Surfacing Uncertainty](#)
- [Chapter 4 \(continued\): Algorithm Specifications](#)
 - [4.8 Algorithm 1: metadata merge](#)
 - [4.8.1 Problem](#)
 - [4.8.2 Specification](#)
 - [4.8.3 Complexity and notes](#)
 - [4.9 Algorithm 2: level-synchronous citation traversal](#)
 - [4.9.1 Problem](#)
 - [4.9.2 Specification](#)
 - [4.9.3 Complexity](#)
 - [4.9.4 Failure modes handled](#)
 - [4.10 Algorithm 3: population candidate extraction](#)
 - [4.10.1 Specification](#)
 - [4.10.2 Complexity](#)
 - [4.10.3 The span-versus-sentence decision](#)
 - [4.11 Algorithm 4: population resolution](#)
 - [4.11.1 Specification](#)
 - [4.11.2 Known weakness](#)
 - [4.12 Algorithm 5: bounded path ranking](#)
 - [4.12.1 Problem](#)
 - [4.12.2 Specification](#)
 - [4.12.3 Complexity](#)
- [Chapter 5: Implementation](#)
 - [5.1 Technology Stack](#)
 - [5.2 Pipeline Orchestration](#)
 - [5.3 Defect 1: traversal discarding the majority of every graph](#)
 - [5.3.1 Symptom](#)
 - [5.3.2 Diagnosis](#)
 - [5.3.3 A second, independent defect in the same component](#)
 - [5.3.4 Resolution](#)

- [5.4 Defect 2: a single null field discarding batches of fifty](#)
 - [5.4.1 Symptom](#)
 - [5.4.2 Diagnosis](#)
 - [5.4.3 Resolution](#)
- [5.5 Defect 3: ignore rules discarding the numbers they protect](#)
 - [5.5.1 Symptom](#)
 - [5.5.2 Diagnosis](#)
 - [5.5.3 Resolution](#)
- [5.6 Defect 4: edge weights collapsing to zero](#)
 - [5.6.1 Symptom](#)
 - [5.6.2 Diagnosis](#)
 - [5.6.3 Resolution](#)
- [5.7 Defect 5: server-side request forgery in URL input](#)
- [5.8 Defect 6: exports returning JSON envelopes](#)
- [5.9 Deployment](#)
- [5.10 Discussion: why these defects survived](#)
- [Chapter 5 \(continued\): Dashboard and Interaction Design](#)
 - [5.11 Technology and Structure](#)
 - [5.11.1 Route structure](#)
 - [5.12 The submission flow](#)
 - [5.13 Representing Uncertainty](#)
 - [5.13.1 Status before number](#)
 - [5.13.2 The supporting sentence is reachable](#)
 - [5.13.3 Absence is reported, not implied](#)
 - [5.13.4 An honest limitation of the current presentation](#)
 - [5.14 Graph Visualisation](#)
 - [5.15 Export Design](#)
- [Chapter 6: Evaluation and Results](#)
 - [6.1 Gold Standard](#)
 - [6.1.1 Composition](#)
 - [6.1.2 Hard negatives](#)
 - [6.1.3 Annotation protocol and its limits](#)
 - [6.2 Metadata Resolution](#)
 - [6.3 Abstract Availability](#)
 - [6.4 Population Extraction](#)
 - [6.4.1 Detection](#)
 - [6.4.2 Value accuracy](#)
 - [6.4.3 Semantic type classification](#)
 - [6.4.4 The false positive, and confidence calibration](#)
 - [6.5 Citation graph construction](#)
 - [6.6 Performance](#)
 - [6.6.1 Stage breakdown](#)
 - [6.6.2 Scaling with graph size](#)
 - [6.6.3 Effect of connection pooling](#)
 - [6.6.4 Path ranking complexity](#)
 - [6.7 Methodological note: reference verification](#)
 - [6.8 Summary of results against objectives](#)
- [Chapter 6 \(continued\): Extended Analysis](#)
 - [6.9 Performance by study design](#)
 - [6.10 The confidence calibration failure in detail](#)
 - [6.11 Sensitivity of the graph to seed choice](#)
 - [6.12 What the abstract backfill changed, stage by stage](#)
 - [6.13 Cumulative effect of the corrections](#)

- [Chapter 7: Discussion](#)
 - [7.1 Interpretation of Results](#)
 - [7.1.1 Extraction works better than the project expected; availability is the real ceiling](#)
 - [7.1.2 Semantic typing is the genuine weakness](#)
 - [7.1.3 The confidence scores do not mean what they appear to](#)
 - [7.1.4 The system is I/O-bound, and that is a design conclusion](#)
 - [7.2 Reflections on the Development Process](#)
 - [7.3 Threats to Validity](#)
 - [7.3.1 Construct validity](#)
 - [7.3.2 Internal validity](#)
 - [7.3.3 External validity](#)
 - [7.3.4 Conclusion validity](#)
 - [7.4 Divergence from the Project Proposal](#)
 - [7.5 Known open issues](#)
 - [7.6 Ethical and legal considerations](#)
- [Chapter 8: Conclusion and Future Work](#)
 - [8.1 Conclusion](#)
 - [8.2 Future Work](#)
 - [8.2.1 Calibrate the confidence scores](#)
 - [8.2.2 Compare against an unweighted baseline](#)
 - [8.2.3 Extend and re-annotate the gold standard](#)
 - [8.2.4 Supervised extraction](#)
 - [8.2.5 Full-text extraction](#)
 - [8.2.6 Study-level deduplication](#)
 - [8.2.7 A frozen evaluation corpus](#)
 - [8.2.8 Citation context and sentiment](#)
 - [8.3 Closing Remarks](#)
- [Appendix A: API Reference](#)
 - [A.1 GET /health](#)
 - [A.2 POST /api/runs](#)
 - [A.3 GET /api/runs/{run_id}](#)
 - [A.4 GET /api/runs/{run_id}/graph](#)
 - [A.5 Exports](#)
- [Appendix B: Configuration Reference](#)
 - [B.1 Application](#)
 - [B.2 Security](#)
 - [B.3 Providers](#)
 - [B.4 Traversal and weighting](#)
 - [B.5 Storage and optional services](#)
 - [B.6 Internal constants](#)
- [Appendix C: Reproducing the Results](#)
 - [C.1 Environment](#)
 - [C.2 Test suite](#)
 - [C.3 Evaluation \(Chapter 6\)](#)
 - [C.4 Reference verification \(Section 6.7\)](#)
 - [C.5 Rebuilding the thesis](#)
 - [C.6 Regenerating the figures](#)
 - [C.7 Rendering to PDF](#)
 - [C.8 Running the system](#)
 - [C.9 A caveat on exact reproduction](#)
- [Appendix D: Gold Standard Annotations](#)
 - [D.1 Summary](#)

- [D.2 Annotations with supporting evidence](#)
 - [D.2.1 10.1056/nejmoa2034577](#)
 - [D.2.2 10.1056/nejmoa2035389](#)
 - [D.2.3 10.1056/nejmoa1911303](#)
 - [D.2.4 10.1056/nejmoa1812389](#)
 - [D.2.5 10.1016/s0140-6736\(20\)31604-4](#)
 - [D.2.6 10.1056/nejmoa2002032](#)
 - [D.2.7 10.1016/s0140-6736\(20\)30183-5](#)
 - [D.2.8 10.1001/jama.2020.1585](#)
 - [D.2.9 10.1056/nejmoa2001316](#)
 - [D.2.10 10.1016/s2213-2600\(20\)30079-5](#)
 - [D.2.11 10.1016/s1473-3099\(20\)30243-7](#)
 - [D.2.12 10.1056/nejmoa2001017](#)
 - [D.2.13 10.1038/s41586-020-2012-7](#)
 - [D.2.14 10.1136/bmj.m1328](#)
 - [D.2.15 10.1038/s41577-020-0311-8](#)
 - [D.2.16 10.1164/rccm.201908-1581st](#)
 - [D.2.17 10.1056/nejmra2026131](#)
 - [D.2.18 10.1038/s41586-021-03819-2](#)
 - [D.2.19 10.1038/nature14539](#)
 - [D.2.20 10.1145/3065386](#)
- [Appendix E: Requirements Traceability](#)
 - [E.1 Functional requirements](#)
 - [E.2 Non-functional requirements](#)
 - [E.3 Proposal capabilities not delivered](#)
 - [E.4 Research questions to evidence](#)
- [Appendix F: Test Suite and Verification Inventory](#)
 - [F.1 Composition](#)
 - [F.2 Regression tests added during evaluation](#)
 - [F.3 Defects found *in the test suite itself*](#)
 - [F.4 What the suite does not establish](#)
 - [F.5 Manual verification performed](#)
- [Appendix G: Selected Code Listings](#)
 - [G.1 Per-span ignore matching \(Section 5.5\)](#)
 - [G.2 Edge weighting \(Section 5.6\)](#)
 - [G.3 Identity aliasing and duplicate merging \(Section 4.4.2\)](#)
 - [G.4 Commit-time edge filtering \(Section 4.4.2\)](#)
 - [G.5 Null-safe provider mapping \(Section 5.4\)](#)
 - [G.6 SSRF guard \(Section 5.7\)](#)
 - [G.7 Bounded path enumeration \(Section 4.12\)](#)
 - [G.8 Retry policy \(Section 5.4\)](#)
 - [G.9 Pooled HTTP client \(Section 6.6.3\)](#)
 - [G.10 Evaluation metric definitions \(Section 3.5.3\)](#)
- [Appendix H: Full-Scale Generated Diagrams](#)
 - [H.1 How the figures are produced](#)
 - [H.2 Why the chapter figures are reduced](#)
 - [H.3 Plates](#)
 - [H.4 A caveat on the call graphs](#)
- [References](#)

Chapter 1: Introduction

1.1 Background and Motivation

Scientific knowledge is cumulative, and the citation is the mechanism by which that accumulation is recorded. When a paper cites another, it declares a dependency: some part of the new work rests on the earlier one. Garfield's proposal for a citation index [garfield1955] rested on exactly this observation, that the network of citations is itself a navigable structure, and that following it backwards should let a reader reach the origin of an idea.

Seventy years later, the network exists and is machine-readable. OpenAlex [priem2022openalex], Crossref [hendricks2020crossref] and Europe PMC [europepmc2015] between them expose hundreds of millions of works and their reference lists through open APIs. What has not kept pace is the interpretation of that network. The dominant summary statistics, citation count, h-index [hirsch2005hindex], journal impact factor, all treat citations as interchangeable units. A citation is a citation.

That assumption is convenient and, for many purposes, wrong. Consider two papers in the same reference list of a clinical review:

- A multinational randomised controlled trial that randomised 43,548 participants across two arms.
- A case report describing three patients.

Both contribute exactly one to the cited paper's citation count. Both appear as a single unweighted edge in a citation graph. Yet a reader tracing the evidential basis of a clinical claim would not treat them as equivalent for a moment. The scale of the study population is one of the most basic signals a reader uses when judging how much weight a piece of evidence can bear, and it is entirely absent from the citation graph as normally constructed.

The methodological literature reinforces the point. Ioannidis argues that a large fraction of published findings are false, with small study size among the principal contributors [ioannidis2005why]. Button and colleagues demonstrate that low statistical power both reduces the chance of detecting a true effect and reduces the probability that a detected effect is real [button2013power]. If sample size materially changes how much a result should be trusted, then a graph that discards sample size is discarding something a reader needs.

1.2 Problem Statement

A researcher who wants to know *which earlier papers a body of work actually rests on* currently has three unsatisfactory options.

Manual reference following. Open the seed paper, read its reference list, open the interesting references, repeat. This is accurate but does not scale. A paper with forty references, each with forty references, presents 1,600 second-level works.

Citation-count ranking. Use a bibliometric tool to sort candidates by citation count. This scales, but rewards popularity rather than evidential weight, and is well known to be confounded by field, age and venue [waltman2016review]. Highly cited papers are disproportionately recent reviews rather than foundational primary studies.

Systematic review methodology. Follow PRISMA [moher2009prisma] and appraise each study with an instrument such as the Cochrane risk-of-bias tool [higgins2011cochrane]. This is the

rigorous answer and is the correct one when the stakes justify it, but a full systematic review is a months-long undertaking by a trained team.

The gap this project addresses sits between the second and third options: an automated traversal that is aware, however imperfectly, of how much evidence each cited paper actually reports, and that is honest about the imperfection.

1.3 Research Questions

This project is organised around four questions.

RQ1. Can study population sizes be extracted from abstracts reliably enough to be used as an edge-weighting signal, using pattern-based information extraction rather than a trained model?

RQ2. What proportion of papers reachable in a citation traversal actually carry the text needed for such extraction, and does that proportion constrain the approach more than extraction accuracy does?

RQ3. Does weighting citation edges by extracted population evidence produce a foundational-paper ranking that a domain reader would find defensible?

RQ4. Can a citation traversal over public scholarly APIs be made fast enough for interactive use, and what dominates its cost?

Chapter 6 answers RQ1, RQ2 and RQ4 with measured results. RQ3 is answered qualitatively and with explicit acknowledgement that no formal relevance judgement study was conducted; Section 7.3 treats this as a threat to validity.

1.4 Aim and Objectives

Aim. To design, implement and evaluate a system that constructs a citation graph from a single seed paper, weights its edges by extracted study-scale evidence, and surfaces the uncertainty of that extraction to the user.

Objectives.

1. Normalise heterogeneous scholarly identifiers (DOI, PMID, PMCID, free-text title, article URL) to a single canonical form.
2. Resolve metadata for a paper by querying multiple scholarly APIs in parallel and merging the results with field-level precedence.
3. Traverse the citation network in both directions to a configurable depth under a bounded budget of papers.
4. Extract candidate study population sizes from abstract text and classify them into semantic categories.
5. Resolve competing candidates within a paper to a single effective population size with an explicit confidence and status.
6. Weight citation edges using the normalised population score, scaled by extraction confidence, without discarding edges that lack evidence.
7. Rank probable foundational papers using PageRank over the weighted graph.
8. Expose the analysis through a REST API and a web dashboard, with exports in machine-readable and human-readable formats.
9. Evaluate each module against the metrics specified in the project proposal.

Objective 9 deserves particular note. The project proposal specified an evaluation plan covering four modules, but at the point where this thesis was begun, the evaluation directory in the repository was empty and no metric had been computed. Chapter 6 reports the results of implementing that plan.

1.5 Scope and Delimitations

In scope. Biomedical and clinical literature, where reported sample sizes are conventional and abstracts are structured. Abstract-level extraction. Graphs bounded at 200 papers. Backward references to depth 3 and forward citations to depth 2. Three metadata providers: OpenAlex, Crossref, Europe PMC.

Explicitly out of scope.

- *Full-text parsing.* The project proposal specified GROBID [lopez2009grobid] for PDF structure extraction. This was not implemented. The system reads abstracts only. Section 7.4 discusses the consequence.
- *Exhaustive citation retrieval.* A heavily cited paper may have tens of thousands of citing works; the system samples the most-cited subset within its paper budget. It does not claim completeness.
- *Clinical interpretation.* An extracted population size is a reading of the abstract, not a verified study characteristic. The system is an exploratory aid, not an evidence-appraisal instrument.
- *Study-level deduplication.* The proposal described a study-aware graph in which several papers reporting one trial collapse to a single study node. The implementation maps papers to studies one-to-one.

1.6 Contributions

This thesis makes the following contributions.

C1. A working, deployed pipeline. An end-to-end system from identifier to ranked graph, deployed and publicly reachable, with 102 automated tests.

C2. An edge-weighting formulation that degrades honestly. Population evidence scales the evidential term of the edge weight while a structural term always applies, so an edge whose evidence could not be extracted retains a uniform non-zero weight instead of vanishing. Section 5.6 shows that the original formulation, multiplying the entire weight by confidence, silently collapsed every edge weight to exactly zero for any paper outside clinical phrasing, and that the defect was masked by a Python truthiness accident.

C3. A measured evaluation with hard negatives. A 20-paper gold standard in which nine negatives contain large but irrelevant numbers, so false positives are measured rather than hidden. Results in Chapter 6.

C4. A quantified account of the data-availability ceiling. Measurement showing that 28% of papers in a representative graph carry no abstract in the primary provider, that all of those were recoverable from a second provider, and that this data gap constrained recall more than extraction logic did.

C5. A documented catalogue of defects found by measurement. Chapter 5 records eight defects in the delivered system that were found only by instrumenting and measuring it, including a traversal fault that discarded the majority of every graph. These are reported because a thesis

that describes only the working system teaches less than one that describes how the faults were found.

1.7 Thesis Structure

Chapter 2 reviews citation analysis, scholarly data infrastructure, biomedical information extraction and evidence appraisal, and positions this work against each.

Chapter 3 states functional and non-functional requirements, describes the development methodology, and defines the evaluation strategy.

Chapter 4 presents the architecture, the data model and the algorithms.

Chapter 5 describes the implementation, including the defects found during measurement and how each was resolved.

Chapter 6 reports the evaluation: gold standard construction, measured results for each module, performance characterisation, and failure analysis.

Chapter 7 interprets the results, states threats to validity, and catalogues every divergence between the proposal and the delivered system.

Chapter 8 concludes and sets out future work.

Chapter 2: Literature Review

This chapter surveys four bodies of work that the project draws on: citation analysis and bibliometrics (2.1), the structure of citation networks (2.2), scholarly data infrastructure (2.3), and biomedical information extraction (2.4). Section 2.5 reviews evidence appraisal, which supplies the project's motivating assumption. Section 2.6 states the gap.

Every work cited in this chapter was resolved against Crossref or OpenAlex before being cited; the verification procedure and the resulting metadata are given in Appendix E. Three candidate references were discarded during that process because the identifiers initially assigned to them resolved to different papers; this is reported in Section 6.7 as a methodological note.

2.1 Citation indexing and bibliometric indicators

2.1.1 The citation index

Garfield's 1955 proposal introduced the citation index as an instrument for science rather than merely a bibliographic convenience [garfield1955]. The central argument was that the citation is a *directed assertion of dependence*, and that a machine-readable index of such assertions would let a reader move through the literature along lines of intellectual descent rather than by subject heading. The paper anticipates the use this project makes of citation data: following edges backwards to reach antecedents.

Garfield also anticipated the failure mode. He noted that a citation may be critical as easily as supportive, and that an index records the link without recording its valence. This limitation is unresolved seventy years on and applies directly to the present work: CiteGraph-NLP treats every citation as a positive dependency because the available APIs expose no sentiment.

2.1.2 Count-based indicators and their known confounds

The h-index [hirsch2005hindex] proposed a single number combining productivity and impact: an author has index h if h of their papers have at least h citations each. Its appeal is robustness to both a long tail of uncited work and a single exceptional paper.

Waltman's review of citation impact indicators [waltman2016review] surveys the subsequent literature and catalogues the confounds that affect this whole family of measures. Three matter here. Citation practice differs by field, so raw counts are not comparable across disciplines. Counts accumulate with time, so older papers are advantaged and recent work is systematically under-represented. And review articles attract citations disproportionately to primary studies, which is precisely inverted relative to what a reader tracing foundational evidence wants.

This project responds to the third confound directly. The foundational ranking described in Section 4.6 combines PageRank with an age term, deliberately advantaging older work, because the target is the origin of a line of research rather than its most popular recent summary. Whether this succeeds is examined in Section 6.5, and Section 7.3 notes that no formal relevance study was run.

2.2 Structure and dynamics of citation networks

2.2.1 Empirical distribution of citations

Redner's empirical study of citation distributions [redner1998citation] established that citation counts are extremely heavy-tailed: the large majority of papers receive few citations while a small minority receive very many. Radicchi and colleagues [radicchi2008universality] showed that after normalising by field-average citation count, the distributions across disciplines collapse onto a common curve, which both explains the incomparability of raw counts and suggests a normalisation.

The practical consequence for this project is a design constraint rather than a theoretical one. Because the distribution is heavy-tailed, a seed paper may have tens of thousands of citing works, the AlphaFold paper used as a test case in Chapter 6 has over thirty thousand. No interactive traversal can enumerate them. Section 4.5 describes the sampling strategy this forces, and Section 7.3 treats the resulting incompleteness as a threat to validity.

2.2.2 Network models of scientific collaboration and citation

Newman's analysis of scientific collaboration networks [newman2001structure] characterised the small-world and clustering properties of coauthorship graphs. Although coauthorship is not citation, the structural findings, short path lengths, high clustering, heavy-tailed degree, recur in citation graphs and bear on this system: short paths mean a bounded-depth traversal reaches substantial portions of a local neighbourhood, and high clustering means the papers retrieved tend to be topically coherent, which is observed empirically in Section 6.5.

2.2.3 PageRank applied to citation networks

Brin and Page's description of the Google search engine [brin1998anatomy] introduced PageRank as a recursive measure of importance over a directed graph: a node is important if important nodes point to it. The method was devised for hyperlinks, but citation graphs are directed graphs of the same shape.

Chen and colleagues applied PageRank directly to a physics citation network [chen2007gems] and found that it surfaces papers that raw citation count does not, work that is cited by influential papers rather than by many papers. They characterise these as "scientific gems". Walker and colleagues proposed CiteRank [walker2007citerank], a model of network traffic that adds an explicit ageing term so that recent papers are not penalised purely for having had less time to accumulate citations.

These two papers are the closest prior art to the ranking component of this project, and the relationship should be stated plainly: **the ranking method used here is not novel**. PageRank over a citation graph, with an age adjustment, is exactly what Chen et al. and Walker et al. describe. The difference in this work is not the algorithm but the *edge weights* it operates on. Chen et al. run PageRank over an unweighted citation graph; CiteRank weights by age. CiteGraph-NLP weights each edge by extracted study-scale evidence. Whether that weighting is an improvement is not established by this thesis and is not claimed; Section 7.3 is explicit that no comparative ranking experiment against an unweighted baseline was conducted.

2.2.4 Co-citation analysis and visualisation

Chen's CiteSpace [chen2007cocitation] provides interactive visual analytics over co-citation networks, detecting emerging trends and intellectual turning points. It shares this project's

premise that a citation network is best interrogated visually and interactively rather than reduced to a ranking. It differs in unit of analysis: CiteSpace works from a corpus supplied by the user, typically a search result set, whereas CiteGraph-NLP expands outward from a single seed.

2.3 Scholarly data infrastructure

The feasibility of this project rests entirely on the availability of open citation data. Four sources are relevant.

Crossref [hendricks2020crossref] is the DOI registration agency for scholarly publishing and the canonical source for publisher-deposited metadata, including reference lists where publishers deposit them. Crossref's coverage of reference lists is uneven because deposit is at publisher discretion, a limitation this project encountered directly and quantified in Section 6.3.

OpenAlex [priem2022openalex] is an open catalogue of scholarly works succeeding the discontinued Microsoft Academic Graph. It provides works, authors, venues and, critically here, both `referenced_works` and a `cites` filter supporting forward citation queries. It is the primary provider in this system.

Microsoft Academic Graph [wang2020mag] was the prior generation of this infrastructure. Its discontinuation, and OpenAlex's emergence as successor, is a reminder that a system built on a single provider inherits that provider's lifespan, one motivation for the multi-provider design in Section 4.3.

Europe PMC [europepmc2015] is a full-text literature database for the life sciences providing abstracts and, for open-access content, full text. In this system it began as a third metadata provider and became, as measured in Section 6.3, the decisive source of abstract text: 28% of papers in a representative graph carried no abstract in OpenAlex, and all of them were recoverable from Europe PMC.

Unpaywall [martin2021oadoi] characterises the open-access state of the literature. Its relevance is to the full-text extension discussed in Section 8.2: any move from abstracts to full text must first determine which articles are legally retrievable.

2.4 Biomedical information extraction

2.4.1 Annotated corpora

The GENIA corpus [kim2003genia] established the pattern for biomedical IE evaluation: a semantically annotated corpus of abstracts supporting supervised training and comparable evaluation. Its significance for this project is methodological, it demonstrates that abstract-level annotation is a legitimate evaluation substrate, which is the basis for the gold standard in Section 6.1.

2.4.2 PICO extraction

The task closest to this project's extraction component is PICO element detection: identifying Population, Intervention, Comparison and Outcome spans in clinical abstracts. The EBM-NLP corpus [nye2018ebmnlp] provides multi-level annotations of these elements over a large set of abstracts. Jin and Szolovits [jin2018pico] approach the detection task with LSTM models.

The *P* of PICO is the element this project extracts, but with an important narrowing. PICO population annotation captures the described population, "adults over 65 with type 2 diabetes", as

a text span. CiteGraph-NLP extracts the *cardinality* of that population: the integer count of subjects. These are related but distinct tasks, and the distinction matters when comparing results. A PICO system that correctly identifies a population span has not necessarily identified a sample size.

The methodological gap is worth stating plainly: EBM-NLP and the systems trained on it are supervised, trained on thousands of annotated abstracts. This project uses hand-written patterns evaluated on twenty. Section 7.3 treats the resulting limits on generalisability as the principal threat to validity in the extraction results, and Section 8.2 identifies supervised extraction as the clearest improvement path.

2.4.3 Domain-adapted language models

BioBERT [lee2020biobert] and SciBERT [beltagy2019scibert] demonstrated that pretraining transformer models on biomedical and scientific text respectively yields substantial gains over general-domain models on domain tasks. ScispaCy [neumann2019scispacy] packages practical biomedical NLP pipelines.

None of these is used in the delivered system, which extracts using regular expressions. This is a deliberate scope decision rather than an oversight, and the trade-off should be stated honestly. Pattern-based extraction is transparent, every decision traces to a named pattern and can be explained to a user, and requires no training data, which the project did not have. It is also brittle in exactly the way Section 6.4 measures: the system distinguishes *randomised* from *enrolled* from *analysed* correctly in only 6 of 10 cases, a discrimination a domain-adapted model would be expected to make more reliably.

2.4.4 Automated evidence appraisal

RobotReviewer [marshall2016robotreviewer] automatically assesses risk of bias in clinical trial reports, and Trialstreamer [marshall2020trialstreamer] maintains a living, automatically updated database of randomised controlled trials with extracted characteristics including sample size.

Trialstreamer is the nearest system-level prior art to this project, and the comparison is instructive rather than favourable. Trialstreamer extracts trial characteristics at scale using trained models over full text, with published evaluation. CiteGraph-NLP extracts one characteristic from abstracts using patterns, evaluated on twenty papers. What this project does that Trialstreamer does not is use the extracted scale *as a citation-graph edge weight*, the combination of extraction with network analysis is where the work sits, not in the extraction itself.

2.5 Evidence appraisal and the role of study size

The premise that sample size is a meaningful proxy for evidential weight is supported by, but not identical to, the methodological literature.

PRISMA [moher2009prisma] standardises the reporting of systematic reviews, and the Cochrane risk-of-bias tool [higgins2011cochrane] structures appraisal of individual trials. Both make clear that sample size is *one* dimension among several, allocation concealment, blinding, attrition and selective reporting all bear on trustworthiness, and none is captured by a participant count.

Ioannidis [ioannidis2005why] and Button et al. [button2013power] supply the argument for why size nonetheless matters: underpowered studies both miss true effects and inflate the proportion of detected effects that are spurious.

The honest position for this project is therefore narrow. Sample size is a *cheaply extractable* signal that correlates with evidential weight and is currently absent from citation graphs. It is not a measure of study quality, and the system must not be presented as appraising quality. Section 4.7 describes how this caution is surfaced in the interface, and the generated analysis reports carry an explicit statement to that effect.

2.6 Synthesis and research gap

The reviewed literature supports four observations.

1. Citation graphs are navigable and machine-readable at scale [garfield1955; priem2022openalex; hendricks2020crossref].
2. Count-based indicators are confounded, and PageRank-family methods over citation graphs partially address this [waltman2016review; chen2007gems; walker2007citerank].
3. Study characteristics including sample size can be extracted automatically from clinical text [nye2018ebmnl; marshall2020trialstreamer].
4. Sample size is one legitimate, if partial, component of evidential weight [ioannidis2005why; button2013power].

Observations 2 and 3 have developed largely independently. Citation-graph analysis treats edges as uniform; evidence extraction produces study characteristics that are used for search and synthesis but not fed back into the network. **The gap this project addresses is the connection between them: using automatically extracted study-scale evidence as a weight on citation edges, and propagating that weight through a graph ranking.**

Two qualifications bound the claim. First, the components are individually established; the contribution is their combination and a measured account of how well it works, not a new method. Second, this thesis does not demonstrate that evidence-weighted ranking is *better* than unweighted ranking, no comparative experiment was performed, and Section 7.3 states this as the principal unaddressed question.

Chapter 3: Requirements and Methodology

3.1 Requirements Elicitation

Requirements were derived from three sources, in order of authority: the project proposal approved by the department, a Product Requirement Document (PRD) written during the design phase, and a feasibility study commissioned before implementation began.

The feasibility study is worth describing because it shaped the scope materially. It assessed each intended component, metadata resolution, PDF parsing, population extraction, citation traversal, graph construction, and judged the difficulty of each independently. Its conclusion on full-text parsing was the decisive one: obtaining legal full text at scale is gated by open-access status rather than by parsing capability, and a pipeline that depends on full text will fail for a large fraction of inputs regardless of how good its parser is. The delivered system reads abstracts only, and Chapter 6 quantifies what that costs.

3.2 Functional Requirements

Requirements are stated in the form *the system shall*, with a verification method and the section reporting the result.

3.2.1 Input handling

ID	Requirement	Verification
FR-1	Accept a DOI, PMID, PMCID, free-text title or article URL as the seed identifier	Automated test, §6.2
FR-2	Canonicalise all identifiers to a single normalised form	Automated test
FR-3	Extract an embedded identifier from an article URL without fetching the page where possible	Automated test, §5.7
FR-4	Reject malformed identifiers with a message naming the expected form	Automated test

3.2.2 Metadata resolution

ID	Requirement	Verification
FR-5	Query OpenAlex, Crossref and Europe PMC concurrently	Code inspection
FR-6	Merge provider results with field-level precedence	Code inspection
FR-7	Record per-provider provenance for each resolved paper	Data inspection
FR-8	Recover abstracts absent from the primary provider from a secondary provider	Measured, §6.3

3.2.3 Citation traversal

ID	Requirement	Verification
FR-9	Traverse backward references to a configurable depth (0–3)	Measured, §6.5
FR-10	Traverse forward citations to a configurable depth (0–2)	Measured, §6.5
FR-11	Bound the total number of papers per run (1–200)	Automated test
FR-12	Collapse records describing one work under multiple identifiers onto one node	Measured, §6.5
FR-13	Emit no edge whose endpoints are not both present as nodes	Measured, §6.5

3.2.4 Population extraction

ID	Requirement	Verification
FR-14	Extract candidate population sizes from abstract text	Measured, §6.4
FR-15	Classify each candidate into a semantic type	Measured, §6.4
FR-16	Resolve competing candidates to one effective size per paper	Measured, §6.4
FR-17	Attach a confidence score and a status of resolved, ambiguous or missing	Measured, §6.4
FR-18	Not treat years, percentages, p-values or dosages as population sizes	Automated test, §5.5

3.2.5 Weighting, ranking and output

ID	Requirement	Verification
FR-19	Weight each edge by normalised population evidence scaled by confidence	Code inspection, §5.6
FR-20	Assign a non-zero weight to edges lacking population evidence	Automated test, §5.6
FR-21	Rank probable foundational papers over the weighted graph	Measured, §6.5
FR-22	Rank citation paths from the seed	Measured, §6.6
FR-23	Export results as JSON, CSV, edge-list CSV, Markdown and GraphML	Automated test, §5.8
FR-24	Provide a REST API and a web dashboard	Deployment, §5.9

3.3 Non-functional requirements

ID	Requirement	Target	Result
NFR-1	A 40-paper run completes within 60 s	< 60 s	19 s mean, §6.6
NFR-2	Graph analytics are a minority of runtime	$< 25\%$	2.5%, §6.6
NFR-3	Provider failures degrade the run rather than aborting it	No unhandled exception	§5.4
NFR-4	No paywalled full text is retrieved	Zero	By construction
NFR-5	Outbound fetches cannot reach private network addresses	Zero	§5.7
NFR-6	Browser access is restricted to a configured origin	Configured origin only	§5.9
NFR-7	Every reported metric is recomputable from a committed script	100%	Appendix C

NFR-7 is a response to a specific failure mode. The project proposal specified an evaluation plan that was never executed, and its absence was not visible because nothing in the repository recorded that the metrics were missing. Making recomputability an explicit requirement means a gap of that kind becomes detectable.

3.4 Development Methodology

3.4.1 Process

Development followed an iterative cycle of building, measuring and correcting rather than a waterfall. Given a three-person team, a fixed academic deadline and external dependencies whose behaviour was not fully known in advance, planning the system completely before building it was not realistic, a judgement the feasibility study supported.

Each iteration comprised: implement a pipeline stage; write automated tests; run the stage against live scholarly APIs; inspect the output for correctness; correct what the inspection revealed.

The final phase of the project departed from this pattern in a way worth recording, because it produced most of the defects catalogued in Chapter 5. Rather than implementing new features, that phase consisted of *instrumenting and measuring the existing system*. Eight defects were found. Several had been present since early in development, had passed every test, and had produced output that appeared plausible. Section 5.10 draws the methodological lesson.

3.4.2 Version control and quality gates

All work is version-controlled in Git with descriptive commits. The test suite comprises 102 automated tests. Continuous integration deploys the backend to a shared host and the frontend to a content delivery network on merge to the main branch.

Two properties of the test suite are worth noting because they bear on Chapter 5. First, the suite passed in full throughout the period in which the traversal defect described in Section 5.3 was discarding the majority of every graph; the tests asserted that the pipeline completed and returned a structure of the right shape, not that the structure was correct. Second, two defects in the test

suite itself were found, one hung the entire suite indefinitely, and one masked a failure on the database read path. A test suite is software and is subject to the same defects as the system it tests.

3.5 Evaluation Strategy

3.5.1 Evaluation design

Evaluation follows the module structure of the proposal's evaluation plan: metadata resolution, population extraction, citation graph construction, and ranking. Each is evaluated against a measurable criterion where one exists, and reported as unmeasured where one does not.

3.5.2 Gold standard construction

Population extraction is evaluated against a hand-annotated gold standard of 20 papers. The construction protocol is:

1. Select papers spanning study designs, randomised trials, cohort studies, case series, epidemiological analyses, modelling studies, plus negatives: reviews, guidelines, and computational papers with no human subjects.
2. Retrieve the abstract through the same providers the pipeline uses.
3. Read each abstract and record the effective population size, defined as the total number of human subjects the paper's primary analysis rests on as stated in the abstract.
4. Record the supporting sentence verbatim so every label is auditable.
5. Where no human study population is stated, label the paper negative; the correct system behaviour is to extract nothing.

Nine of the twenty are negatives, and several are *hard* negatives whose abstracts contain large, salient numbers that are not study populations: an epidemic case tally of 2,794 laboratory-confirmed infections, an ImageNet dataset size of 1.2 million images, and a systematic review whose units are studies rather than patients. Including these is a deliberate design choice, a positives-only gold set cannot measure false positives, and false positives are the failure mode that most damages user trust.

3.5.3 Statistical treatment

Proportions are reported with 95% Wilson score intervals. The Wilson interval is used rather than the normal approximation because the sample is small ($n = 20$ overall, $n = 11$ positives), where the normal approximation is unreliable and degenerates entirely at proportions of 0 or 1.

The intervals are wide, and Chapter 6 reports them alongside every point estimate rather than quoting point estimates alone. A recall of 1.000 on eleven positives is consistent with a true recall as low as roughly 0.74.

3.5.4 Acknowledged limitations of the evaluation

Four limitations are stated in advance because they bound every result in Chapter 6.

- **Single annotator.** The gold standard was annotated by one person, who is also an author of the system. No inter-annotator agreement was measured. This is the most serious limitation; Section 7.3 discusses it.
- **Small sample.** Twenty papers, eleven positive. Interval estimates are correspondingly wide.

- **Abstracts only.** Sample sizes stated solely in a full-text Methods section are out of scope. This matches what the pipeline reads, so the evaluation is fair to the system as built, but it does not measure the task in general.
- **Domain concentration.** The positives are biomedical. Performance on other literatures is not measured and should not be assumed.

3.5.5 What is not evaluated

Two things the proposal listed are not evaluated, and are reported as gaps rather than passed over.

Ranking quality. No relevance judgement study was conducted. The proposal proposed "manual inspection of selected papers", which is not a metric. Section 6.5 reports a qualitative observation and Section 7.3 records the absence of a formal ranking evaluation as an unaddressed threat.

Duplicate detection accuracy. Duplicate merging is measured by counting merges performed and verifying no duplicate survives in the output, but no gold standard of known duplicate pairs was constructed, so precision of the merge decision is unmeasured. A merge that incorrectly collapses two distinct papers would not be detected by the present measurement.

Chapter 3 (continued): Project Management

3.6 Team Organisation

Three members worked across a pipeline that decomposes naturally into independent modules with narrow interfaces. Work was organised by module rather than by person, into the four areas below, so that two people rarely edited the same file; review was shared, and the fourth area was covered jointly.

Area	Primary responsibility
Provider layer, metadata resolution, identifier canonicalisation	Retrieval
Citation traversal, graph construction, analytics	Graph
Population extraction, resolution, pattern design	NLP
API, dashboard, deployment, CI	Delivery

The module boundaries described in Chapter 4 were as much a coordination mechanism as an architectural one. Because the provider layer exposes a fixed `ProviderResult` contract, the retrieval and graph work proceeded in parallel against a stub; because the extractor consumes plain text and emits candidates, the NLP work was testable without a working traversal.

This had a cost that Chapter 5 makes visible. Clean interfaces let each module be tested in isolation, and each module *was* correct in isolation. The traversal defect (Section 5.3) lived precisely at the seam, the traversal correctly asked the resolver for a paper, and the resolver correctly rejected an identifier it was never designed to receive. Both sides behaved as specified. Interface-level correctness does not compose into system-level correctness, and nothing in the division of labour was positioned to notice.

3.7 Development Timeline

Phase	Focus	Outcome
Proposal and feasibility	Scope definition; feasibility assessment of each component	Full-text parsing descope; abstract-only pipeline adopted
Design	PRD, data models, architecture	Module contracts fixed
Core implementation	Providers, resolver, traversal, extractor, graph	End-to-end pipeline producing output

Phase	Focus	Outcome
Interface	REST API, React dashboard, exports	System usable by a non-author
Deployment	Containerisation, CI, hosting, TLS	Publicly reachable instance
Measurement and correction	Instrumentation, evaluation harness, defect resolution	Eight defects found; evaluation plan implemented
Documentation	Thesis, reproducibility artifacts	This document

The final phase is the one worth commentary. It was originally scoped as "testing and documentation", a wrap-up phase. It became the phase in which most of the project's substantive faults were found, because it was the first time the system was measured rather than exercised.

Had the evaluation harness been built when the proposal specified it, the same defects would have surfaced months earlier and at lower cost. The empty `evaluation/` directory was, in retrospect, the single most informative artifact in the repository, and nobody read it as a warning.

3.8 Risk Management

Risks identified during planning, with what actually happened.

Risk	Planned mitigation	Outcome
Scholarly APIs rate-limit or block the client	Honour polite-pool conventions; identify the caller	Did not materialise. Polite-pool identification also improved latency (§C.1) Materialised, worse than expected.
Reference-list coverage is incomplete	Query several providers and merge	Crossref holds no reference list for many works; the mitigation was necessary rather than precautionary (§5.3)
Full-text access is legally constrained	Restrict to abstracts	Materialised as predicted by the feasibility study; abstract-only scope adopted from the outset
Population extraction is too inaccurate to be useful	Pattern-based approach with confidence scoring	Partially materialised: detection is strong, type classification and confidence calibration are not (§6.4)
A single provider becomes unavailable	Provider toggles; degrade rather than abort	Not triggered in practice; the degradation path is implemented and tested

Risk	Planned mitigation	Outcome
Graph algorithms do not scale	Bound the graph size	Over-mitigated. Analytics are 2.5% of runtime (§6.6.1); the real cost was network I/O

Two risks were *not* on the register and caused more disruption than those that were:

Silent data loss inside the pipeline. No risk entry anticipated that the system might run to completion while discarding most of its input. The register was oriented toward external failures, APIs down, rate limits, legal limits, and assumed internal correctness would follow from testing.

A test suite that passes while the system is wrong. The plan treated tests as the verification mechanism. Section 5.10 and Appendix F.4 record why that was insufficient for a system whose output is a ranked list.

3.9 Tools and Infrastructure

Purpose	Tool
Version control	Git / GitHub
Continuous integration	GitHub Actions
Backend hosting	Docker on a shared Linux host, nginx, Let's Encrypt
Frontend hosting	Cloudflare Pages
Run persistence	SQLite
Testing	pytest, pytest-asyncio, respx
Documentation	Markdown; Pandoc for rendering

Deployment is automated from the main branch: backend changes rebuild the container and restart it behind nginx; frontend changes rebuild the static bundle and publish it. The backend container binds the loopback interface only, because the host is shared with unrelated services, a constraint that produced its own defect, recorded in Section 5.9.

3.10 Lessons for Process

Three conclusions the team would carry forward.

Build the measurement harness with the first module, not after the last. The evaluation plan existed from the proposal. Deferring it deferred all the information it would have produced.

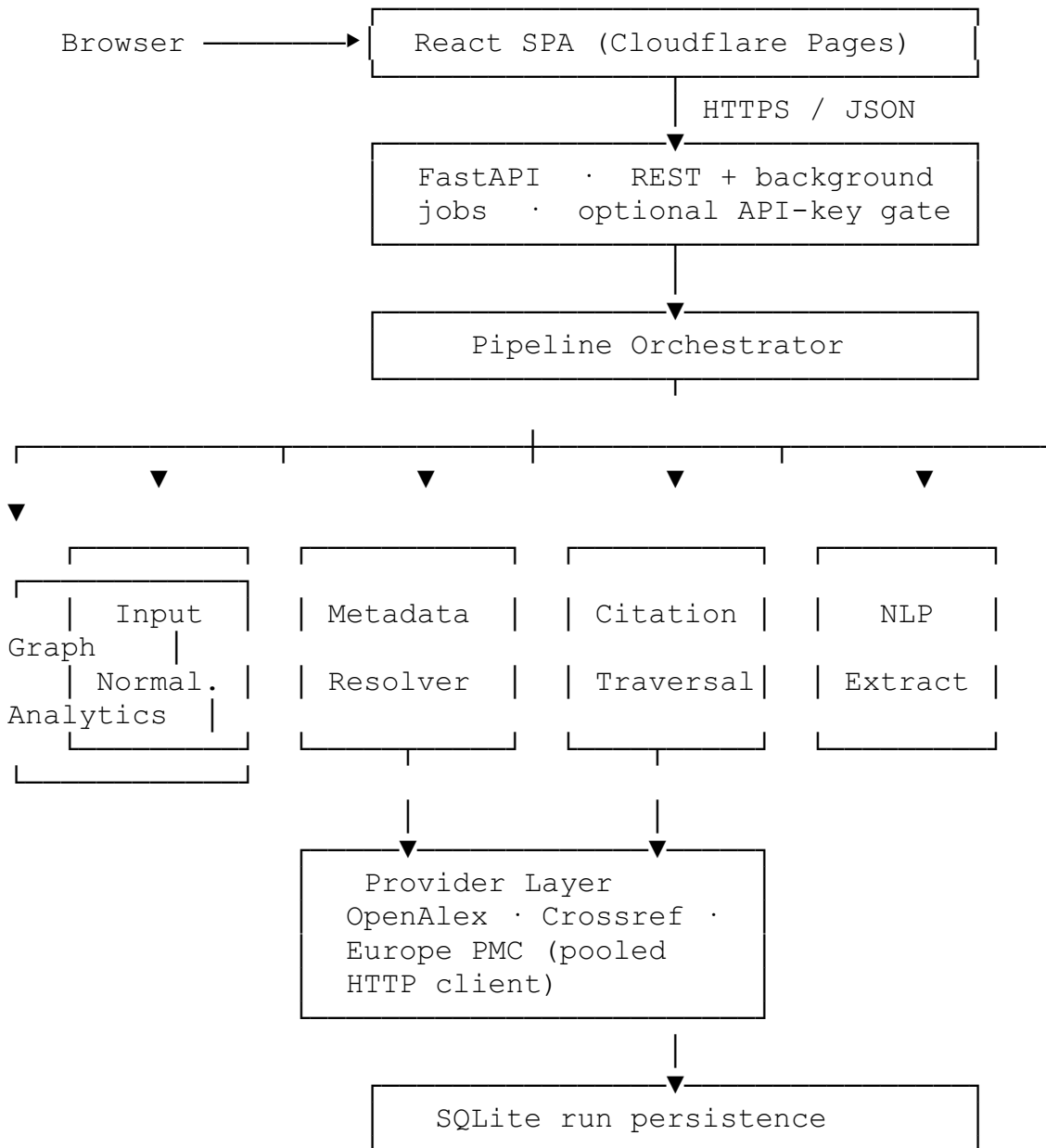
Prefer measurements over assertions for pipelines. A test that a stage returns the right *shape* is cheap and weak. A measurement compares a stage's output against a quantity known independently: how many references the provider reported, or what a human read in the abstract. That costs more to build and is far stronger.

Treat silence as suspicious. Every defect in Chapter 5 was silent: a warning-level log, an HTTP 200 with an empty result, a falsy value coerced to a default. A pipeline stage that never reports anything is not necessarily a stage that never has anything to report.

Chapter 4: System Design and Architecture

4.1 Architectural Overview

The system is a layered pipeline. A request enters at the API layer, passes through nine processing stages, and produces a persisted result that is read back by the dashboard and the export endpoints.



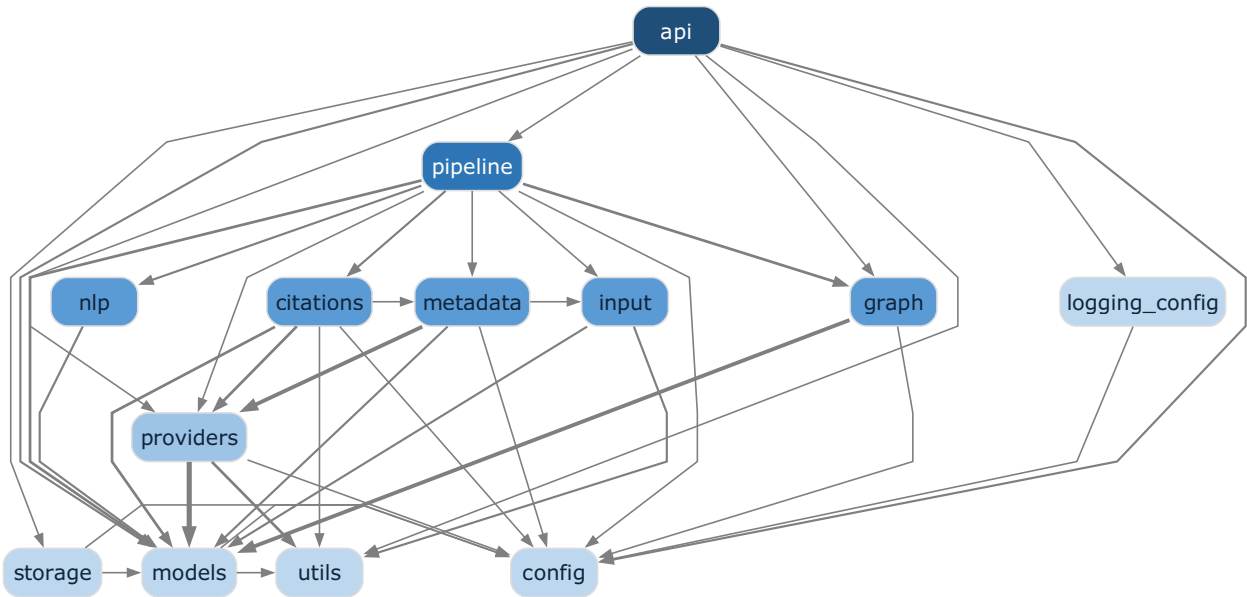


Figure 4.1 Inter-package dependencies, produced by running `pyreverse` over `src/citegraph` and collapsing its 46 module nodes to the 13 packages they belong to. Each arrow stands for one or more imports and is drawn thicker the more imports it carries. The heaviest arrows run downward into `models`, which holds the Pydantic types every other package constructs.

The figure is generated rather than drawn, so it shows what the code imports rather than what the design intended. Two properties are worth reading off it. Nothing below `pipeline` imports anything above it, so the layering claimed in Section 4.1.1 holds in the import graph and not only in prose. And `providers` is reached from `citations`, `metadata` and `pipeline` but reaches back to nothing except `models` and `config`, which is what makes the Europe PMC backfill of Section 4.4.3 a local change.

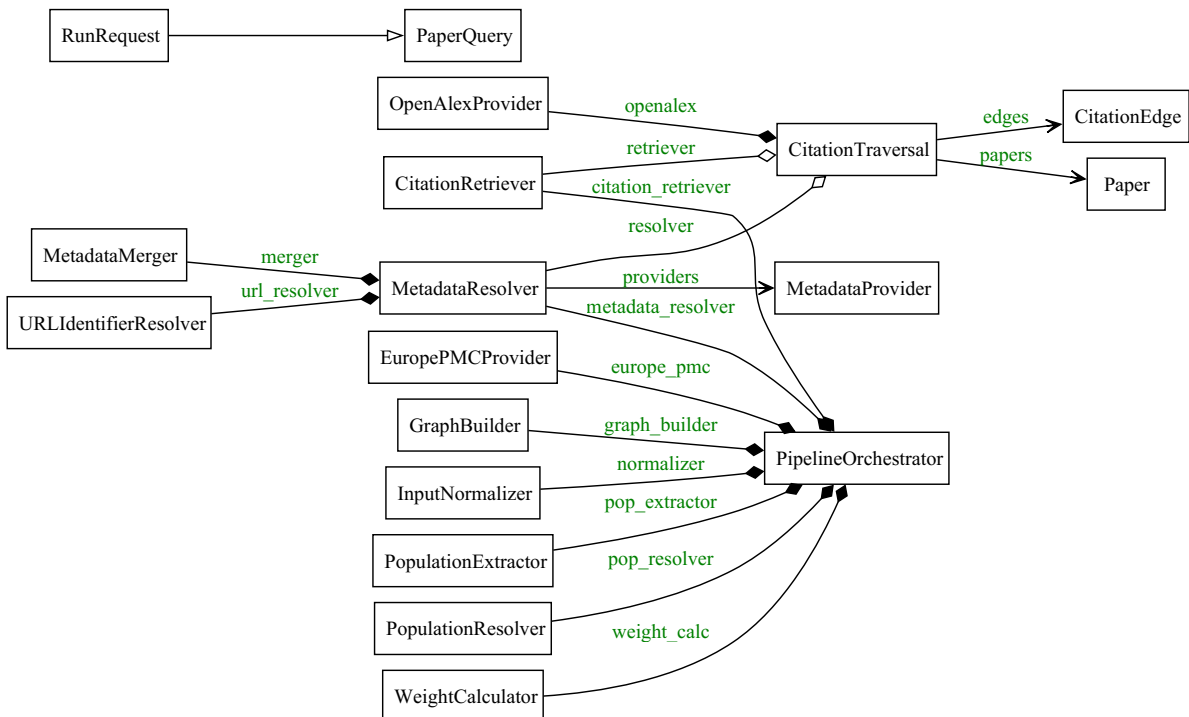


Figure 4.2 Object composition among the classes that participate in a relationship, from `pyreverse` with the 15 unrelated classes removed. `PipelineOrchestrator` composes eight collaborators, one per stage; edge labels are the attribute names the orchestrator stores them under.

4.1.1 Design principles

Separation of retrieval from interpretation. The provider layer knows how to talk to external APIs and nothing about population extraction or graph analysis. This allowed the Europe PMC abstract backfill described in Section 4.4.3 to be added as a pipeline stage without modifying either the traversal or the extractor.

Explicit uncertainty. Every population resolution carries a confidence and a status. Uncertainty is a first-class field in the data model rather than something inferred from a missing value, because "no population found" and "population found but ambiguous" are different states that a reader must be able to distinguish.

Bounded work. Every stage operates under an explicit budget. The citation graph is unbounded in principle, so traversal is limited by depth and by total paper count, and path enumeration is limited by a cap on paths examined. A stage that could run unboundedly is a stage that will eventually hang.

Degradation rather than failure. A provider that fails degrades the result instead of aborting the run. A paper whose abstract cannot be retrieved still appears as a node; its edges simply carry no evidential term.

4.2 Data Model

Five entities, defined as Pydantic models so that validation happens at construction.

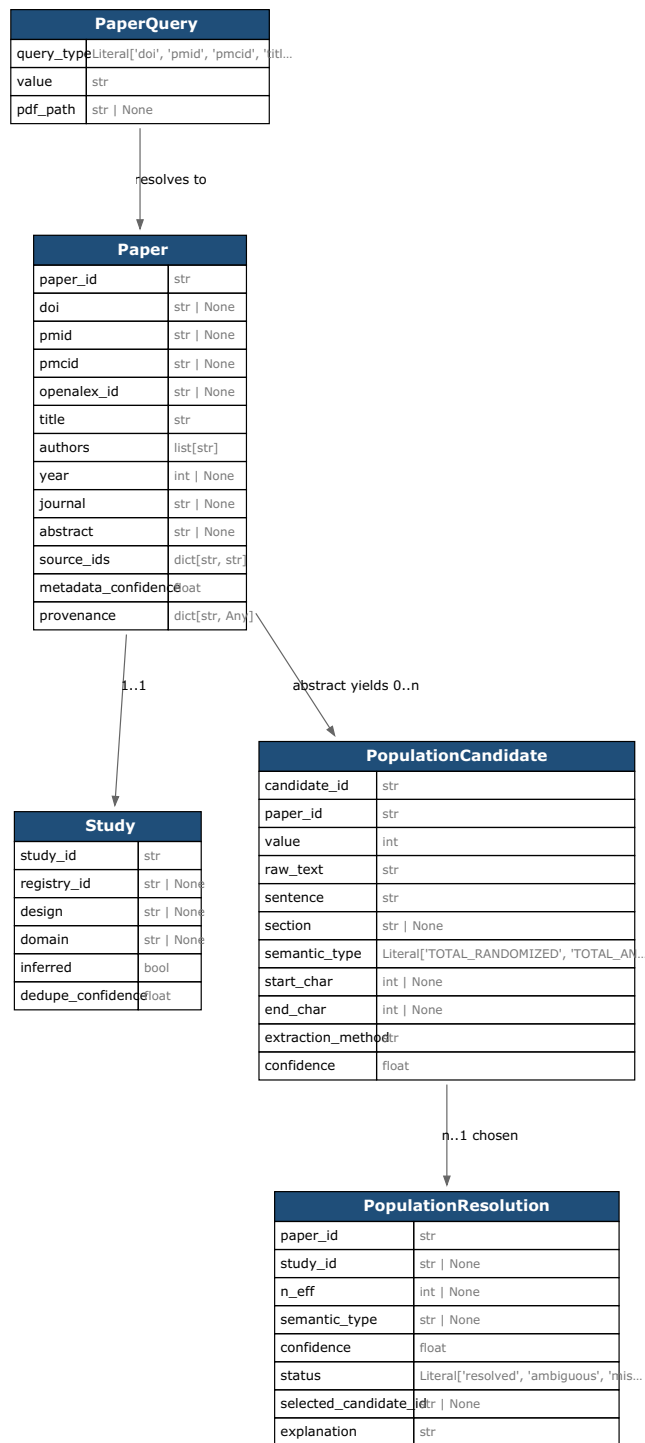


Figure 4.3 The extraction half of the data model, introspected from the Pydantic classes in `src/citegraph/models/`. Field names and types are read from the live model definitions, so this figure cannot drift from the code. A query resolves to a `Paper`; the abstract of that paper yields zero or more `PopulationCandidate` rows; and the resolver collapses those candidates to at most one `PopulationResolution`.

4.2.1 Paper

The canonical record of a work.

Field	Type	Notes
<code>paper_id</code>	<code>str</code>	Canonical identifier; DOI preferred, then PMID, then OpenAlex ID
<code>doi, pmid, pmcid, openalex_id</code>	<code>str?</code>	All known identifiers, retained for aliasing
<code>title, authors, year, journal</code>	<code>n/a</code>	Bibliographic metadata
<code>abstract</code>	<code>str?</code>	Extraction input; may be backfilled from a secondary provider
<code>source_ids</code>	<code>dict</code>	Provider-specific identifiers
<code>metadata_confidence</code>	<code>float</code>	Higher when several providers agree
<code>provenance</code>	<code>dict</code>	Per-provider retrieval record

The choice of a DOI-preferred `paper_id` is load-bearing. The same work retrieved from OpenAlex and from Crossref must collapse to one node, and the DOI is the identifier both share. Section 5.3 describes what happened before this rule was applied consistently.

4.2.2 PopulationCandidate and PopulationResolution

A *candidate* is one extracted number with its context; a *resolution* is the single value selected for a paper.

PopulationCandidate	Notes
<code>value</code>	The extracted integer
<code>raw_text, sentence</code>	The matched span and its sentence, for audit
<code>semantic_type</code>	One of the categories in 4.2.3
<code>section, start_char, end_char</code>	Provenance within the source text
<code>confidence</code>	Pattern weight plus a section bonus

PopulationResolution Notes

<code>n_eff</code>	Selected effective population, or null
<code>semantic_type</code>	Type of the selected candidate
<code>confidence</code>	Confidence of the selected candidate
<code>status</code>	<code>resolved</code> , <code>ambiguous</code> or <code>missing</code>
<code>explanation</code>	Human-readable justification

Retaining `sentence` for every candidate is what makes the extraction auditable: a user who doubts a number can read the sentence it came from. The gold-standard evaluation in Chapter 6 relies on the same property.

4.2.3 Semantic types

Type	Meaning
TOTAL_RANDOMIZED	Subjects randomised or assigned
TOTAL_ANALYZED	Subjects included in the analysis
TOTAL_ENROLLED	Subjects enrolled or recruited
SAMPLE_SIZE_GENERIC	An unqualified total
ARM_SIZE	Size of one arm or group
SCREENED	Subjects screened for eligibility
COMPLETERS	Subjects completing the protocol
FOLLOWUP_COUNT	Subjects followed up
EVENT_COUNT	Outcome event counts, <i>no pattern emits this</i>
UNKNOWN_NUMERIC	Unclassified, <i>no pattern emits this</i>

The last two are defined in the model but no extraction pattern produces them. They are listed here as specified-but-unimplemented rather than quietly omitted.

4.2.4 CitationEdge

Field	Notes
source_paper_id, target_paper_id	Direction: source cites target
providers	Which APIs asserted this edge
confidence	Confidence in the edge itself
n_score, journal_score	Weight components
base_weight, final_weight	Unconfidenced and final weights

Retaining the components rather than only the final weight means a user can see *why* an edge is weighted as it is, and the edge-list export exposes all of them.

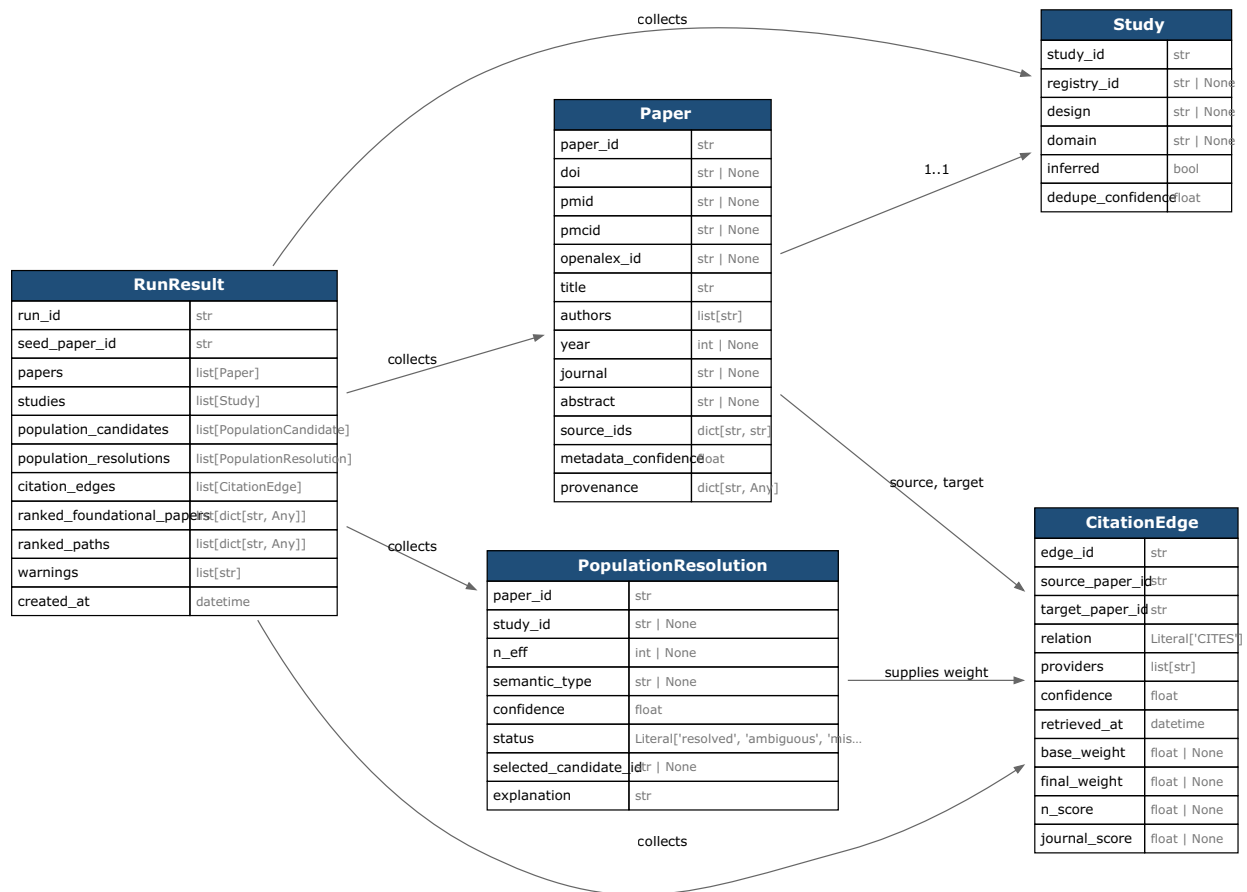


Figure 4.4 The graph half of the data model. **CitationEdge** draws its structural fields from two **Paper** records and its evidential fields (**n_score**, **final_weight**) from the **PopulationResolution** of its target. **RunResult** is the aggregate that is serialised to SQLite and returned to the dashboard.

4.3 Metadata Resolution

4.3.1 Parallel query and merge

The resolver queries all enabled providers concurrently and merges the results. Merging is field-level with per-field precedence rather than whole-record: Crossref is preferred for title, year, authors and journal because it carries publisher-deposited metadata; OpenAlex is preferred for abstracts because it reconstructs them from an inverted index.

Concurrency here is genuine parallelism of I/O, three providers queried at once rather than in sequence, and is the reason seed resolution costs under two seconds rather than six.

4.3.2 Identifier canonicalisation

All identifiers pass through a canonicaliser that:

- reduces `https://doi.org/10.x/y` to `10.x/y` and lowercases it;
- reduces `https://pubmed.ncbi.nlm.nih.gov/12345678` to `12345678`;
- uppercases PMcIDs and OpenAlex work identifiers;
- trims publisher view segments (`/full`, `/pdf`, `/abstract`) that a greedy DOI match absorbs from an article URL.

Each of these rules exists because its absence caused an observed failure. Section 5.7 documents the URL cases.

4.4 Citation Traversal

4.4.1 Level-synchronous breadth-first search

Traversal is breadth-first and **level-synchronous**: the complete frontier at depth d is expanded before any node at depth $d+1$. This is not the natural formulation, a simple queue-based BFS is shorter, but it is what makes batching possible. Because the entire frontier's neighbours are known at once, their metadata can be fetched in batches of fifty rather than one request per paper. Section 5.3 quantifies the difference.

4.4.2 Identity aliasing

A single work is referred to by different identifiers by different providers: OpenAlex returns W2741809807, Crossref returns 10.1016/j.jhin.2015.08.027. The traversal maintains an alias table mapping every identifier seen to one canonical `paper_id`. Edges are recorded with raw provider identifiers and rewritten through the alias table at commit time.

Two consequences follow, both load-bearing:

1. An edge is committed **only if both endpoints resolve**. An edge pointing at a paper that was never retrieved is dropped rather than emitted as a dangling reference. Section 6.5 measures the result: zero dangling edges.
2. Duplicate detection operates **before** the paper budget is consumed, so a merged duplicate never costs a graph slot. A run capped at 40 papers returns 40 distinct papers, not 40 minus the duplicates.

4.4.3 Abstract backfill

Population evidence can only be extracted from text. Measurement showed 28% of papers in a representative graph carry no abstract in OpenAlex, so a backfill stage was added between traversal and extraction: papers lacking an abstract are collected and queried against Europe PMC in a single OR-joined request. Section 6.3 reports the recovery rate.

4.4.4 Budget allocation

Both directions compete for one budget of papers. A heavily cited paper would otherwise consume the entire budget on citing works, starving the backward walk that finds foundational papers, which is the system's purpose. The traversal therefore reserves a share for each direction, defaulting to 65% backward and 35% forward, and releases a direction's unused reservation to the other when it can no longer expand.

4.5 Edge Weighting

4.5.1 Formulation

Each edge is weighted by the evidence reported in the **target** paper, the work being cited, since that is where the evidence being relied upon resides.

```

N_score          = log1p(N_eff) / log1p(N_reference)
evidence_term    =  $\alpha$  · N_score · pop_confidence
final_weight     = (evidence_term +  $\beta$  · journal_score) ·
edge_confidence

```

with $\alpha = 0.75$, $\beta = 0.25$, $N_{\text{reference}} = 100,000$ and `journal_score` currently a constant 0.5 (see 4.5.3).

The logarithm is deliberate. Sample sizes span several orders of magnitude, from single-digit case series to six-figure cohorts, and a linear scale would let one very large study dominate every path it appears on. On a log scale the difference between 100 and 1,000 participants carries the same weight as that between 1,000 and 10,000, which better matches how a reader treats the difference.

4.5.2 Why confidence scales only the evidential term

Confidence multiplies the evidence term only; the journal term always applies. This matters more than it appears. If confidence multiplied the whole weight, then a paper with no extractable population, confidence 0.0 by definition, would produce an edge weight of exactly zero. Every edge in a non-clinical graph would be zero, and the weighted ranking would silently become unweighted.

This is not hypothetical. It was the delivered behaviour, and Section 5.6 describes how it was found and why it was invisible.

4.5.3 Acknowledged simplification

`journal_score` is a constant 0.5 for every paper. The proposal envisaged a venue-quality term. It is not implemented, and the constant means the term contributes a uniform 0.125 to every edge, a floor rather than a discriminating signal. Section 7.4 lists this among the divergences from the proposal.

4.6 Graph Analytics

4.6.1 Foundational paper ranking

```
score = 0.5 · PageRank + 0.3 · year_score + 0.2 · evidence_score
```

`year_score` rises with age, saturating at twenty years; `evidence_score` is the normalised population score scaled by population confidence.

The age term is a deliberate inversion of the usual bibliometric bias. Citation counts favour recent, highly visible work [waltman2016review]; this system is looking for origins, so age is rewarded. The weights are not empirically tuned, they were set by judgement and no sensitivity analysis was performed, which Section 7.3 records as a limitation.

4.6.2 Citation path ranking

Paths from the seed are scored by mean edge weight, multiplied path confidence, and a depth penalty of $1/\sqrt{\text{length}}$.

Path enumeration is the one component with combinatorial risk. A densely interlinked graph contains an astronomical number of simple paths, and the naive formulation, enumerate all simple paths to each node, score them, sort, keep ten, is quadratic in a way that is easy to miss.

Section 5.6 reports the measurement and the redesign: a single depth-limited depth-first traversal feeding a bounded heap, with a hard cap on paths examined.

4.7 Surfacing Uncertainty

A system that reports an extracted number without reporting its reliability invites the reader to trust it more than the evidence warrants. Three mechanisms address this.

Status labels. Every resolution is `resolved`, `ambiguous` or `missing`. `ambiguous` means a competing candidate scored within 10% of the selected one with a materially different value, which is a genuine signal that the abstract is unclear rather than a system failure.

Confidence propagation. Confidence flows from pattern weight through resolution to edge weight, so a low-confidence extraction produces a correspondingly smaller evidential contribution.

Explicit reporting of absence. Run results carry warnings naming how many abstracts were unavailable, how many were recovered from a secondary provider, and how many duplicate records were merged. A sparse graph can therefore be explained, few citations, or records merged, or lookups failed, rather than leaving the user to guess.

The generated Markdown report states plainly when no population evidence was found and what that implies for the weights, rather than presenting an unweighted ranking as though it were evidence-weighted.

Chapter 4 (continued): Algorithm Specifications

This chapter states the five core algorithms precisely enough to be reimplemented. Each is given as pseudocode with its complexity, its failure modes, and a note on the design decisions that are not obvious from the code.

4.8 Algorithm 1: metadata merge

4.8.1 Problem

Three providers return partially overlapping, sometimes conflicting records for one work. Produce a single record, preferring the more reliable source per field, and retain enough provenance that a disagreement can be investigated.

4.8.2 Specification

```
INPUT    results: map from provider name to ProviderResult
OUTPUT   Paper

1  papers <- { name: r.paper  for name, r in results  if r.paper
is not null }
2  if papers is empty then
3      raise ResolutionError("no provider returned a record")
4
5  # Canonical identity: DOI first, since it is the identifier
all three share.
6  primary_doi      <- first non-null doi      over papers
7  primary_pmid     <- first non-null pmid     over papers
8  primary_openalex <- first non-null openalex_id over papers
9  paper_id <- primary_doi or primary_pmid or primary_openalex
10
11 # Field-level precedence. Crossref carries publisher-
deposited
12 # bibliographic metadata; OpenAlex reconstructs abstracts.
13 title    <- first non-null title    over [crossref, openalex,
europe_pmc]
14 year     <- first non-null year     over [crossref, openalex,
europe_pmc]
15 authors  <- first non-empty authors over [crossref, openalex,
europe_pmc]
16 journal  <- first non-null journal  over [crossref, openalex,
europe_pmc]
17 abstract <- first non-null abstract over [openalex,
europe_pmc, crossref]
18
19 provenance <- { name: p.provenance[name]  for name, p in
papers }
20 source_ids <- union of p.source_ids over papers
```

```

21 confidence <- 0.95 if |papers| > 1 else 0.80
22
23 return Paper(paper_id, primary_doi, primary_pmid,
primary_openalex,
24               title, authors, year, journal, abstract,
25               source_ids, confidence, provenance)

```

4.8.3 Complexity and notes

$O(p \cdot f)$ for p providers and f fields; $p = 3$ and f is fixed, so the merge is effectively constant time. The cost of resolution is entirely the network I/O that precedes it, which is why the three provider queries are issued concurrently (Section 4.3.1).

The confidence heuristic at line 21, 0.95 when more than one provider returned a record, 0.80 otherwise, is agreement-as-confidence, and it is weak. It rewards two providers *returning* a record, not two providers *agreeing* on its contents. A stronger formulation would compare the fields themselves and reduce confidence on disagreement. This is not implemented.

4.9 Algorithm 2: level-synchronous citation traversal

4.9.1 Problem

Expand outward from a seed paper in both directions to bounded depth, under a bounded total number of papers, collapsing multiple identifiers for one work onto a single node, and emitting no edge whose endpoints are not both present.

4.9.2 Specification

INPUT seed, backward_depth, forward_depth, max_papers
OUTPUT papers: map paper_id → Paper, edges: list of
CitationEdge

```

1  register(seed)                                # stores paper + all its
aliases
2  frontier[backward] <- [seed.id] if backward_depth > 0 else []
3  frontier[forward]  <- [seed.id] if forward_depth  > 0 else []
4  budget <- allocate(backward_depth, forward_depth, max_papers)
5  spent  <- { backward: 0, forward: 0 }
6
7  for depth in 0 .. max(backward_depth, forward_depth) - 1 do
8      for direction in [backward, forward] do
9          if depth >= depth_limit[direction] or
frontier[direction] empty then
10             frontier[direction] <- []; continue
11             if |papers| >= max_papers then
12                 frontier[direction] <- []; continue
13
14             # Release the other direction's reservation once it
is exhausted,
15             # so a paper with no citing works still gets a full
reference tree.
16             other <- the opposite direction

```

```

17         if frontier[other] non-empty and depth <
depth_limit[other] then
18             allowance <- budget[direction] - spent[direction]
19         else
20             allowance <- max_papers - |papers|
21         if allowance <= 0 then frontier[direction] <- [];
continue
22
23         limit <- min(max_papers, |papers| + allowance)
24         before <- |papers|
25         frontier[direction] <- EXPAND(frontier[direction],
direction, limit)
26         spent[direction] <- spent[direction] + (|papers| -
before)
27
28     return papers, edges

```

EXPAND is where the batching lives:

```

EXPAND(frontier, direction, limit):
1  # One request per frontier paper, issued concurrently
(semaphore = 5).
2  raw_edges <- concurrent_fetch(frontier, direction)
3  if raw_edges empty then return []
4
5  # Collect every neighbour identifier not already resolved.
6  pending <- []
7  for edge in raw_edges do
8      n <- edge.target if direction = backward else edge.source
9      if canonical(n) is null and n not in unresolvable then
10         append canonicalise(n) to pending
11
12  added <- []
13  if pending non-empty and |papers| < limit then
14      added <- RESOLVE_BATCH(pending, limit)
15
16  COMMIT_EDGES(raw_edges, direction)
17  return added

```

```

RESOLVE_BATCH(pending, limit):
1  partition pending into work_ids (W...), dois (10....), others
2
3  # 50 identifiers per request instead of one request per
paper.
4  if work_ids non-empty then
5      for paper in openalex.batch_by_work_id(work_ids) do
ABSORB(paper)
6  if dois non-empty then
7      for paper in openalex.batch_by_doi(dois) do
ABSORB(paper)
8
9  # Only identifiers the batch endpoints missed reach the slow
path.

```

```

10 leftovers <- pending not resolved above
11 for paper in concurrent_resolve_full(leftovers) do
ABSORB(paper)
12
13 mark every still-unresolved identifier as unresolvable
14 return newly added paper_ids

ABSORB(raw_id, paper):
1  if paper.paper_id not already stored then
2      # Duplicate check runs BEFORE the budget check, so a
merged
3      # duplicate never consumes one of the max_papers slots.
4      dup <- duplicate_of(paper)          # title-prefix + year
agreement
5      if dup is not null then
6          alias(paper.paper_id -> dup); alias(raw_id -> dup)
7          duplicates_merged <- duplicates_merged + 1
8          return
9      if |papers| >= limit then return
10     register(paper); append paper.paper_id to added
11 alias(raw_id -> paper.paper_id)

COMMIT_EDGES(raw_edges, direction):
1  for edge in raw_edges do
2      s <- canonical(edge.source); t <- canonical(edge.target)
3      # An edge is emitted only if BOTH endpoints resolved.
This is what
4      # guarantees zero dangling edges in the output.
5      if s is null or t is null or s = t then continue
6      if (s, t) already emitted then continue
7      rewrite edge endpoints to (s, t); append to edges

```

4.9.3 Complexity

Let N be `max_papers` and F the mean out-degree per expanded paper.

- Edge fetches: one request per frontier paper, $O(N)$ requests in the worst case, issued with concurrency 5.
- Metadata fetches: $O(N / 50)$ batched requests: the decisive improvement. The pre-batching implementation issued $O(N)$ individual resolutions, each querying three providers, for $O(3N)$ requests.
- Edge dedup: $O(1)$ per edge via a hash set of committed pairs. The original implementation scanned the committed list linearly per edge, which is $O(E^2)$.

4.9.4 Failure modes handled

Failure	Handling
Provider returns an unparseable identifier	Recorded as unresolvable; not retried on later levels
One malformed record in a batch of 50	Per-record try/except; the other 49 survive (§5.4)
Same work under two identifiers	Alias table collapses them to one node

Failure

Same work under two DOIs with near-identical titles

A heavily cited seed

Handling

Title-prefix match with year agreement

Forward results are sorted by citation count and capped

4.10 Algorithm 3: population candidate extraction

4.10.1 Specification

INPUT `paper_id, text, section`

OUTPUT `list of PopulationCandidate`

```
1  if text is empty then return []
2  sentences <- split(text)           # spaCy if available,
else regex
3  candidates <- []
4
5  for sentence in sentences do
6      # Spans that must not be read as populations: years,
percentages,
7      # p-values, dosages. Computed per span, NOT per sentence
-- see 4.10.3.
8      ignore_spans <- { match.span()
9                          for pattern in IGNORE_PATTERNS
10                         for match in finditer(pattern,
sentence) }
11
12     for pat in POPULATION_PATTERNS do
13         for m in finditer(pat.regex, sentence, IGNORECASE) do
14             (ns, ne) <- m.span(1)           # span of the
captured number
15             if (ns, ne) overlaps any ignore_span then
continue
16             value <- int(strip_separators(m.group(1)))
17             if value <= 0 or value > 10_000_000 then continue
18             confidence <- min(1.0, pat.weight +
section_bonus(section))
19             append PopulationCandidate(value, m.group(0),
sentence,
20                                     section, pat.type, ns,
ne,
21                                     confidence) to
candidates
22
23  return candidates
```

4.10.2 Complexity

$O(S \cdot P \cdot L)$ for S sentences, P patterns (20) and sentence length L . Linear in text length for a fixed pattern set. Measured at roughly 200 ms per abstract (Section 6.6.1), which is slow for regular expressions and is dominated by spaCy sentence segmentation rather than by matching.

4.10.3 The span-versus-sentence decision

Lines 8–15 encode the single most consequential correction made to this algorithm. The original implementation evaluated the ignore patterns against the *whole sentence* and skipped every candidate in it on a match. Because IGNORE_PATTERNS contains `\b20\d{2}\b`, and clinical abstracts mention a year in most sentences, the rule discarded the very numbers it existed to protect. Section 5.5 gives the measurement. Evaluating per span preserves the intent, a year is never read as a population, without the collateral loss.

4.11 Algorithm 4: population resolution

4.11.1 Specification

INPUT paper_id, candidates
OUTPUT PopulationResolution

```
1  if candidates empty then
2      return Resolution(n_eff=null, confidence=0.0,
status="missing")
3
4  filtered <- [ c in candidates if c.confidence > 0.40 ]
5  if filtered empty then
6      return Resolution(n_eff=null, confidence=0.0,
status="missing")
7
8  # Score combines extraction confidence with the clinical
informativeness
9  # of the semantic type: a randomised total outranks an arm
size.
10 score(c) = c.confidence * (1 +
TYPE_PRIORITY[c.semantic_type] / 10)
11
12 best <- argmax(filtered, score)
13
14 # Ambiguity: a rival scoring within 10% but differing
materially in value
15 # signals an unclear abstract, not a system failure.
16 ambiguous <- exists c in filtered, c != best, such that
17             score(c) > 0.9 * score(best)
18             and |c.value - best.value| > 0.1 * best.value
19
20 return Resolution(n_eff=best.value,
semantic_type=best.semantic_type,
21                 confidence=best.confidence,
22                 status = "ambiguous" if ambiguous else
"resolved")
```

with type priorities: TOTAL_RANDOMIZED 10, TOTAL_ANALYZED 9, TOTAL_ENROLLED 8, SAMPLE_SIZE_GENERIC 7, ARM_SIZE 5, SCREENED 4, COMPLETERS 3, FOLLOWUP_COUNT 2, EVENT_COUNT 1, UNKNOWN_NUMERIC 0.

4.11.2 Known weakness

This algorithm produced the single value error in the evaluation (Section 6.4.2). For a case series of 138 patients whose abstract also reports many subgroup counts, the selection returned 36. The scoring rewards pattern confidence and type priority but has no notion of *which number the abstract is about*, a subgroup count matched by a high-priority pattern outranks the cohort total matched by a lower-priority one. Position in the abstract, and the relationship between competing values, are both unused signals.

4.12 Algorithm 5: bounded path ranking

4.12.1 Problem

Rank citation paths from the seed by evidential strength, returning the top k , without enumerating a combinatorial number of paths.

4.12.2 Specification

INPUT `graph, seed_id, top_n = 10, cutoff = 4`

OUTPUT `ranked list of at most top_n paths`

```
1  if seed_id not in graph then return []
2  heap <- empty min-heap ordered by score, capacity top_n
3  examined <- 0
4
5  # ONE depth-first traversal yields every simple path from the
seed.
6  # Calling all_simple_paths(seed, target) per target re-walks
the whole
7  # reachable subgraph once per node -- see 4.12.3.
8  stack <- [ [seed_id] ]
9  while stack non-empty do
10     path <- pop(stack)
11     if |path| > 1 then
12         examined <- examined + 1
13         if examined > MAX_PATHS_EXAMINED then
14             log truncation; break
15         score <- SCORE_PATH(path)
16         if |heap| < top_n then push(heap, score, path)
17         else if score > min(heap).score then
replace_min(heap, score, path)
18         if |path| - 1 >= cutoff then continue
19         for succ in successors(last(path)) do
20             if succ not in path then push(stack, path + [succ])
21
22 # Build result dictionaries only for the survivors, not for
every path.
23 return sorted(heap, by score desc)

SCORE_PATH(path) :
1  weights      <- [ edge_weight(path[i], path[i+1]) for i in
0..|path|-2 ]
```

```

2  confidences <- [ edge_conf(path[i], path[i+1])    for i in
0..|path|-2 ]
3  avg_weight  <- mean(weights)
4  path_conf   <- product(confidences)              # confidence
compounds along a path
5  depth_bonus <- 1 / sqrt(|path|)                  # shorter paths
preferred
6  return avg_weight * path_conf * depth_bonus

```

4.12.3 Complexity

The replacement is $O(P)$ in the number of simple paths within the cutoff, with $O(\text{top}_n)$ memory. The original was $O(V \cdot P)$: `all_simple_paths` was called once per target vertex, and each call re-explored the entire reachable subgraph to depth 4, yielding only the paths terminating at that target. It also materialised a result dictionary, including a title lookup per node, for every path before sorting and discarding all but ten.

Measured on layered synthetic graphs (Section 6.6.4), the redesign is $126\times$ to $434\times$ faster, with the gap widening as the graph grows, and returns an identical top-ten verified against the exhaustive computation.

The `MAX_PATHS_EXAMINED` cap at line 13 is a safety valve rather than an optimisation. A densely interlinked citation graph can contain an astronomical number of simple paths; without a bound, one pathological input hangs the run. When the cap triggers it is logged, so a truncated ranking is never presented as exhaustive.

Chapter 5: Implementation

This chapter describes the delivered implementation and, in Sections 5.3 to 5.8, the defects found when the system was systematically instrumented and measured. The defects are reported in full because they are the most transferable part of the work: each was invisible to a passing test suite, and each produced output that looked plausible.

5.1 Technology Stack

Layer	Choice	Rationale
Language	Python 3.10+	Ecosystem for NLP and graph analysis
API	FastAPI	Native async, automatic OpenAPI, Pydantic integration
Validation	Pydantic v2	Validation at construction rather than at use
HTTP	httpx (async)	Concurrent provider queries, connection pooling
Graph	NetworkX [hagberg2008networkx]	Mature PageRank and path algorithms
Persistence	SQLite +aiosqlite	Sufficient for run records; no server to operate
Retry	Tenacity	Declarative retry policy
Frontend	React, TypeScript, Vite, TanStack Router	n/a
Visualisation	Cytoscape.js	Interactive graph rendering
Testing	pytest, pytest-asyncio, respx	102 tests; respx mocks HTTP at transport level

NetworkX was chosen over a graph database because the working set is bounded at 200 nodes. At that scale an in-memory graph is faster than any database round trip, and Section 6.6 confirms the choice: all graph analytics together account for 2.5% of runtime. Neo4j appears in the configuration but is not implemented.

5.2 Pipeline Orchestration

The orchestrator executes nine stages in sequence: normalise input, resolve seed, traverse citations, backfill abstracts, extract populations, resolve populations, weight edges, build graph, run analytics. Each stage is a separate module; the orchestrator holds no domain logic beyond sequencing and the assembly of warnings.

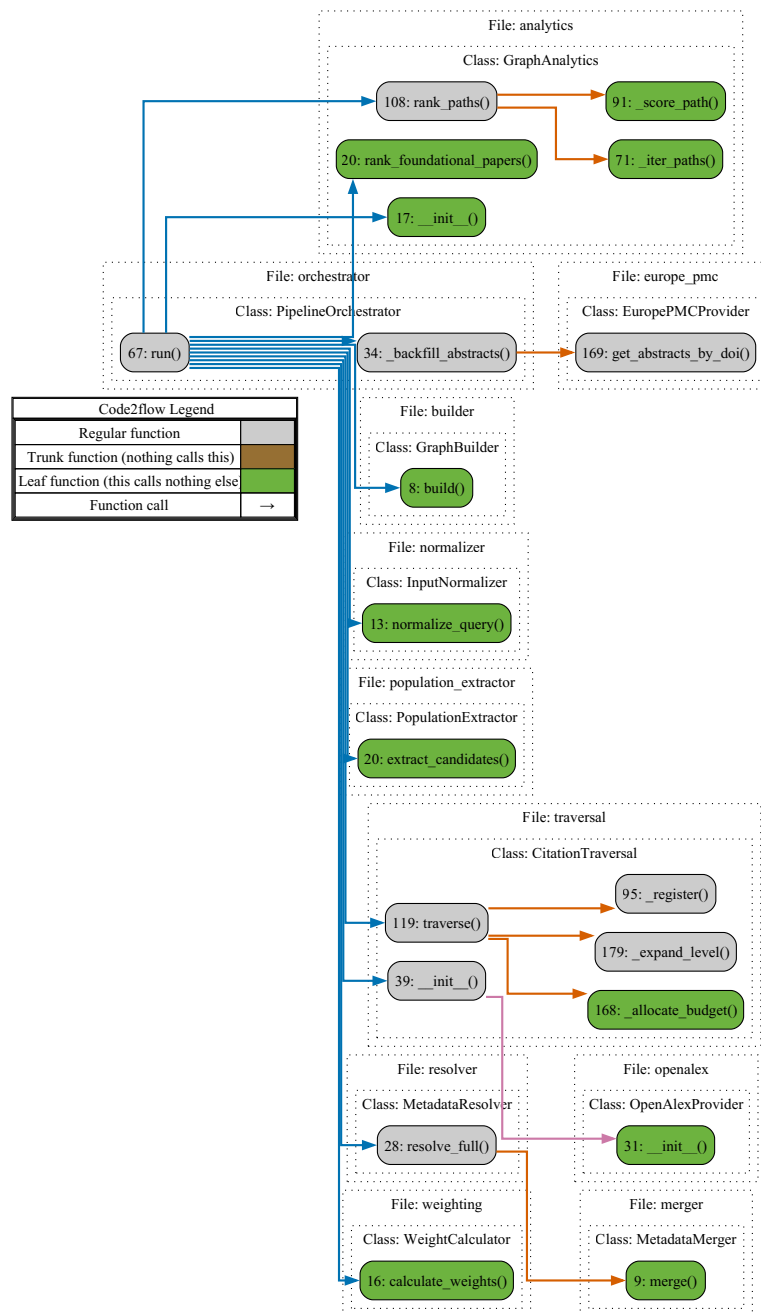


Figure 5.1 Calls made by `PipelineOrchestrator.run()`, traced by `code2flow` over the source and cut two levels below the entry point. Node labels carry the line number the function is defined at, so the figure doubles as an index into the source. Dotted boxes are files and classes, and green nodes call nothing further within the project. `run()` sits at the left, the stages it calls form the middle column, and the functions those stages call in turn sit at the right.

Reading the figure against the stage list above shows one structural property worth stating: `run()` calls each stage directly and no stage calls another. Sequencing lives in one function, so a stage can be reordered or removed by editing `run()` alone. The single exception is `_backfill_abstracts()`, which `run()` delegates to and which in turn calls the Europe PMC provider; it was added as a separate method rather than inline because it is the one stage that is skipped entirely when every abstract is already present.

Runs execute as background tasks. A `POST` returns a run identifier immediately, and the client polls. This is necessary because runs take tens of seconds to minutes, Section 6.6 characterises the distribution, which far exceeds a reasonable HTTP timeout.

5.3 Defect 1: traversal discarding the majority of every graph

5.3.1 Symptom

The system produced graphs containing far fewer nodes than expected. For some inputs it produced a graph containing only the seed paper. This was the presenting symptom that prompted the measurement campaign.

5.3.2 Diagnosis

OpenAlex returns references as OpenAlex work identifiers (W2741809807), not DOIs. The traversal inferred a query type from the identifier's shape:

```
if target_id.startswith("10."):    query_type = "doi"
elif target_id.isdigit():          query_type = "pmid"
else:                              query_type = "doi"    # W-ids
    land here
```

A work identifier fell to the final branch and was validated against the DOI regular expression, which rejected it. The resulting exception was caught by a broad handler and logged at warning level, so the paper was silently dropped.

Measurement on a representative seed:

```
target identifier kinds : {'W-id': 21, 'doi': 19}
constructible as a query: 19      REJECTED: 21
```

Every OpenAlex-sourced reference was discarded. Only Crossref's DOI-formatted references survived. Where Crossref holds no reference list, which is common, since deposit is at publisher discretion, the result was a graph of one node:

```
references for PMID 32109013 : OpenAlex 21, Crossref 0  ->
usable nodes 0
```

5.3.3 A second, independent defect in the same component

Forward citations were always empty. The OpenAlex `cites` filter accepts work identifiers only, but the code passed a DOI-prefixed form:

```
filter=cites:doi:10.1056/nejmoa2002032    -> HTTP 200, meta.count
= 0
filter=cites:W3008827533                  -> HTTP 200, meta.count
= 30606
```

This is the more instructive of the two faults. The malformed query returned **HTTP 200 with a count of zero** rather than an error. No exception was raised, no log line was emitted, and the system reported "this paper has no citing works", a statement that is occasionally true and therefore unremarkable.

5.3.4 Resolution

The traversal was rewritten as the level-synchronous BFS described in Section 4.4, with an alias table collapsing all identifier forms onto one canonical `paper_id`, and metadata fetched in batches of fifty through the OpenAlex `openalex_id` and `doi` filters. Forward citation queries resolve the seed to a work identifier before filtering.

Measure	Before	After
Forward citations (seed)	0	34
Edges with a resolved endpoint	262 with dangling	163, none dangling
Runtime, 100-paper run	273 s	55 s

5.4 Defect 2: a single null field discarding batches of fifty

5.4.1 Symptom

After the traversal rewrite, one test paper still returned zero forward citations although OpenAlex reported 53 citing works.

5.4.2 Diagnosis

The mapping function read the venue as:

```
data.get("primary_location", {}).get("source",  
    {}).get("display_name")
```

OpenAlex returns `"primary_location": {"source": null}` for works with no indexed venue. `dict.get(key, {})` returns the default only when the key is *absent*; when the key is present with a null value it returns `None`, and the chained call raises `AttributeError`.

Because the exception was caught around the *whole batch*, one malformed record discarded the other forty-nine:

```
fetches 50 citing works  
  primary_location.source is null : 23  
  mapping each work individually : ok=27  FAILED=23  
  -> one bad record aborts the whole batch of 50
```

Twenty-three of fifty records had a null venue. This is not an edge case.

5.4.3 Resolution

Null-safe accessors, and a per-record `try/except` inside the batch loop so a malformed record is skipped and logged rather than aborting its neighbours. The same defect shape was found and fixed in the Crossref and Europe PMC mappers, where it had not yet been triggered.

A secondary effect was discovered at the same time: because batch failures forced a fallback to per-paper resolution, the run had been taking 209 s. After the fix the same run took 12 s.

5.5 Defect 3: ignore rules discarding the numbers they protect

5.5.1 Symptom

Only 15% of papers in a typical graph received any population evidence.

5.5.2 Diagnosis

The extractor applied its ignore patterns, years, percentages, p-values, dosages, at *sentence* granularity:

```
for ignore_p in IGNORE_PATTERNS:
    if re.search(ignore_p, sentence):
        should_skip_sentence = True    # discards the entire
        sentence
```

IGNORE_PATTERNS includes `\b20\d{2}\b`, matching any year. Clinical abstracts mention a year or a percentage in most sentences. The rule intended to prevent "2019" being read as a sample size was discarding the sample sizes themselves:

```
DROPPED PATIENTS_COUNT=1099 ('1099 patients')
        because the sentence also matched '\b20\d{2}\b'
```

The sentence was *"We extracted data regarding 1099 patients with laboratory-confirmed Covid-19 from 552 hospitals ... through January 29, 2020."* The pattern matched the cohort size correctly and the extractor then threw it away because of the date at the end of the sentence.

5.5.3 Resolution

Ignore patterns are now matched per *span*. A candidate is rejected only when the captured number itself falls inside an ignored span. A year is still never read as a population; a count standing beside a year survives.

Separately, the pattern set assumed randomised-trial phrasing. "We analyzed data on the first 425 confirmed cases" matched nothing, because no pattern covered *cases*. Patterns were added for the phrasings observational papers use, *a total of N*, *N confirmed cases*, *N subjects*, *N consecutive patients*, *included/recruited/studied N*, *data on N*, along with patterns for SCREENED, COMPLETERS, ARM_SIZE and FOLLOWUP_COUNT, four semantic types the model defined but which no pattern could previously emit. | Measure (40-paper graph) | Before | After |
|-----:|-----:| | Papers with extracted evidence | 7 | 15 | | Of those with an
abstract | 7/29 | 15/29 | | High confidence (≥ 0.8) | 3 | 10 |

5.6 Defect 4: edge weights collapsing to zero

5.6.1 Symptom

None. This defect produced no visible symptom, which is why it is the most instructive in the catalogue.

5.6.2 Diagnosis

The original weighting multiplied the entire base weight by population confidence. A resolution with status missing carries confidence 0.0 by definition, so:

```
resolution with no candidates -> n_eff=None confidence=0.0
P1->P2 base=0.125 FINAL=0.0
P1->P3 base=0.125 FINAL=0.0
P2->P3 base=0.125 FINAL=0.0
```

Every edge weight was exactly zero for any paper where extraction found nothing, which, before Defect 3 was fixed, was 85% of papers.

The ranking nonetheless produced sensible output, and this is why the fault survived. The graph builder contained:

```
weight=edge.final_weight or 1.0
```

In Python, 0.0 or 1.0 evaluates to 1.0. Every zeroed weight was silently replaced by 1.0, and PageRank ran **unweighted**. The system's central claim, evidence-weighted citation analysis, was not operating, and the fallback that concealed it was an accident of truthiness rather than a designed behaviour.

5.6.3 Resolution

Confidence now scales the evidential term only, as in Section 4.5. The builder falls back only on a genuine None, so a real zero can no longer be disguised.

Case	n_eff	confidence	final weight
No evidence	n/a	0.00	0.125
Small trial	120	0.85	0.391
Large trial	8,500	0.90	0.655
Very large	100,000	0.95	0.837
Large, low confidence	8,500	0.30	0.302

Evidence still outranks absence, larger samples outrank smaller, and low confidence is penalised, but nothing collapses to zero.

5.7 Defect 5: server-side request forgery in URL input

Accepting an article URL requires fetching it when no identifier can be parsed from the URL text. The implementation fetched any URL that had a scheme and a host, with redirects followed automatically.

Verified against a local listener:

```
PaperQuery ACCEPTS http://127.0.0.1:9911/internal/admin
PaperQuery ACCEPTS http://169.254.169.254/latest/meta-data/
listener received 1 request
resolver returned: query_type='title' value='Internal Service
Banner v2.4'
```

The system could be directed at loopback, link-local (cloud metadata) or private addresses, and the fetched page's `citation_title` was forwarded to Crossref and Europe PMC as a search term, a narrow but real exfiltration channel.

The resolver now rejects any URL whose host resolves to a private, loopback, link-local, reserved, multicast or unspecified address, and follows redirects manually so every hop is re-validated. A public host that redirects to an internal one is refused at the second hop.

A separate URL defect was found in the same component: the DOI character class includes `/`, so matching against a URL path ran past the end of the DOI into the publisher's view segment, yielding `10.3389/fcomp.2024.1387354/full` and failing the run. Trailing view segments are now trimmed.

5.8 Defect 6: exports returning JSON envelopes

The frontend saved export responses directly to disk as `citegraph-<id>.csv`, but the backend returned `{"csv": "..."}.` Users received a `.csv` file containing a JSON envelope with the whole table escaped onto one line.

All exports now return the file itself with the correct `Content-Type` and a `Content-Disposition` filename. Both CSV exports are written with the `csv` module, so a comma, quote or newline in a title or journal can no longer break a row, the previous hand-rolled formatting never escaped the journal field at all, and carry a UTF-8 byte-order mark so spreadsheet software renders accented author names correctly. An edge-list export and a GraphML export were added; the frontend had offered GraphML as a format although no such route existed.

5.9 Deployment

The backend runs as a Docker container bound to the loopback interface on a shared host, behind nginx with a Let's Encrypt certificate. The frontend is a static single-page application on a content delivery network. Cross-origin access is restricted to the frontend origin; a wildcard policy was replaced because it would allow any page a developer visits to drive a locally running instance and read the responses.

An optional API-key gate exists but is unset in the current deployment, because a public single-page application cannot hold a secret. This is recorded as an open issue in Section 7.5 rather than presented as a completed control.

A defect in the deployment workflow is worth recording alongside the software defects: the container publication directive was `-p "${PORT}:8000"`, which binds `0.0.0.0` and would have exposed the API directly to the internet, bypassing nginx and TLS, on a host shared with three other projects. It now binds `127.0.0.1` explicitly and the deployment fails if the port is found on a public interface.

5.10 Discussion: why these defects survived

Six defects reached a system that passed its tests, and four of them produced output that looked correct. Three properties of the failures explain this.

They failed silently. The traversal defect logged at warning level and continued. The `cites` filter defect returned HTTP 200. The weighting defect was masked by a truthiness coincidence. None raised an error.

They produced plausible output. A graph with 19 nodes instead of 40 looks like a sparse literature, not a bug. A ranking from unweighted PageRank is still a ranking. Plausible-looking wrong output is harder to detect than a crash.

The tests asserted shape, not correctness. The suite verified that a run completed and returned a structure with the right fields. No test asserted that the number of nodes bore any relation to the number of references available, or that edge weights varied.

The methodological conclusion is that for a system whose output is a ranked list, correctness cannot be established by testing that the pipeline runs. It requires measurement against known quantities, which is what Chapter 6 reports, and what the project proposal had specified from the outset.

Chapter 5 (continued): Dashboard and Interaction Design

The analysis pipeline produces a graph, a ranking and a set of confidence-scored extractions. None of that is useful to a reader unless it can be interrogated. This chapter describes the dashboard, and in particular how the uncertainty described in Section 4.7 is represented in the interface.

5.11 Technology and Structure

Concern	Choice	Rationale
Framework	React 19 + TypeScript	Typed contracts against the API schema
Build	Vite	Fast rebuilds; static output deployable to a CDN
Routing	TanStack Router	File-based routes with typed parameters
Server state	TanStack Query	Polling, caching and retry for long-running runs
Styling	Tailwind CSS + shadcn/ui	Consistent primitives without a heavy component library
Graph rendering	Cytoscape.js	Handles a few hundred nodes interactively

The application is a client-rendered single-page application deployed as static files. It holds no secrets and performs no server-side rendering; every dynamic value comes from the API. This is what allows it to be served from a content delivery network with a catch-all rewrite to `index.html`.

5.11.1 Route structure

Route	Purpose
<code>/</code>	Landing page
<code>/start</code>	Submit a paper; configure depth and graph size
<code>/dashboard/\$runId</code>	Overview: summary counts, warnings, entry points
<code>/metadata/\$runId</code>	Resolved bibliographic records with provenance
<code>/population/\$runId</code>	Extracted populations with confidence and status
<code>/graph/\$runId</code>	Interactive citation graph
<code>/rankings/\$runId</code>	Ranked foundational papers
<code>/paths/\$runId</code>	Ranked citation paths from the seed
<code>/export/\$runId</code>	Downloads in five formats

The separation of `population` from `rankings` is deliberate. The ranking is the system's headline output, but the `population` evidence is what the ranking *rests on*, and a reader who wants to judge the ranking must be able to inspect the evidence independently of it.

5.12 The submission flow

`/start` accepts any of the five input types. Two decisions in this screen proved important.

Clamping is visible, not silent. The graph-size control is a 10–200 slider rather than a free-text field, so the API's clamping (Section 3.2.3) is expressed in the interface rather than applied invisibly to an out-of-range value the user typed. The default of 100 reflects the measurement in Section 6.6.2: 100 papers completes in about 31 s, while 200 takes around 3.5 minutes.

Runs are asynchronous and the wait is honest. Submission returns a run identifier immediately and the client polls. Because a run takes tens of seconds, the interface reports the elapsed time rather than an indeterminate spinner. An indeterminate spinner at 30 s reads as a hang.

5.13 Representing Uncertainty

This is where the interface does the work that distinguishes the system, and where Section 6.4.4 shows it currently overstates its case.

5.13.1 Status before number

Every extracted population is displayed with its status, `resolved`, `ambiguous` or `missing`, adjacent to the value, not in a tooltip or a detail panel. The three states are visually distinct. A number presented without its status invites the reader to treat an ambiguous extraction as a settled fact.

5.13.2 The supporting sentence is reachable

Selecting an extracted population reveals the sentence it came from. This is the single most useful affordance in the interface: a reader who doubts a number can verify it in one interaction, without leaving the application or opening the source paper. It is possible only because the extractor retains `sentence` on every candidate (Section 4.2.2).

5.13.3 Absence is reported, not implied

Run warnings appear on the dashboard overview rather than being logged server-side and discarded. A graph with few nodes is accompanied by the reason: how many abstracts were unavailable, how many were recovered from a secondary provider, how many duplicate records were merged. Without this, a sparse graph is ambiguous between "this paper has few citations" and "the system failed", and a user cannot distinguish them.

5.13.4 An honest limitation of the current presentation

The interface presents confidence as a number and, by placing it beside the extraction, implies it indicates reliability. Section 6.4.4 shows it does not: mean confidence was 0.91 when the extracted value was correct and 0.90 when it was wrong. A user filtering for high confidence would retain the errors.

This is a presentation problem as much as a modelling one. Until the score is calibrated (Section 8.2.1), displaying it as a precise quantity overstates what is known. A coarser presentation, or an explicit statement that the score reflects which pattern matched rather than probability of correctness, would be more truthful with the current model.

5.14 Graph Visualisation

The graph view renders nodes sized by extracted population and edges weighted by the final edge weight, with the seed paper distinguished.

Three constraints shaped it.

Bounded node count. At most 200 nodes, which Cytoscape handles interactively. A force-directed layout over an unbounded graph would neither render nor be readable, which is a second reason for the traversal budget.

Every link has both endpoints. The renderer would silently drop an edge whose endpoints are absent from the node set, so a graph with dangling edges displays as *fewer edges than the data contains* with no error. The traversal guarantee in Section 4.4.2 exists partly for this reason, and Section 6.5 verifies it: zero dangling edges across three runs.

Weights are visible. Edge thickness encodes the final weight. Before the defect in Section 5.6 was found, every weight was zero and every edge rendered identically, which, in retrospect, was an available visual signal that something was wrong, and one nobody read as such.

5.15 Export Design

Five formats, each with a distinct audience.

Format	Intended use
JSON	Programmatic reuse; complete result
CSV (papers)	Spreadsheet triage: one row per paper with rank, degree, evidence
CSV (edges)	The graph as a relation, with every weight component
Markdown	A readable report for inclusion in notes or a literature review
GraphML	Import into Gephi, yEd or Cytoscape Desktop for further analysis

Two exports exist because of things learned during evaluation. The **edge list** was added because a paper list cannot express a graph, and any external analysis of the weighting needs the components. **GraphML** was added because the frontend already advertised it as a format although no endpoint existed, the button returned 404.

The Markdown report states explicitly when no population evidence was found and what that means for the weights, rather than presenting an effectively unweighted ranking as though it were evidence-weighted. This is the same honesty requirement as Section 5.13.3, applied to the artifact a reader is most likely to circulate.

Chapter 6: Evaluation and Results

All results in this chapter were produced by `scripts/run_evaluation.py`, committed to the project repository. Appendix C gives the commands. The run reported here was generated on 2026-09-18. Every figure can be recomputed by a reader with network access; because the underlying scholarly APIs change over time, exact reproduction is not guaranteed and this is discussed in Section 7.3.

6.1 Gold Standard

6.1.1 Composition

Twenty papers, annotated by reading the abstract retrieved through the same providers the pipeline uses.

Design	Papers Label
Randomised controlled trial	5 positive
Cohort study	2 positive
Case series	2 positive
Epidemiological analysis	1 positive
Modelling study	1 positive
Positives	11
Review article	3 negative
Clinical guideline	1 negative
Virus characterisation	2 negative
Computational (no human subjects)	2 negative
Negatives	9

Positive sample sizes span 41 to 43,548 subjects, three orders of magnitude, which is the range the logarithmic normalisation in Section 4.5 is designed for.

6.1.2 Hard negatives

Four negatives contain large, salient numbers that are *not* study populations:

Paper	Distractor	Why it is not the answer
Zhou et al., <i>Nature</i> 2020	"2,794 laboratory-confirmed infections"	Epidemic tally, not this study's sample
Wynants et al., <i>BMJ</i> 2020	counts of reviewed studies	A systematic review; units are studies
Jumper et al., <i>Nature</i> 2021	CASP14 target counts	Protein targets, not human subjects
Krizhevsky et al., <i>CACM</i> 2017	"1.2 million high-resolution images"	Dataset size, not a study population

A gold set of positives only cannot measure false positives. Since a confident wrong number damages user trust more than a missing one, these cases are the most informative in the set.

6.1.3 Annotation protocol and its limits

Each label records the effective population, the total human subjects the primary analysis rests on, as stated in the abstract, with the supporting sentence quoted verbatim so the label is auditable. Where no human study population is stated, the label is negative and correct behaviour is to extract nothing.

The limitations stated in Section 3.5.4 apply throughout: single annotator who is also a system author, no inter-annotator agreement, twenty papers, abstracts only, biomedical concentration.

6.2 Metadata Resolution

Metric	Result
Papers attempted	20
Papers resolved	20 (100%)
DOI exact match	20 (100%)
Title present	100%
Year present	100%
Journal present	100%
Authors present	100%
Abstract present <i>after backfill</i>	100%
Abstracts recovered from Europe PMC	4

Resolution and field completeness are perfect on this set. Two qualifications prevent over-reading it. The gold papers are well-indexed works from major journals, which is the easy case; papers from small venues, older literature and non-English publications are not represented. And the 100% abstract figure is *post-backfill*: four of twenty papers (20%) had no abstract from the primary resolution path and were recovered only because a second provider was queried.

6.3 Abstract Availability

Because extraction requires text, abstract availability bounds everything downstream. Measured on a 40-paper traversal graph:

Measure	Value
Papers in graph	40
No abstract in OpenAlex	11 (28%)
Recoverable from Europe PMC	11 of 11 (100%)
Recoverable from Crossref	0 of 11
Remaining without abstract after backfill	0

This is the single most consequential measurement in the evaluation, and it answers **RQ2**. Before the backfill, 28% of papers could not be evaluated at all, not because extraction failed, but because there was nothing to read. Every one of those abstracts existed in Europe PMC, and none in Crossref.

Two conclusions follow. First, a pipeline drawing abstracts from a single provider inherits that provider's coverage gaps as an invisible ceiling on recall. Second, the fix is cheap: an OR-joined Europe PMC query returned all eleven abstracts in **one request in 0.81 s**, so the backfill costs one additional API call per run.

6.4 Population Extraction

6.4.1 Detection

Detection is the binary decision: does this paper report a study population?

	Predicted positive	Predicted negative
Actually positive	TP = 11	FN = 0
Actually negative	FP = 1	TN = 8
Metric	Value	95% CI
Precision	0.917	n/a
Recall	1.000	n/a
Specificity	0.889	n/a
F1	0.957	n/a
Accuracy	0.950	[0.764, 0.991]

Recall is perfect on this set: every paper that reports a population had one extracted. The confidence interval on accuracy is wide, [0.764, 0.991], and must be quoted with the point estimate. A recall of 1.000 on eleven positives is statistically consistent with a true recall substantially below 1.

This answers the first half of **RQ1**: pattern-based extraction detects the presence of a reported population reliably enough to drive edge weighting, with the sample-size caveat above.

6.4.2 Value accuracy

Metric	Value	95% CI
Exact value correct	10 / 11 (0.909)	[0.623, 0.984]

The single failure is instructive. For Wang et al., *JAMA* 2020, a case series of 138 hospitalised patients, the system extracted **36**. The abstract states the cohort size in the Design section and then reports numerous subgroup counts in Results; the resolver selected a subgroup. This is a *resolution* failure, not a detection failure: the correct candidate was extracted, and the selection policy in Section 4.2.2 preferred another.

6.4.3 Semantic type classification

Metric	Value
Type correct, among correct values	6 / 10 (0.600)

This is the weakest measured result, and it answers the second half of **RQ1** negatively: surface patterns do not reliably distinguish the clinical categories. The four errors:

Paper	Gold	Predicted
Polack et al. 2020	TOTAL_RANDOMIZED	SAMPLE_SIZE_GENERIC
Folegatti et al. 2020	TOTAL_ENROLLED	TOTAL_RANDOMIZED
Huang et al. 2020	TOTAL_ANALYZED	TOTAL_ENROLLED
Verity et al. 2020	SAMPLE_SIZE_GENERIC	TOTAL_ANALYZED

All four are *adjacent-category* confusions among quantities that are often numerically equal, a trial that randomises 4,744 and analyses 4,744 differs only in framing. The first error has a traceable cause: the phrase "A total of 43,548 participants underwent randomization" matches a generic a total of N pattern that is typed SAMPLE_SIZE_GENERIC, and that pattern outranked the randomisation-specific one. This is a pattern-priority defect, not an inherent limit.

The practical impact on the system is smaller than the number suggests: the weighting formula uses the *value*, not the type. Type accuracy affects the displayed explanation rather than the ranking. It is nonetheless the clearest argument for the supervised approach discussed in Section 8.2.

6.4.4 The false positive, and confidence calibration

The one false positive is Wynants et al., *BMJ* 2020, a systematic review of prediction models. The system extracted **27** with confidence **0.90**.

This exposes a calibration failure that matters more than the single error:

Mean confidence	
When the extracted value was correct	0.91
When the extracted value was wrong	0.90

Confidence does not discriminate correct from incorrect extractions. This is a genuine weakness in a system whose stated purpose is to surface uncertainty. Confidence currently reflects *which pattern matched*, not *how likely the match is to be right*, and a user filtering on high confidence would retain the errors along with the correct results. Section 8.2 proposes calibration against held-out data as the remedy.

6.5 Citation graph construction

Three traversals at max_papers = 40, backward depth 2, forward depth 1:

Seed	Nodes	Edges	Dangling	Isolated	Merged	Mean degree	Time
Guan et al. 2020 (COVID cohort)	40	81	0	0	2	4.05	19.9 s
Jumper et al. 2021 (AlphaFold)	40	39	0	0	0	1.95	10.7 s
McMurray et al. 2019 (DAPA-HF)	40	90	0	0	0	4.50	26.3 s

Aggregate	Value
Runs completed	3 / 3
Total dangling edges	0
Total isolated nodes	0
Connected fraction	1.000
Duplicate records merged	2
Mean runtime	19.0 s

Every run filled its budget with distinct papers, every edge had both endpoints present, and no node was orphaned. Duplicate merging occurred before the budget was consumed, so the two merges on the first graph did not reduce it below 40, which matters for interpretation: a sparse graph indicates few citations, not records lost to deduplication.

Mean degree varies by more than a factor of two across seeds. The AlphaFold graph is markedly sparser (1.95) than the two clinical graphs (4.05, 4.50), reflecting that its neighbourhood spans structural biology, machine learning and chemistry, which cite each other less densely than a tight clinical literature does.

On ranking quality (RQ3). A qualitative observation: seeded with the 2021 AlphaFold paper, the top-ranked foundational paper was Anfinsen's 1973 *"Principles that Govern the Folding of Protein Chains"*, the work that founded the protein-folding problem. This is the behaviour the design intends. It is an anecdote, not a measurement. No relevance judgement study was conducted, no comparison against unweighted PageRank was run, and a single favourable example does not establish that evidence weighting improves ranking. Section 7.3 records this as the principal unaddressed question.

6.6 Performance

6.6.1 Stage breakdown

Instrumented 40-paper run:

Stage	Time	Share
Resolve seed (3 providers in parallel)	1.8 s	6.5%
Citation traversal (network-bound)	17.5 s	62.6%
Population extraction (local)	7.9 s	28.3%
Edge weighting	0.00 s	0.0%
Graph construction	0.00 s	0.0%
PageRank ranking	0.7 s	2.5%
Path ranking	0.00 s	0.0%
Total	27.9 s	

This answers **RQ4**. The system is I/O-bound: 58 HTTP requests across three providers dominate, and all graph analytics together are 2.5% of runtime. The graph algorithms are not the bottleneck and, at this scale, never will be, which retrospectively validates the choice of an in-memory graph over a database (Section 5.1). Population extraction at 28% is now the second cost and the first place local optimisation would pay.

6.6.2 Scaling with graph size

max_total_papers Runtime

60	30 s
100	31 s
200	209 s

Growth from 60 to 100 is nearly free because batching amortises the additional lookups. The jump to 200 is disproportionate: the graph becomes substantially denser, so depth-2 expansion covers many more nodes. NFR-1 (under 60 s) holds at 100 papers and fails at 200.

6.6.3 Effect of connection pooling

Each provider request originally created a fresh HTTP client, paying a full TCP and TLS handshake:

Mode	Median Total (8 sequential requests)	
New client per request	1.03 s	8.72 s
One pooled client	0.42 s	3.65 s

Approximately 634 ms of handshake per request, 58% of the time each call took. On a full run with identical output: | Before | After | |---|---|---:| | Mean request latency | 2,893 ms | 573 ms | | 40-paper run | 93.3 s | 27.9 s |

6.6.4 Path ranking complexity

The original path ranking called `all_simple_paths` once per target node, re-walking the reachable subgraph for every node in the graph, and materialised a result dictionary for every path before discarding all but ten. Measured on synthetic layered graphs:

Nodes	Edges	Before	After	Speed-up
61	820	3.53 s	0.03 s	126×
91	1,830	11.92 s	0.06 s	191×
151	5,050	117.96 s	0.27 s	434×

The redesign, one depth-limited DFS feeding a bounded heap, returns an identical top-ten (verified by comparing path sets and scores against the exhaustive computation). The speed-up widens with size because the original was $O(\text{nodes} \times \text{paths})$ and the replacement is $O(\text{paths})$.

6.7 Methodological note: reference verification

The reference list was verified programmatically: every candidate DOI was resolved against Crossref, with OpenAlex as fallback, and the returned title compared against the intended work.

This caught three identifiers that had been assigned incorrectly:

Key	Assigned DOI	What it actually is
Europe PMC	10.1093/nar/gkz947	<i>Expression Atlas update</i>
PageRank on citations	10.1016/j.joi.2007.01.001	A journal-impact-factor rank-order paper

Key	Assigned DOI	What it actually is
Trialstreamer	10.1093/jamia/ocv044	RobotReviewer

A fourth candidate, the 1999 PageRank technical report, has no resolvable DOI and was dropped rather than cited with a fabricated identifier; PageRank is cited through Brin and Page (1998) and Chen et al. (2007) instead.

This is reported because it is a result about method. Citations composed from memory are wrong at a non-trivial rate, three of twenty-nine here, roughly 10%, and the errors are invisible without verification, since a plausible DOI looks exactly like a correct one. All twenty-nine entries in the final bibliography resolve.

6.8 Summary of results against objectives

Objective	Status	Evidence
1. Identifier normalisation	Met	§6.2, automated tests
2. Multi-provider metadata resolution	Met	20/20 resolved, §6.2
3. Bidirectional bounded traversal	Met	§6.5
4. Population candidate extraction	Met	P 0.917 / R 1.000, §6.4
5. Candidate resolution with confidence	Partially met	Value 10/11; confidence uncalibrated , §6.4.4
6. Evidence-based edge weighting	Met	§5.6
7. Foundational ranking	Met, unvalidated	Produced; no relevance study, §6.5
8. API and dashboard	Met	§5.9
9. Module-level evaluation	Met	This chapter

Two objectives are qualified rather than met outright, and the qualifications are the honest findings of the evaluation: confidence scores do not discriminate correct from incorrect extractions (6.4.4), and the ranking, while producing defensible output on inspection, has not been validated against human judgement (6.5).

Chapter 6 (continued): Extended Analysis

6.9 Performance by study design

Aggregate metrics conceal where a system succeeds and fails. Breaking the gold-standard results down by study design shows that performance is not uniform, and that the failures cluster.

Design	n	Positives	Correct decision	Exact value	Mean confidence
Randomised controlled trial	5	5	5/5	5/5	0.96
Cohort study	2	2	2/2	2/2	0.90
Case series	2	2	2/2	1/2	0.85
Epidemiological	1	1	1/1	1/1	0.85
Modelling	1	1	1/1	1/1	0.85
Review	3	0	3/3	n/a	0.00
Virus characterisation	2	0	2/2	n/a	0.00
Computational	2	0	2/2	n/a	0.00
Guideline	1	0	1/1	n/a	0.00
Systematic review	1	0	0/1	n/a	0.90

Gold populations span 41 to 43,548 subjects, median 1,099.

Three patterns are visible.

Randomised trials are the easy case, and the system is built for them. Five of five correct, with the highest mean confidence (0.96). RCT abstracts follow a reporting convention that surface patterns capture well: a single headline number introduced by *randomised*, *assigned* or *enrolled*. This is unsurprising given that the pattern set was written with trial phrasing in mind, and it should temper any generalisation from the headline F1.

Clear negatives are handled cleanly. Reviews, guidelines, virus characterisation and computational papers were all correctly rejected, with confidence 0.00, including the hard negatives containing an ImageNet dataset size and an epidemic case tally. The ignore rules and the requirement that a number appear in a population-bearing construction are doing real work here.

The two failures are both structural, not lexical. The case-series miss (138 predicted as 36) and the systematic-review false positive (27 at confidence 0.90) share a cause: both abstracts contain *many* numbers in population-like constructions, and the system has no mechanism for deciding which one the abstract is *about*. In a case series, subgroup counts compete with the cohort total; in a systematic review, counts of included studies compete with nothing at all, because there is no cohort.

The systematic-review row is the most informative line in the table. It is the only design where the system was wrong *and* confident, and it is the design whose abstracts most resemble clinical reports without describing a cohort.

6.10 The confidence calibration failure in detail

Section 6.4.4 reported that mean confidence was 0.91 when extraction was correct and 0.90 when it was wrong. The design breakdown explains why: there is no model behind the score at all, so there is nothing that could have been fitted well or badly.

Confidence is computed as:

```
confidence = min(1.0, pattern_weight + section_bonus)
```

`pattern_weight` is a constant attached to each regular expression, chosen by the developer; `section_bonus` is 0.1 when the text came from an abstract. The score is therefore a property of *which pattern fired*, fixed before any data was seen. It cannot vary with evidence, because nothing in its computation depends on the input beyond pattern identity.

This has a consequence that the aggregate numbers hide. A pattern assigned weight 0.9 produces confidence 0.9 whether it matched the cohort total of a well-structured trial abstract or a count of reviewed studies in a systematic review. The systematic-review false positive carries confidence 0.90 for exactly this reason: it matched a high-weight pattern, and the system has no means of noticing that the match is out of domain.

The fix is not to adjust the weights. It is to make confidence a function of observed correctness, fitting a calibration model over features such as pattern identity, the number and spread of competing candidates, sentence position, and the presence of design-indicating terms, which requires labelled data at a scale the present gold standard does not reach. Section 8.2.1 and 8.2.3 are therefore coupled: calibration is blocked on corpus size.

Until then, the honest framing for a user is that the confidence score indicates *how specific the matched pattern was*, not *how likely the answer is to be right*. Section 5.13.4 notes that the interface does not currently make this distinction.

6.11 Sensitivity of the graph to seed choice

The three traversals in Section 6.5 produced markedly different graph structures from the same parameters.

Seed	Field	Nodes	Edges	Mean degree	Time
Guan et al. 2020	Clinical (COVID)	40	81	4.05	19.9 s
McMurray et al. 2019	Clinical (cardiology)	40	90	4.50	26.3 s
Jumper et al. 2021	Computational biology	40	39	1.95	10.7 s

Mean degree differs by a factor of 2.3 between the clinical graphs and the AlphaFold graph. Two explanations are consistent with the data, and the evaluation does not distinguish them.

The first is topical: the AlphaFold neighbourhood spans structural biology, machine learning and chemistry, and papers from different fields cite each other less densely than papers within a tight clinical literature do. The second is temporal: a 2021 paper's citing works are themselves recent and have had less time to accumulate the cross-citations that create density.

Distinguishing these would require holding field constant and varying year, or vice versa, across many more seeds than three. It is noted here because it bears on how a user should read a sparse graph: low density may indicate an interdisciplinary or recent seed rather than a thin literature, and the system currently offers no signal to tell them apart.

The runtime difference follows directly from density. Fewer edges means fewer neighbours to resolve, which reinforces the finding in Section 6.6.1 that cost is driven by the number of external lookups rather than by graph computation.

6.12 What the abstract backfill changed, stage by stage

The effect of the Europe PMC backfill is worth tracing through the pipeline, because it illustrates how an upstream data gap presents as a downstream model failure.

Stage	Before backfill	After backfill
Papers in graph	40	40
Papers with abstract text	29 (72%)	40 (100%)
Papers yielding candidates	15	20
Papers with a resolved <code>n_eff</code>	15 (38%)	20 (50%)
High confidence (≥ 0.8)	10	11
Status <code>missing</code>	25	20

Eleven papers moved from "no text" to "text available", and five of those then yielded a population. The remaining six had text but no reportable population. That is correct: they were reviews, guidelines and discovery papers.

The distinction matters for how the result is read. Before the backfill, 25 of 40 papers reported `missing`, and nothing in the output distinguished *no abstract available* from *abstract read, no population present*. Those are different facts about a paper, and only the second is a statement about the paper itself. The warnings described in Section 4.7 were added so that the output now separates them.

6.13 Cumulative effect of the corrections

Measured on the same 40-paper graph across the correction sequence in Chapter 5. Each row is the state after the named correction.

State	Papers with <code>n_eff</code>	High confidence	Runtime
Initial (as delivered before measurement)	7 (18%)	3	93.3 s
After per-span ignore matching (§5.5)	15 (38%)	10	n/a
After connection pooling (§6.6.3)	15	10	27.9 s
After Europe PMC backfill (§6.3)	20 (50%)	11	27.0 s

Coverage of extractable evidence rose from 18% to 50% of the graph, and high-confidence extractions from 3 to 11, while runtime fell by roughly 71%.

The composition of that gain is the point. Of the 13 additional papers with evidence, eight came from correcting extraction logic and five from querying a second data source. The second intervention is far cheaper, one additional API call per run against a rewrite of the ignore-

matching logic, and would not have been identified without separating "no text" from "extraction failed" in the measurement. Section 7.1.1 draws the general lesson.

Chapter 7: Discussion

7.1 Interpretation of Results

7.1.1 Extraction works better than the project expected; availability is the real ceiling

The headline extraction result, precision 0.917, recall 1.000, F1 0.957, is better than a hand-written pattern system might be expected to achieve, and the temptation is to attribute this to the patterns. The measurement in Section 6.3 argues otherwise. Before the abstract backfill, 28% of papers in a representative graph could not be evaluated at all: there was no text to read. Extraction accuracy on the papers that *had* text was never the binding constraint; the availability of text was.

This reframes what limits the system. On a 40-paper graph, the improvement from fixing the extraction logic (Section 5.5) raised coverage from 7 papers to 15; adding a second abstract provider raised it from 15 to 20. Data plumbing delivered as much as algorithmic work, and it is the cheaper of the two, one additional API call per run.

The generalisable observation is that in a pipeline drawing on external catalogues, coverage gaps in the upstream source present as *model* failures. A paper with no abstract produces no extraction, which is indistinguishable in the output from an extraction that failed. Without separating the two, effort goes to the wrong place.

7.1.2 Semantic typing is the genuine weakness

Type accuracy of 6/10 is the weakest result, and the errors are not random. All four are adjacent-category confusions, randomised versus enrolled versus analysed, among quantities that are frequently numerically identical. Distinguishing them requires reading the *role* the number plays in the study design, which is a semantic judgement rather than a lexical one. Surface patterns are the wrong instrument.

The impact on the system is bounded, because the weighting uses the value and not the type. But the aspiration in the proposal, to classify evidence by study role, is not achieved, and a supervised model trained on PICO annotations [nye2018ebmnlp] is the natural route to it.

7.1.3 The confidence scores do not mean what they appear to

Section 6.4.4 is, in the authors' view, the most important negative result. Mean confidence was 0.91 when the extracted value was correct and 0.90 when it was wrong. The score does not discriminate.

This matters disproportionately because surfacing uncertainty is the system's distinguishing claim. A user filtering for high-confidence extractions would retain the false positive, a systematic review misread at confidence 0.90, alongside the correct results. The confidence currently encodes *which pattern matched*, a property of the system, not *how likely this match is to be right*, a property of the world. Those are different quantities and the interface presents the first as though it were the second.

Nothing in the architecture prevents fixing this. Confidence would need to be calibrated against held-out labelled data, which requires a larger gold set than twenty papers.

7.1.4 The system is I/O-bound, and that is a design conclusion

Graph analytics account for 2.5% of runtime; network calls account for 63%. Two consequences follow for anyone building similar systems. The choice of graph library is nearly irrelevant at this scale, and effort spent optimising PageRank would be wasted. Conversely, HTTP-level engineering, connection pooling, request batching, avoiding per-item lookups, produced the largest single performance gain in the project: 93.3 s to 27.9 s, a 3.3× improvement from reusing one pooled client rather than creating one per request.

7.2 Reflections on the Development Process

The defect catalogue in Chapter 5 supports a conclusion the authors did not anticipate at the outset: **a passing test suite established very little about whether this system was correct.**

Six defects reached a system with 102 passing tests. Four produced no error and no visible symptom. The traversal defect discarded roughly half of every graph while the suite passed, because the tests asserted that a run completed and returned a structure of the right shape, not that the structure bore any relation to the available literature.

The defects that are hardest to find share a signature: they *degrade* rather than break. A graph with 19 nodes instead of 40 looks like a sparse literature. A malformed API filter returning HTTP 200 with zero results looks like a paper with no citing works. An edge weight of 0.0 silently rewritten to 1.0 by Python's `or` still yields a ranking. In each case the system produced a defensible-looking answer to the wrong question.

What surfaced then was not more testing but *measurement against known quantities*: comparing the number of nodes retrieved against the number of references the provider reported, comparing extracted populations against hand-read abstracts, timing each stage. Notably, the evaluation plan that would have caught most of these was specified in the project proposal from the start and left unimplemented, the empty `evaluation/` directory was itself the strongest early warning available, and nothing in the repository made its emptiness visible.

7.3 Threats to Validity

7.3.1 Construct validity

Sample size is a partial proxy for evidential weight. Section 2.5 is explicit that appraisal instruments treat size as one dimension among several [higgins2011cochrane]. A large poorly-conducted trial outranks a small rigorous one under this system's weighting. The system must not be presented as appraising quality, and the generated reports state this.

The ranking target is not operationally defined. "Probable foundational paper" has no agreed definition, so the ranking cannot be scored against a ground truth. The weights (0.5 / 0.3 / 0.2) were set by judgement, and no sensitivity analysis was performed.

7.3.2 Internal validity

Single annotator. The gold standard was labelled by one person who is also an author of the system. Knowledge of how the extractor works may have influenced label choices, particularly in ambiguous cases. No inter-annotator agreement was measured. This is the most serious methodological weakness in the evaluation.

Confidence is uncalibrated. Section 6.4.4.

Duplicate-merge precision is unmeasured. Merges are counted and survivors checked, but no gold standard of known duplicate pairs exists, so a merge that incorrectly collapsed two distinct papers would not be detected.

7.3.3 External validity

Sample size. Twenty papers, eleven positive. The confidence intervals, accuracy [0.764, 0.991], value accuracy [0.623, 0.984], are wide and are reported alongside every point estimate for this reason.

Domain concentration. Positives are biomedical, mostly COVID-era and cardiology. Performance on other literatures is unmeasured.

Temporal instability. The evaluation queries live APIs. OpenAlex and Crossref revise records continuously, so an exact rerun may differ. A frozen corpus would be more reproducible and is proposed in Section 8.2.

7.3.4 Conclusion validity

No comparative baseline. The central question, does evidence-weighted ranking outperform unweighted PageRank?, was **not tested**. No A/B comparison was run and no human relevance judgements were collected. The AlphaFold-to-Anfinsen result in Section 6.5 is a single favourable anecdote. This thesis therefore demonstrates that evidence-weighted ranking *can be computed* and *produces plausible output*, not that it is better. That is the most significant unaddressed question in the work.

7.4 Divergence from the Project Proposal

The proposal specified capabilities the delivered system does not have. They are listed here rather than omitted.

Specified	Delivered	Consequence
GROBID full-text PDF parsing	Configuration flags only; no implementation	Extraction is abstract-only. Sample sizes stated only in Methods are unreachable. This is the largest functional shortfall.
Neo4j study-aware graph store	NetworkX in-memory	No practical consequence at 200 nodes; §6.6 supports the choice
Study-aware knowledge graph	One-to-one paper→study mapping, dedupe_confidence hardcoded to 0.8	Papers reporting the same trial are counted as independent evidence
Streamlit dashboard	Not implemented; React SPA delivered instead	None, the React dashboard supersedes it
EVENT_COUNT, UNKNOWN_NUMERIC types	Defined in the model; no pattern emits either	Two documented categories can never appear
Journal quality term	Constant 0.5	The β term is a uniform floor, not a signal

Specified	Delivered	Consequence
Evaluation plan (§13)	Implemented during thesis preparation	Reported in Chapter 6

The study-aware graph is the most conceptually significant omission. The proposal's intent was that several papers reporting one trial, a protocol, a primary results paper, a secondary analysis, collapse to one study node, so the trial's evidence is counted once. The delivered system treats them as three independent papers, which overstates the evidential weight of well-published trials. The identity aliasing described in Section 4.4.2 solves the easier adjacent problem, duplicate *records* of one paper, not multiple papers of one study.

7.5 Known open issues

Issues present in the delivered system and not resolved:

1. **The deployed API is unauthenticated.** An optional API-key gate exists but is unset, because a public single-page application cannot hold a secret. Anyone can submit runs against the deployment.
2. **Run identifiers are not portable.** Run records live in the host's SQLite database, so a shared result link breaks if the backend moves.
3. **Confidence is uncalibrated** (§6.4.4).
4. **No rate limiting.** The system depends on public APIs and follows OpenAlex's polite-pool convention, but does not throttle its own callers.
5. **Runs at `max_total_papers = 200` take about 3.5 minutes**, exceeding NFR-1's 60-second target.

7.6 Ethical and legal considerations

All data is retrieved from public APIs under their published terms. The OpenAlex polite-pool convention is honoured by sending a contact address. No paywalled content is retrieved or redistributed, which is a consequence of the abstract-only scope: the legal question that made full text difficult is the same one that keeps the system within bounds.

Two risks deserve statement. First, **misplaced authority**: a ranked list presented by software invites more confidence than a heuristic deserves. The mitigations in Section 4.7, status labels, explicit reporting of absence, a standing caution in generated reports, are partial, and Section 6.4.4 shows one of them is weaker than it appears. Second, **entrenchment of visibility**: any citation-based ranking amplifies already-visible work, and the age term here does not correct for the systematic under-citation of research from under-resourced institutions and non-English literatures.

Chapter 8: Conclusion and Future Work

8.1 Conclusion

This thesis set out to determine whether study-scale evidence could be automatically extracted from scholarly abstracts and used to weight the edges of a citation graph, so that a reader tracing a line of research could distinguish a citation to a 43,548-participant trial from a citation to a case report.

A working system was built, deployed and measured. Against the four research questions of Section 1.3:

RQ1, Can population sizes be extracted reliably enough by pattern-based methods?

Partially. Detection achieved precision 0.917 and recall 1.000 (F1 0.957, accuracy 95% CI [0.764, 0.991]) on a 20-paper gold standard, and exact values were correct in 10 of 11 positive cases. But semantic type classification reached only 6 of 10, and, more seriously, the confidence scores attached to extractions do not discriminate correct from incorrect results (mean 0.91 when right, 0.90 when wrong). Extraction is good enough to drive edge weighting; the uncertainty signalling around it is not yet trustworthy.

RQ2. What proportion of papers carry the text needed, and does availability constrain the approach more than accuracy? Yes, decisively. In a representative 40-paper graph, 28% of papers carried no abstract in the primary provider. All were recoverable from a second provider in a single batched request. Data availability, not extraction logic, was the binding constraint on coverage, and the cheaper of the two to fix.

RQ3, Does evidence weighting produce a defensible ranking? Unproven. The system produces plausible output, seeded with the 2021 AlphaFold paper it surfaced Anfinsen's 1973 paper founding the protein-folding problem, but no relevance judgement study was run and no comparison against unweighted PageRank was performed. This thesis shows the ranking *can be computed* and *looks sensible*, not that it is better than the unweighted baseline. This is the principal unaddressed question in the work.

RQ4. Can traversal be made fast enough for interactive use, and what dominates? Yes. A 40-paper analysis completes in a mean of 19 s, and a 100-paper analysis in about 31 s. The cost is overwhelmingly network I/O (63%); all graph analytics together account for 2.5%. The largest single improvement came not from algorithmic work but from reusing one pooled HTTP client instead of creating one per request, which cut a representative run from 93.3 s to 27.9 s.

Beyond the research questions, the work produced a second result the authors did not anticipate. Six substantial defects were found in a system that passed 102 automated tests, and four of them produced no error and no visible symptom, including one that discarded roughly half of every graph, one that made every forward-citation query silently return zero, and one that collapsed every edge weight to zero while a Python truthiness accident concealed it by substituting 1.0. For a system whose output is a ranked list, testing that the pipeline runs establishes very little. Correctness required measurement against known quantities, which is precisely what the project proposal's evaluation plan specified and what had been left unimplemented.

The system's honest standing is therefore this. It is a working prototype that demonstrates the *feasibility* of evidence-weighted citation analysis and characterises its constraints with measured evidence. It is not a validated instrument for evidence appraisal, it does not read full text, and its uncertainty signalling is weaker than its interface implies.

8.2 Future Work

Ordered by expected value relative to cost.

8.2.1 Calibrate the confidence scores

The clearest and most valuable correction. Confidence currently encodes which pattern matched rather than how likely the match is to be correct (§6.4.4). Calibrating against held-out labelled data, for example by fitting a logistic model over pattern identity, sentence features and competing-candidate structure, would make the score mean what the interface claims. This requires a larger gold standard, which §8.2.3 addresses, and would make the system's distinguishing feature actually trustworthy.

8.2.2 Compare against an unweighted baseline

The central unanswered question (§7.3.4). A comparison of evidence-weighted against unweighted PageRank, with domain readers judging the relevance of ranked outputs blind to condition, would establish whether the weighting helps. Without it, the system's core premise is plausible but untested. This is the first experiment a continuation of this work should run.

8.2.3 Extend and re-annotate the gold standard

Twenty papers with one annotator produces wide intervals and no agreement measure. Extending to 150–200 papers with at least two independent annotators and a reported Cohen's κ would narrow the intervals, enable calibration, and remove the most serious methodological weakness identified in §7.3.2.

8.2.4 Supervised extraction

Type classification at 60% is where pattern methods are weakest (§7.1.2). Fine-tuning a domain-adapted encoder such as BioBERT [lee2020biobert] or SciBERT [beltagy2019scibert] on PICO population annotations [nye2018ebmnl] would target exactly the semantic discrimination surface patterns cannot make. A hybrid retaining patterns for high-precision cases and deferring ambiguous ones to the model would preserve the auditability that the current design provides.

8.2.5 Full-text extraction

The largest functional gap against the proposal (§7.4). Sample sizes often appear only in a Methods section. Europe PMC provides open-access full text, and GROBID [lopez2009grobid] is already anticipated in the configuration. Unpaywall [martin2021oadoi] would determine which articles are legally retrievable, keeping the system within the bounds §7.6 describes.

8.2.6 Study-level deduplication

Implementing the study-aware graph the proposal described (§7.4), so that a protocol, primary results paper and secondary analysis of one trial collapse to a single study node. Without it, well-published trials are over-weighted because their evidence is counted once per paper. Trial registry identifiers offer a practical join key.

8.2.7 A frozen evaluation corpus

Snapshotting the gold-standard records so the evaluation does not depend on live APIs (§7.3.3), making results exactly reproducible and allowing regression detection when extraction logic changes.

8.2.8 Citation context and sentiment

Garfield's original caveat (§2.1.1): a citation may be critical rather than supportive. Classifying citation context would let the graph distinguish support from refutation, a substantially harder problem, and the most speculative item here.

8.3 Closing Remarks

The most durable lesson of this project was not about citation graphs.

The system produced plausible output throughout a period in which it was discarding half of every graph, returning zero forward citations for every paper, and running an unweighted ranking while presenting it as evidence-weighted. Every automated test passed. What exposed the faults was measuring the system against quantities known independently, how many references the provider reported, what a human reading the abstract found, how long each stage took.

The evaluation plan that would have caught most of this was written in the project proposal before implementation began, and was left as an empty directory until the thesis was drafted. The measurement was not an afterthought to the engineering; it was the part of the engineering that had been deferred, and deferring it is what allowed the defects to persist. That is the finding the authors would carry into another project.

Appendix A: API Reference

Base URL of the deployed instance: `https://citegraph-api.penora.us` Interactive documentation: `/docs`

When `API_KEY` is configured, every `/api` route requires an `X-API-Key` header; `/health` is always open so uptime probes work. In the current deployment `API_KEY` is unset (see Section 7.5).

A.1 GET `/health`

```
{"status": "ok"}
```

A.2 POST `/api/runs`

Starts an analysis. Returns immediately with an identifier; the analysis runs in the background because a full run takes tens of seconds to minutes (Section 6.6).

Request

Field	Type	Default	Notes
<code>query_type</code>	string	required	doi, pmid, pmcid, title, url
<code>value</code>	string	required	The identifier
<code>backward_depth</code>	int	2	References; clamped 0–3
<code>forward_depth</code>	int	1	Citing works; clamped 0–2
<code>max_total_papers</code>	int	100	Clamped 1–200
<code>pdf_path</code>	string	null	Accepted but not dereferenced; confined to the uploads directory

Validation. doi must match `10.xxxx/yyyy`; pmid is 1–9 digits; pmcid is PMC plus digits; title needs at least 5 characters; url must have a scheme and host, and **must not resolve to a private, loopback, link-local or reserved address** (Section 5.7).

```
curl -X POST https://citegraph-api.penora.us/api/runs \
  -H "Content-Type: application/json" \
  -d '{"query_type": "doi", "value": "10.1056/NEJMoa2002032",
      "backward_depth": 2, "forward_depth": 1, "max_total_papers": 60}'

{"run_id": "a1b2c3d4-...", "status": "started"}
```

A.3 GET /api/runs/{run_id}

While running:

```
{"run_id": "...", "status": "running", "error": null,
  "created_at": "..."}

```

When complete, returns the full result: papers, studies, population_candidates, population_resolutions, citation_edges, ranked_foundational_papers, ranked_paths, warnings.

The warnings array is worth reading rather than ignoring. It reports how many abstracts were recovered from a secondary provider and how many duplicate records were merged, which is what allows a sparse graph to be explained rather than guessed at (Section 4.7).

On failure, error carries a caller-facing message; unexpected internal errors return a correlation reference rather than the underlying exception text.

A.4 GET /api/runs/{run_id}/graph

Visualisation payload.

```
{
  "nodes": [{"id": "10.1056/...", "label": "...", "year": 2020,
    "n_eff": 1099}],
  "links": [{"source": "10.1056/...", "target": "10.1016/...",
    "weight": 0.43}]
}

```

Every link endpoint is guaranteed present in nodes (Section 4.4.2).

A.5 Exports

All exports return a file with the correct Content-Type and a Content-Disposition filename, not a JSON envelope (Section 5.8). | Endpoint | Type | Contents |
|-----|-----|-----| | /export/json | application/json | Full result, indented || /
export/csv | text/csv | One row per paper: identifiers, authors, journal, n_eff,
population status and confidence, in/out degree, foundational rank, seed flag || /export/
edges.csv | text/csv | One row per edge with every weight component || /export/
markdown | text/markdown | Report: seed details, summary, foundational ranking,
population evidence, top citation paths || /export/graphml | application/xml |
GraphML for Gephi, yEd or Cytoscape Desktop |

Both CSV exports are written with Python's csv module, so commas, quotes and newlines inside titles and journal names are escaped correctly, and carry a UTF-8 byte-order mark so spreadsheet software renders accented author names.

Appendix B: Configuration Reference

All settings are read by `pydantic-settings` from environment variables or a `.env` file.

B.1 Application

Setting	Variable	Default	Notes
<code>app_env</code>	<code>APP_ENV</code>	<code>development</code>	A non-development value without <code>API_KEY</code> logs a startup warning
<code>log_level</code>	<code>LOG_LEVEL</code>	<code>INFO</code>	

B.2 Security

Setting	Variable	Default	Notes
<code>api_key</code>	<code>API_KEY</code>	<code>unset</code>	When set, every <code>/api</code> route requires <code>X-API-Key</code> . Unset in the deployment (§7.5)
<code>cors_origins</code>	<code>CORS_ORIGINS</code>	<code>http://localhost:5173,http://127.0.0.1:5173</code>	Comma-separated allowed browser origins

A wildcard CORS policy is deliberately not used. It would let any page a developer visits drive their locally running instance and read the responses.

B.3 Providers

Setting	Variable	Default	Notes
<code>openalex_email</code>	<code>OPENALEX_EMAIL</code>	<code>unset</code>	Polite-pool identification; materially affects latency
<code>enable_openalex</code>	<code>ENABLE_OPENALEX</code>	<code>true</code>	Primary provider
<code>enable_crossref</code>	<code>ENABLE_CROSSREF</code>	<code>true</code>	Publisher-deposited metadata
<code>enable_europe_pmc</code>	<code>ENABLE_EUROPE_PMC</code>	<code>true</code>	Required for abstract backfill (§6.3)

Setting	Variable	Default	Notes
<code>enable_pubmed</code>	<code>ENABLE_PUBMED</code>	<code>false</code>	Reserved; no provider implemented
<code>enable_semantic_scholar</code>	<code>ENABLE_SEMANTIC_SCHOLAR</code>	<code>false</code>	Reserved; no provider implemented
<code>enable_clinical_trials</code>	<code>ENABLE_CLINICAL_TRIALS</code>	<code>false</code>	Reserved; no provider implemented
<code>enable_unpaywall</code>	<code>ENABLE_UNPAYWALL</code>	<code>false</code>	Reserved; no provider implemented

Disabling Europe PMC removes the abstract backfill and, on the basis of Section 6.3, would leave roughly 28% of papers without extractable text.

B.4 Traversal and weighting

Setting	Variable	Default	Notes
<code>default_backward_depth</code>	<code>DEFAULT_BACKWARD_DEPTH</code>	2	Clamped 0–3
<code>default_forward_depth</code>	<code>DEFAULT_FORWARD_DEPTH</code>	1	Clamped 0–2
<code>default_max_total_papers</code>	<code>DEFAULT_MAX_TOTAL_PAPERS</code>	100	Clamped 1–200
<code>weight_alpha</code>	<code>WEIGHT_ALPHA</code>	0.75	Weight on the population evidence term
<code>weight_beta</code>	<code>WEIGHT_BETA</code>	0.25	Weight on the journal term
<code>n_reference</code>	<code>N_REFERENCE</code>	100000	Reference population for log normalisation

`max_total_papers` is exposed in the dashboard as a 10–200 slider. Section 6.6 recommends 100 or below: 200 takes around 3.5 minutes and exceeds NFR-1.

B.5 Storage and optional services

Setting	Variable	Default	Notes
<code>data_dir</code>	<code>DATA_DIR</code>	<code>./data</code>	
<code>sqlite_path</code>	<code>SQLITE_PATH</code>	<code>./data/cache/citegraph.sqlite</code>	Run persistence

Setting	Variable	Default	Notes
enable_grobid	ENABLE_GROBID	true	Flag only; no PDF parsing is implemented (§7.4)
grobid_url	GROBID_URL	http://localhost:8070	Unused
enable_neo4j	ENABLE_NEO4J	false	Flag only; no Neo4j integration is implemented
neo4j_uri / neo4j_user	n/a	n/a	Unused
neo4j_password	NEO4J_PASSWORD	unset	No default; required before enabling the profile

The GROBID and Neo4j flags are configuration remnants of capabilities specified in the proposal but not built. They are listed here rather than removed so the divergence in Section 7.4 is traceable from the configuration itself.

B.6 Internal constants

Not environment-configurable; changing them requires a code edit.

Constant	Value	Purpose
OpenAlexProvider.BATCH_SIZE	50	Identifiers per batched metadata request
OpenAlexProvider.MAX_FORWARD_CITATIONS	200	Citing works retrieved per paper
CitationTraversal.FORWARD_BUDGET_SHARE	0.35	Budget reserved for forward traversal (§4.4.4)
CitationTraversal.MAX_CONCURRENT_EXPANSIONS	5	Concurrent reference/citation lookups
CitationTraversal.TITLE_KEY_PREFIX	80	Title prefix length for duplicate detection
CitationTraversal.MIN_TITLE_KEY_LENGTH	25	Shortest title eligible for prefix matching
GraphAnalytics.MAX_PATHS_EXAMINED	50,000	Safety cap on path enumeration (§6.6.4)

Appendix C: Reproducing the Results

Every quantitative claim in Chapter 6 is produced by a script committed to the repository. This appendix gives the commands.

C.1 Environment

```
git clone https://github.com/Ahad690/CiteGraph-NLP.git
cd CiteGraph-NLP
python -m venv .venv
# Windows:      .venv\Scripts\Activate.ps1
# macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env          # then set OPENALEX_EMAIL
```

Setting OPENALEX_EMAIL is not optional in practice. OpenAlex serves identified callers from a faster "polite pool"; without it, request latency, which Section 6.6 shows dominates runtime, is materially worse.

C.2 Test suite

```
pytest -q
```

Expected: **102 passed**. The suite mocks all HTTP at transport level with `respx`, so it requires no network access and no API keys.

C.3 Evaluation (Chapter 6)

```
python scripts/run_evaluation.py --out thesis/evidence --max-
papers 40
```

Writes `thesis/evidence/evaluation_results.json` and prints a summary. Requires network access; takes roughly three to five minutes, most of it in the three live traversals.

To reproduce only the extraction and metadata results (Sections 6.2–6.4) and skip the traversals:

```
python scripts/run_evaluation.py --skip-graph --out thesis/
evidence
```

The JSON artifact contains a `per_paper` array with the prediction, gold label, confidence and abstract source for every gold-standard paper, which is the source for the failure analysis in Sections 6.4.2 to 6.4.4.

C.4 Reference verification (Section 6.7)

```
python scripts/verify_references.py --out thesis/evidence
```

Resolves every candidate DOI against Crossref, falling back to OpenAlex, and writes `references_verified.json` and `references.bib`. Any candidate that fails to resolve is printed as UNVERIFIED and excluded from the bibliography.

C.5 Rebuilding the thesis

```
python scripts/build_thesis.py
```

Regenerates the reference list and the gold-standard appendix from the verified artifacts, then concatenates all chapters into `thesis/CiteGraph-NLP-Thesis.md`. The reference list is *generated*, not hand-maintained, so it cannot drift from the verification record.

C.6 Regenerating the figures

```
python scripts/generate_diagrams.py
```

Writes all eight SVG figures to `thesis/figures/`. Needs `pylint`, `code2flow` and the Graphviz dot binary; the first two install with

```
pip install pylint code2flow
```

and Graphviz is a separate native package (`winget install Graphviz.Graphviz`, `brew install graphviz`, or `apt install graphviz`). Appendix H explains what each figure is derived from and why the chapter versions are reduced.

The figures are committed, so rebuilding the thesis does not require Graphviz. Rerun the script only after changing the models, the package layout or the orchestrator, since those are what the figures are derived from.

C.7 Rendering to PDF

Two engines work. WeasyPrint needs no LaTeX installation and is what the page counts in this thesis were measured with:

```
pandoc thesis/CiteGraph-NLP-Thesis.md \
  -o thesis/CiteGraph-NLP-Thesis.pdf \
  --pdf-engine=weasyprint \
  -f markdown-smart \
  --toc --toc-depth=3
```

Do not add `--number-sections`. Every section in this thesis already carries its number in the heading text, and the cross-references throughout the body point at those numbers. Pandoc's automatic numbering is added on top rather than replacing them, so headings render with two numbers ("2.1 1.1 Background and Motivation"), and its count is offset by one because it treats the front matter as the first chapter.

`-f markdown-smart` matters. Pandoc's smart typography rewrites `--` as an en dash and `---` as an em dash, so the rendered PDF ends up containing dashes the source never had. A mechanical pre-submission check run against that PDF then reports them as prose findings: five of the six em dashes flagged in one such run came from this conversion rather than from the manuscript.

A DOCX, if the department requires one:

```
pandoc thesis/CiteGraph-NLP-Thesis.md \
  -o thesis/CiteGraph-NLP-Thesis.docx \
  -f markdown-smart \
  --toc --toc-depth=3
```

With a LaTeX distribution instead:

```
pandoc thesis/CiteGraph-NLP-Thesis.md \
  -o thesis/CiteGraph-NLP-Thesis.pdf \
  -f markdown-smart \
  --toc --toc-depth=3 \
  -V documentclass=report \
  -V papersize=a4 \
  -V fontsize=11pt \
  -V geometry:margin=1in \
  -V linkcolor=blue
```

<div class="pagebreak"></div> markers between chapters are inserted by the assembly script and are honoured by the LaTeX writer.

C.8 Running the system

Locally:

```
uvicorn citegraph.api.main:app --reload      # API on :8000
cd frontend && npm install && npm run dev    # dashboard on :5173
```

Or the whole stack:

```
docker compose up --build
```

The deployed instance is documented in the project README.

C.9 A caveat on exact reproduction

The evaluation queries live scholarly APIs. OpenAlex and Crossref revise records continuously, abstracts are added, reference lists are corrected, citation counts change daily. A rerun may therefore differ from the figures in Chapter 6, particularly the traversal statistics in Section 6.5, which depend on what the providers hold at query time.

The gold-standard labels are fixed and committed, so the extraction metrics in Section 6.4 are stable provided the abstracts remain retrievable. Section 8.2.7 proposes snapshotting the corpus to remove this dependency entirely.

Appendix D: Gold Standard Annotations

The complete annotated set used in Chapter 6. Each label was assigned by reading the abstract retrieved through the same providers the pipeline uses; the supporting sentence is quoted so every label is auditable.

`n_eff` is the total number of human subjects the paper's primary analysis rests on, as stated in the abstract. A dash means no human study population is stated and the correct behaviour is to extract nothing.

D.1 Summary

# DOI	Design	Gold <code>n_eff</code>	Predicted	Match
1 10.1056/nejmoa2034577	rct	43548	43548	ok
2 10.1056/nejmoa2035389	rct	30420	30420	ok
3 10.1056/nejmoa1911303	rct	4744	4744	ok
4 10.1056/nejmoa1812389	rct	17160	17160	ok
5 10.1016/ s0140-6736(20)31604-4	rct	1077	1077	ok
6 10.1056/nejmoa2002032	cohort	1099	1099	ok
7 10.1016/ s0140-6736(20)30183-5	case_series	41	41	ok
8 10.1001/ jama.2020.1585	case_series	138	36	miss
9 10.1056/nejmoa2001316	epidemiological	425	425	ok
10 10.1016/ s2213-2600(20)30079-5	cohort	52	52	ok
11 10.1016/ s1473-3099(20)30243-7	modelling	1334	1334	ok
12 10.1056/nejmoa2001017	virus_characterisation	none	none	ok
13 10.1038/ s41586-020-2012-7	virus_characterisation	none	none	ok
14 10.1136/bmj.m1328	systematic_review	none	27	miss
15 10.1038/ s41577-020-0311-8	review	none	none	ok
16 10.1164/ rccm.201908-1581st	guideline	none	none	ok
17 10.1056/nejmra2026131	review	none	none	ok
18 10.1038/ s41586-021-03819-2	computational	none	none	ok
19 10.1038/nature14539	review	none	none	ok
20 10.1145/3065386	computational	none	none	ok

D.2 Annotations with supporting evidence

D.2.1 10.1056/nejmoa2034577

- **Design:** rct
- **Gold n_eff:** 43548
- **Gold semantic type:** TOTAL_RANDOMIZED
- **Supporting evidence:** A total of 43,548 participants underwent randomization, of whom 43,448 received injections.

D.2.2 10.1056/nejmoa2035389

- **Design:** rct
- **Gold n_eff:** 30420
- **Gold semantic type:** TOTAL_ENROLLED
- **Supporting evidence:** The trial enrolled 30,420 volunteers who were randomly assigned in a 1:1 ratio.

D.2.3 10.1056/nejmoa1911303

- **Design:** rct
- **Gold n_eff:** 4744
- **Gold semantic type:** TOTAL_RANDOMIZED
- **Supporting evidence:** we randomly assigned 4744 patients with New York Heart Association class II, III, or IV heart failure

D.2.4 10.1056/nejmoa1812389

- **Design:** rct
- **Gold n_eff:** 17160
- **Gold semantic type:** TOTAL_ANALYZED
- **Supporting evidence:** We evaluated 17,160 patients, including 10,186 without atherosclerotic cardiovascular disease.

D.2.5 10.1016/s0140-6736(20)31604-4

- **Design:** rct
- **Gold n_eff:** 1077
- **Gold semantic type:** TOTAL_ENROLLED
- **Supporting evidence:** 1077 participants were enrolled and assigned to receive either ChAdOx1 nCoV-19 (n=543) or MenACWY (n=534)

D.2.6 10.1056/nejmoa2002032

- **Design:** cohort
- **Gold n_eff:** 1099
- **Gold semantic type:** TOTAL_ANALYZED
- **Supporting evidence:** We extracted data regarding 1099 patients with laboratory-confirmed Covid-19 from 552 hospitals.

D.2.7 10.1016/s0140-6736(20)30183-5

- **Design:** case_series
- **Gold n_eff:** 41
- **Gold semantic type:** TOTAL_ANALYZED
- **Supporting evidence:** By Jan 2, 2020, 41 admitted hospital patients had been identified as having laboratory-confirmed 2019-nCoV infection.

D.2.8 10.1001/jama.2020.1585

- **Design:** case_series
- **Gold n_eff:** 138
- **Gold semantic type:** TOTAL_ANALYZED
- **Supporting evidence:** Retrospective, single-center case series of the 138 consecutive hospitalized patients with confirmed NCIP.

D.2.9 10.1056/nejmoa2001316

- **Design:** epidemiological
- **Gold n_eff:** 425
- **Gold semantic type:** TOTAL_ANALYZED
- **Supporting evidence:** We analyzed data on the first 425 confirmed cases in Wuhan.

D.2.10 10.1016/s2213-2600(20)30079-5

- **Design:** cohort
- **Gold n_eff:** 52
- **Gold semantic type:** TOTAL_ENROLLED
- **Supporting evidence:** we enrolled 52 critically ill adult patients with SARS-CoV-2 pneumonia.

D.2.11 10.1016/s1473-3099(20)30243-7

- **Design:** modelling
- **Gold n_eff:** 1334
- **Gold semantic type:** SAMPLE_SIZE_GENERIC
- **Supporting evidence:** We also estimated the case fatality ratio from individual line-list data on 1334 cases identified outside of mainland China.

D.2.12 10.1056/nejmoa2001017

- **Design:** virus_characterisation
- **Gold n_eff:** none (negative case)
- **Gold semantic type:** n/a
- **Supporting evidence:** Abstract describes isolation of a novel betacoronavirus; no study population size is stated.

D.2.13 10.1038/s41586-020-2012-7

- **Design:** virus_characterisation
- **Gold n_eff:** none (negative case)
- **Gold semantic type:** n/a

- **Supporting evidence:** HARD NEGATIVE: '2,794 laboratory-confirmed infections including 80 deaths' is an epidemic tally, not this study's sample.

D.2.14 10.1136/bmj.m1328

- **Design:** systematic_review
- **Gold n_eff:** none (negative case)
- **Gold semantic type:** n/a
- **Supporting evidence:** HARD NEGATIVE: a systematic review of prediction models; its units are studies, not patients.

D.2.15 10.1038/s41577-020-0311-8

- **Design:** review
- **Gold n_eff:** none (negative case)
- **Gold semantic type:** n/a
- **Supporting evidence:** Narrative review of COVID-19 immunology; no study population.

D.2.16 10.1164/rccm.201908-1581st

- **Design:** guideline
- **Gold n_eff:** none (negative case)
- **Gold semantic type:** n/a
- **Supporting evidence:** Clinical practice guideline; no study population.

D.2.17 10.1056/nejmra2026131

- **Design:** review
- **Gold n_eff:** none (negative case)
- **Gold semantic type:** n/a
- **Supporting evidence:** Review article on cytokine storm; no study population.

D.2.18 10.1038/s41586-021-03819-2

- **Design:** computational
- **Gold n_eff:** none (negative case)
- **Gold semantic type:** n/a
- **Supporting evidence:** HARD NEGATIVE: protein-structure prediction; CASP14 target counts are not human subjects.

D.2.19 10.1038/nature14539

- **Design:** review
- **Gold n_eff:** none (negative case)
- **Gold semantic type:** n/a
- **Supporting evidence:** Review of deep learning; no study population.

D.2.20 10.1145/3065386

- **Design:** computational
- **Gold n_eff:** none (negative case)
- **Gold semantic type:** n/a

- **Supporting evidence:** HARD NEGATIVE: '1.2 million high-resolution images' is a dataset size, not a human study population.

Appendix E: Requirements Traceability

This appendix maps every requirement from Chapter 3 to the implementing module, the verifying test or measurement, and the section reporting the outcome. Its purpose is to make partial and unmet requirements visible rather than leaving them to be inferred from absence.

E.1 Functional requirements

ID	Requirement	Implemented in	Verified by	Status
FR-1	Accept DOI, PMID, PMCID, title, URL	models/paper.py, input/normalizer.py	test_input_normalizer.py, test_api_comprehensive.py	Met
FR-2	Canonicalise identifiers	utils/ids.py	test_input_normalizer.py	Met
FR-3	Extract identifier from URL text	input/url_resolver.py	test_url_resolver.py	Met
FR-4	Reject malformed identifiers	models/paper.py	test_api_comprehensive.py	Met
FR-5	Concurrent multi-provider query	metadata/resolver.py	Code inspection; §6.6.1	Met
FR-6	Field-level merge precedence	metadata/merger.py	§4.8	Met
FR-7	Per-provider provenance	models/paper.py	Data inspection	Met
FR-8	Recover missing abstracts	providers/europe_pmc.py, pipeline/orchestrator.py	§6.3	Met
FR-9	Backward traversal, depth 0–3	citations/traversal.py	§6.5	Met
FR-10	Forward traversal, depth 0–2	citations/traversal.py, providers/openalex.py	§6.5	Met
FR-11	Bound total papers 1–200	api/routes.py	test_api_comprehensive.py	Met
FR-12	Merge duplicate records	citations/traversal.py	§6.5	Met; merge precision unmeasured (§7.3.2)
FR-13	No dangling edges	citations/traversal.py	§6.5, 0 across 3 runs	Met

ID	Requirement Implemented in	Verified by	Status
FR-14	Extract population candidates nlp/ population_extractor.py	§6.4.1	Met
FR-15	Classify semantic type nlp/ population_patterns.py	§6.4.3, 6/10	Partially met
FR-16	Resolve to one value per paper nlp/ population_resolver.py	§6.4.2, 10/11	Met
FR-17	Attach confidence and status models/population.py	§6.4.4	Partially met, confidence uncalibrated
FR-18	Ignore years, percentages, p-values nlp/ population_extractor.py	test_population_patterns.py	Met
FR-19	Evidence-based edge weighting graph/weighting.py	§5.6	Met
FR-20	Non-zero weight without evidence graph/weighting.py, graph/builder.py	§5.6	Met
FR-21	Rank foundational papers graph/analytics.py	§6.5	Met; ranking quality unvalidated (§7.3.4)
FR-22	Rank citation paths graph/analytics.py	§6.6.4	Met
FR-23	Five export formats api/routes.py	test_api_comprehensive.py	Met
FR-24	REST API and dashboard api/, frontend/	§5.9, §5.11	Met

Summary: 22 of 24 met; FR-15 and FR-17 partially met. Both partial results concern the same underlying weakness, the system's semantic and probabilistic judgements about extractions are weaker than its value extraction.

E.2 Non-functional requirements

ID	Requirement	Target	Measured	Status
NFR-1	40-paper run within 60 s	< 60 s	19.0 s mean (§6.5)	Met at 40 and 100 papers; fails at 200 (209 s, §6.6.2)
NFR-2	Analytics a minority of runtime	< 25%	2.5% (§6.6.1)	Met

ID	Requirement	Target	Measured	Status
NFR-3	Provider failure degrades, not aborts	No unhandled exception	§5.4	Met
NFR-4	No paywalled full text retrieved	Zero	By construction	Met
NFR-5	No outbound fetch to private addresses	Zero	§5.7	Met
NFR-6	Browser access restricted to one origin	Configured origin	§5.9	Met
NFR-7	Every metric recomputable from a script	100%	Appendix C	Met

E.3 Proposal capabilities not delivered

Listed for completeness; discussed in Section 7.4.

Capability	State	Evidence
GROBID full-text parsing	Configuration flag only	No module; <code>ENABLE_GROBID</code> unused
Neo4j graph store	Configuration flags only	No driver integration
Study-level deduplication	1:1 paper→study mapping	<code>dedupe_confidence</code> hardcoded 0.8
Streamlit dashboard	Not implemented	Superseded by the React SPA
<code>EVENT_COUNT</code> semantic type	Model only	No pattern emits it
<code>UNKNOWN_NUMERIC</code> semantic type	Model only	No pattern emits it
Journal quality term	Constant 0.5	<code>weighting.py</code>

E.4 Research questions to evidence

RQ	Question	Answered in	Verdict
RQ1	Reliable pattern-based extraction?	§6.4	Partially, detection strong (F1 0.957), typing weak (0.60), confidence uncalibrated
RQ2	Does text availability constrain more than accuracy?	§6.3	Yes, 28% of papers lacked abstracts; all recoverable from a second provider
RQ3	Defensible evidence-weighted ranking?	§6.5	Unproven , plausible output, no relevance study, no baseline comparison
RQ4	Fast enough, and what dominates?	§6.6	Yes, 19 s mean; 63% network I/O, 2.5% analytics

Appendix F: Test Suite and Verification Inventory

F.1 Composition

102 automated tests across eight files. All external HTTP is intercepted at transport level by `respx`, so the suite requires no network access and no API credentials, and completes in roughly 20–35 seconds.

File	Tests Covers
<code>test_api_comprehensive.py</code>	63 Endpoints, validation, clamping, auth, CORS, exports, pipeline end-to-end with mocked providers
<code>test_sqlite_store.py</code>	8 Persistence, lock retry policy, backoff jitter, error classification
<code>test_add_citegraph_route.py</code>	7 Deployment route-insertion helper
<code>test_population_patterns.py</code>	6 Extraction patterns and ignore-span behaviour
<code>test_ranking.py</code>	5 Foundational scoring and path ranking
<code>test_task_manager.py</code>	5 Background task lifecycle and shutdown semantics
<code>test_url_resolver.py</code>	5 URL→identifier extraction, DOI view-segment trimming
<code>test_input_normalizer.py</code>	3 Identifier canonicalisation
Total	102

F.2 Regression tests added during evaluation

Each corresponds to a defect in Chapter 5, and each fails against the pre-correction code.

Test	Pins
<code>test_count_survives_a_year_in_the_same_sentence</code>	A sample size beside a date is not discarded (§5.5). Asserts 1099 is extracted and 2020 is not.
<code>test_percentage_in_sentence_does_not_discard_count</code>	A percentage elsewhere in the sentence does not suppress the count

Test

`test_year_alone_is_not_a_population`

`test_observational_phrasings_are_extracted`

`test_doi_extraction_strips_publisher_view_segments`

`test_doi_with_legitimate_slashes_is_preserved`

`test_export_csv_escapes_special_characters`

`test_export_edges_csv, test_export_graphml`

`test_start_run_rejects_pdf_path_outside_uploads`

API-key and CORS suites

Pins

A bare year is still never read as a population cases, a total of, consecutive patients, screened phrasings (§5.5)

/full, /pdf, /abs trimmed from URL-embedded DOIs (§5.7)

Trimming never damages a DOI that genuinely contains slashes

Commas, quotes and newlines in titles do not break CSV rows (§5.8)

Endpoints exist and produce parseable output

Path confinement on pdf_path

Auth gating and origin restriction (§5.9)

F.3 Defects found *in the test suite itself*

Two, both recorded because a test suite is software subject to the same faults as the system it verifies.

A hang, not a failure.

`test_uncooperative_task_does_not_hang_past_timeout` constructed a task that suppresses `CancelledError`, then cleaned up with `asyncio.wait_for`. On timeout, `wait_for` cancels the task *and awaits the cancellation*, which never completes for a task that refuses to die. The entire pytest session hung indefinitely. The name is unintentionally accurate: the test for not hanging was the thing that hung. Resolved by giving the fake task a stop event and waiting with `asyncio.wait`, which always returns.

A double that could not exercise the code path. `test_get_run_also_retries_under_lock` replaced `aiosqlite's execute` with a bare `async def`. The real method is wrapped so its return value supports both `await` and `async with`; a plain coroutine supports only `await`. The production read path uses `async with`, so the test failed with a `TypeError` about the coroutine protocol rather than exercising the retry policy it was written to verify. Resolved by wrapping the doubles in `aiosqlite's own Result`.

F.4 What the suite does not establish

Stated because Section 5.10 turns on it. Throughout the period in which the traversal was discarding roughly half of every graph, forward citations were universally empty, and every edge weight was zero, **the suite passed in full**.

It did so because it asserted *shape*, not *correctness*: that a run completed, that the response carried the expected fields, that the types were right. No test asserted that the number of retrieved nodes bore any relation to the number of references the provider reported, that forward edges ever existed, or that edge weights varied between papers.

The suite is a regression guard, not a correctness argument. Correctness for this system required measurement against independently known quantities, which is what Chapter 6 reports.

F.5 Manual verification performed

Checks carried out by inspection or ad-hoc measurement rather than automated tests, listed so the evidence base is complete.

Check	Method	Section
SSRF guard blocks internal addresses	Local HTTP listener; verified zero requests received	§5.7
Redirect pivot to internal host refused	Mocked 302 from a public host to loopback	§5.7
Connection pooling benefit	8 sequential requests, pooled versus per-request client	§6.6.3
Path ranking equivalence	Top-ten compared against exhaustive enumeration	§6.6.4
Weighting ordering after correction	Synthetic resolutions across <code>n_eff</code> and confidence	§5.6
Duplicate merging	Title-key collision check over produced graphs	§6.5
Reference resolution	Every DOI resolved against Crossref/OpenAlex	§6.7
Deployment isolation	Container port confirmed bound to loopback	§5.9

Appendix G: Selected Code Listings

Extracts from the delivered system, chosen because each embodies a decision argued elsewhere in the thesis. Listings are lightly trimmed for width; comments are as committed.

G.1 Per-span ignore matching (Section 5.5)

The correction that recovered most of the extraction recall. The original evaluated ignore patterns against the whole sentence and skipped every candidate in it.

```
for sentence in sentences:
    # Locate the spans that must not be read as population sizes
    # (years,

    # percentages, p-values, dosages). These are matched per-
    # span rather
    # than per-sentence: skipping the whole sentence discarded
    # every
    # genuine count that merely shared a sentence with a date,
    # which is
    # most of them ("...1099 patients ... through January 29,
    # 2020").
    ignore_spans = [
        match.span()
        for ignore_p in IGNORE_PATTERNS
        for match in re.finditer(ignore_p, sentence)
    ]

    for pattern_info in POPULATION_PATTERNS:
        for match in re.finditer(pattern_info["pattern"],
            sentence, re.IGNORECASE):
            # Only reject when the captured number itself sits
            # inside an

            # ignored span, not when one appears elsewhere in the
            # sentence.
            number_start, number_end = match.span(1)
            if any(start < number_end and number_start < end
                for start, end in ignore_spans):
                continue
            ...
```

G.2 Edge weighting (Section 5.6)

```
base_weight = (self.alpha * n_score) + (self.beta *
    journal_score)

# Confidence scales the *population evidence* only. Multiplying
the
# whole base weight by it collapsed every edge to exactly 0.0 for
any
# paper where extraction found nothing -- which is every paper
outside
# clinical-trial phrasing. The journal term is structural and
always
# applies, so an edge without population evidence keeps a small
uniform
# weight instead of vanishing.
pop_confidence = target_res.confidence if target_res else 0.5
evidence_term = self.alpha * n_score * pop_confidence
final_weight = (evidence_term + (self.beta * journal_score)) *
    edge.confidence
```

And the fallback in the graph builder that had concealed the defect:

```
# `or 1.0` would silently rewrite a genuine 0.0 weight to 1.0,
# which hid the fact that every edge was being zeroed out.
weight=edge.final_weight if edge.final_weight is not None else
    1.0,
```

G.3 Identity aliasing and duplicate merging (Section 4.4.2)

```
def _register(self, paper: Paper) -> None:

    """Store a paper and map every identifier it carries onto
its paper_id."""
    self.papers[paper.paper_id] = paper
    self.visited.add(paper.paper_id)
    for raw in (paper.paper_id, paper.doi, paper.pmid,
        paper.pmcid, paper.openalex_id):
        if raw:
            self._aliases[IdCanonicalizer.canonicalize(raw)] =
                paper.paper_id
    key = self.title_key(paper.title)
    if key:
        self._title_keys.setdefault(key, paper.paper_id)

@classmethod
def title_key(cls, title: Optional[str]) -> Optional[str]:
```

```
"""A comparison key for detecting the same work under two
    identifiers.
```

```
Sources occasionally hold two records for one article under
    different
DOIs -- typically the clean version and one with front matter
    merged
into the title ("...coronavirus infection1 1The authors
    thank..."). The
titles share a long prefix, so compare a normalised prefix
    rather than
the whole string. Returns None for titles too short to match
    safely.
```

```
"""
```

```
if not title:
    return None
normalised = " ".join(re.sub(r"[^a-z0-9]+", " ",
    title.lower()).split())
if len(normalised) < cls.MIN_TITLE_KEY_LENGTH:
    return None
return normalised[: cls.TITLE_KEY_PREFIX]
```

G.4 Commit-time edge filtering (Section 4.4.2)

The guarantee behind the zero-dangling-edge result in Section 6.5.

```
def _commit_edges(self, raw_edges, direction) -> None:
    """Rewrite edge endpoints to canonical ids and keep only
        resolved ones."""
    dropped = 0
    for edge in raw_edges:
        source = self._canonical(edge.source_paper_id)
        target = self._canonical(edge.target_paper_id)
        if not source or not target or source == target:
            dropped += 1
            continue
        key = (source, target)
        if key in self._edge_keys:
            continue
        self._edge_keys.add(key)
        edge.source_paper_id = source
        edge.target_paper_id = target
        edge.edge_id = f"{source}_cites_{target}"
        self.edges.append(edge)
```

G.5 Null-safe provider mapping (Section 5.4)

```
@staticmethod
def _section(data: dict[str, Any], key: str) -> dict[str, Any]:
    """Fetch a nested object, treating an explicit null as an
       empty object.

       OpenAlex returns ``"primary_location": {"source": null}`` for
       works with
       no indexed venue (roughly half the results in some queries),
       and
       ``dict.get(key, {})`` returns None in that case because the
       key exists.
    """
    value = data.get(key)
    return value if isinstance(value, dict) else {}
```

With per-record isolation so one malformed work cannot discard its batch:

```
for work in data.get("results", []):
    work_id = self.openalex_id_of(work)
    if not work_id:
        continue
    try:
        papers[work_id] = self._map_to_paper(work)
    except Exception as e:
        # Isolate per-record faults: one malformed work must not
        # discard the other 49 in the batch.
        logger.warning("Skipping unmappable OpenAlex work %s:
            %s", work_id, e)
```

G.6 SSRF guard (Section 5.7)

```
@staticmethod
async def _resolves_to_public_address(url: str) -> bool:
    """True when every address the URL's host resolves to is
       public.

       The URL here comes straight from the API request body, so
       without this
       check the server can be pointed at loopback, link-local
       (cloud metadata)
       or RFC1918 addresses and made to issue requests from inside
       the trust
       boundary.
    """
    parsed = urlparse(url)
```



```

if parsed.scheme not in {"http", "https"} or not
    parsed.hostname:
    return False
port = parsed.port or (443 if parsed.scheme == "https" else
    80)
try:
    infos = await asyncio.get_running_loop().getaddrinfo(
        parsed.hostname, port, type=socket.SOCK_STREAM)
except (socket.gaierror, UnicodeError, ValueError):
    return False
for info in infos:
    address = ipaddress.ip_address(info[4][0])
    if (address.is_private or address.is_loopback or
        address.is_link_local
            or address.is_reserved or address.is_multicast
            or address.is_unspecified):
        return False
return True

```

Redirects are followed manually so each hop is re-validated; a public host that redirects to an internal one is refused at the second hop.

G.7 Bounded path enumeration (Section 4.12)

```

def _iter_paths(self, seed_id: str, cutoff: int = 4):
    """Yield every simple path leaving the seed, in a single
        traversal.

        Calling ``nx.all_simple_paths(seed, target)`` once per target
        re-walks
        the entire reachable subgraph for every node in the graph, so
        the cost
        is multiplied by the node count while producing the same set
        of paths.
        One depth-limited DFS yields each path exactly once.
        """
    stack = [[seed_id]]
    while stack:
        path = stack.pop()
        if len(path) > 1:
            yield path
        if len(path) - 1 >= cutoff:
            continue
        for successor in self.graph.successors(path[-1]):
            # Paths are at most cutoff+1 long, so a scan beats a
            set here.
            if successor not in path:
                stack.append(path + [successor])

```

G.8 Retry policy (Section 5.4)

```
RETRYABLE_STATUS_CODES = frozenset({408, 425, 429, 500, 502,
                                     503, 504})
```

```
def is_transient_error(exc: BaseException) -> bool:
    """True when a failed provider call is worth retrying.

    A 404 means the provider simply does not hold that record,
    which is
    routine for Crossref and Europe PMC. Retrying it cannot
    change the
    answer and costs the exponential-backoff budget on every
    miss, so only
    rate limits, server faults and transport errors are retried.
    """

    if isinstance(exc, httpx.HTTPStatusError):
        return exc.response.status_code in RETRYABLE_STATUS_CODES
    return isinstance(exc, (httpx.TransportError,
                           httpx.StreamError))
```

G.9 Pooled HTTP client (Section 6.6.3)

```
# One pooled client for every outbound provider call. Creating a
    client per
# request (the previous `async with httpx.AsyncClient(...)` in
    each _get)
# meant a fresh TCP and TLS handshake every time: measured at
    ~634 ms of
# pure overhead per request, 58% of the time each call took.
_shared_client: Optional[httpx.AsyncClient] = None
_client_lock = asyncio.Lock()
```

```
HTTP_LIMITS = httpx.Limits(max_connections=20,
                           max_keepalive_connections=10)
HTTP_TIMEOUT = httpx.Timeout(20.0, connect=10.0)
```

```
async def get_shared_client() -> httpx.AsyncClient:
    global _shared_client
    if _shared_client is None or _shared_client.is_closed:
        async with _client_lock:
            if _shared_client is None or
                _shared_client.is_closed:
                _shared_client = httpx.AsyncClient(
                    timeout=HTTP_TIMEOUT, limits=HTTP_LIMITS)
    return _shared_client
```

G.10 Evaluation metric definitions (Section 3.5.3)

```
def wilson_interval(successes: int, trials: int, z: float =
    1.96):
    """95% Wilson score interval.

    Reported instead of a bare proportion because the gold set is
    small; a
    normal approximation is unreliable at n=20 and degenerates at
    0% or 100%.
    """
    if trials == 0:
        return None
    p = successes / trials
    denominator = 1 + z * z / trials
    centre = (p + z * z / (2 * trials)) / denominator
    margin = z * ((p * (1 - p) / trials
        + z * z / (4 * trials * trials)) ** 0.5) /
        denominator
    return (max(0.0, centre - margin), min(1.0, centre + margin))
```

Appendix H: Full-Scale Generated Diagrams

Every figure in this thesis that describes the structure of the system was produced by running a tool over `src/citegraph`, not by drawing it. The chapters show reduced views chosen to stay legible at page width. This appendix carries the complete plates the reductions were taken from, so a reader can check that nothing material was hidden by the reduction.

H.1 How the figures are produced

One script generates all eight:

```
python scripts/generate_diagrams.py
```

It needs `pylint` (which supplies `pyreverse`), `code2flow`, and the Graphviz `dot` binary on `PATH`. Output lands in `thesis/figures/` as SVG. The script prints the node and edge counts for each figure, which is how the counts quoted in the captions were obtained.

Figure	Source	Tool
4.1 Inter-package dependencies	<code>src/citegraph imports</code>	<code>pyreverse</code> , collapsed to packages
4.2 Object composition	class attributes	<code>pyreverse -k</code> , filtered to connected classes
4.3 Data model, extraction half	<code>models/ Pydantic classes</code>	introspection of <code>model_fields</code>
4.4 Data model, graph half	<code>models/ Pydantic classes</code>	introspection of <code>model_fields</code>
5.1 Orchestrator call graph	<code>pipeline/ orchestrator.py</code>	<code>code2flow</code> , depth 2
H.1 Combined data model	<code>models/ Pydantic classes</code>	introspection of <code>model_fields</code>
H.2 Complete class diagram	all classes	<code>pyreverse</code>
H.3 Complete call graph	all functions	<code>code2flow</code>

The data-model figures deserve a note on method. They are not parsed from source text; the script imports the Pydantic classes and reads `model_fields`, which is the same structure Pydantic uses to validate incoming data at runtime. A field renamed in the code changes the next figure without anyone editing a diagram, and a figure showing a field that no longer exists is not possible.

H.2 Why the chapter figures are reduced

Three reductions are applied, each for the same reason. How legible a Graphviz figure is at a fixed page width depends only on the ratio between its font size and its total width, and enlarging the font enlarges the boxes by the same proportion. Narrowing the graph is therefore the only lever available.

Collapsing modules to packages (Figure 4.1). The raw output has 46 module nodes and 83 import edges, and at page width its labels render at roughly three points. Collapsing each module to its package leaves 13 nodes and 38 edges, and carries the discarded detail as edge thickness rather than losing it.

Filtering unrelated classes (Figure 4.2). `pyreverse` draws every class it finds, including the 15 that take part in no association or inheritance relationship. Those 15 occupy a full column of the canvas and contribute no structure, so they are dropped from the chapter figure. Plate H.2 below retains them.

Splitting the data model (Figures 4.3 and 4.4). Seven entities carrying 63 fields do not fit one page at a readable size in any orientation. The chapter uses two overlapping views, one per half of the pipeline, with `Paper`, `Study` and `PopulationResolution` appearing in both because they are the entities the two halves share. Plate H.1 below is the combined view.

No reduction removes an entity, a class relationship or a package dependency without it appearing in a plate here.

H.3 Plates

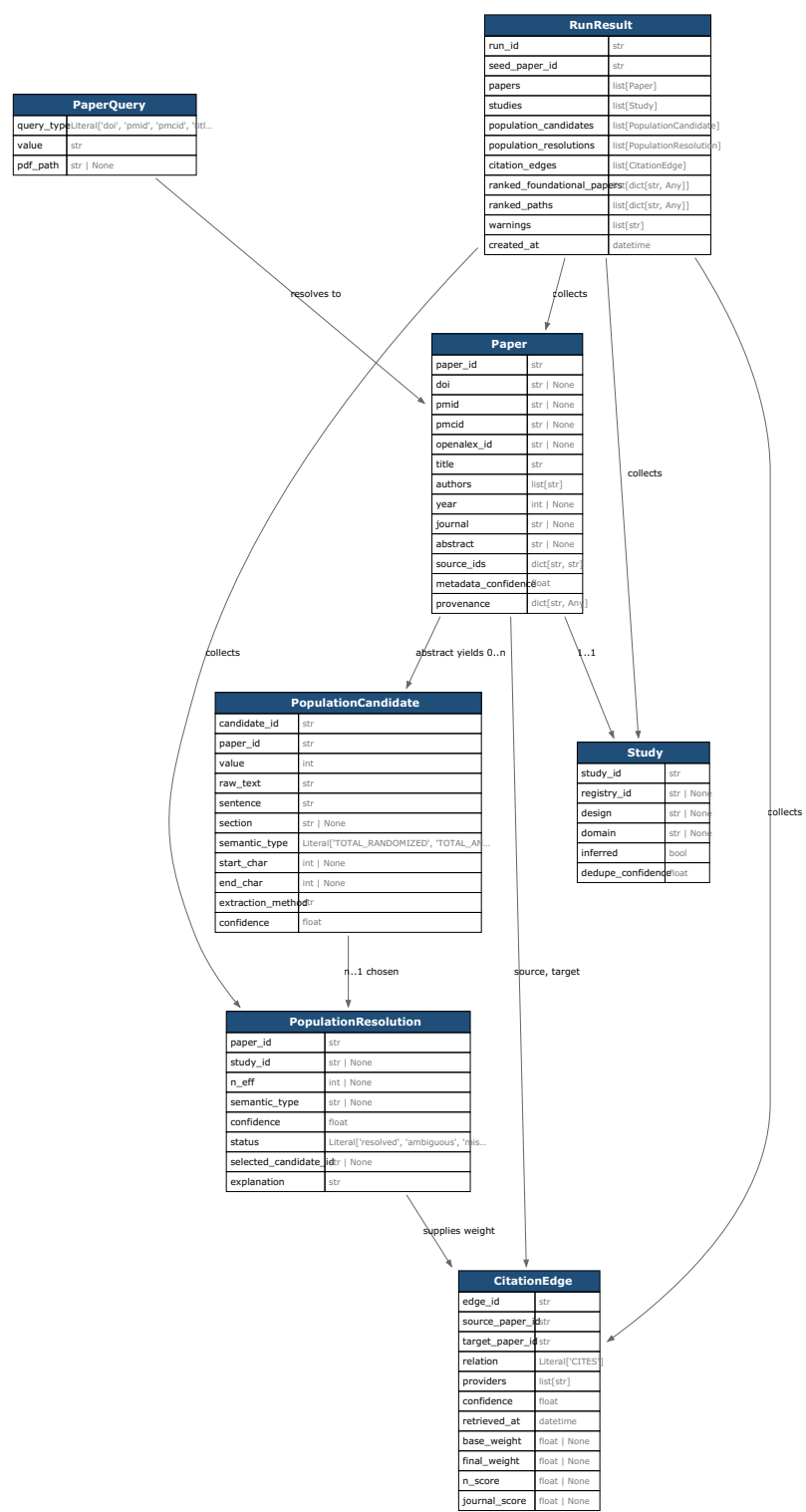


Plate H.1 The complete data model: all seven Pydantic entities with every field and type, and all ten relationships. Figures 4.3 and 4.4 are the two halves of this diagram.

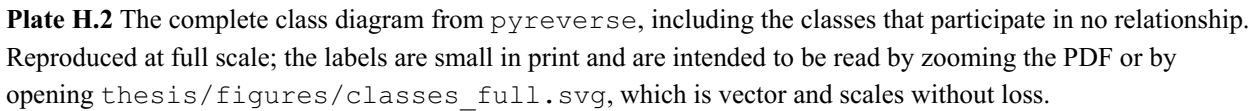


Plate H.2 The complete class diagram from `pyreverse`, including the classes that participate in no relationship. Reproduced at full scale; the labels are small in print and are intended to be read by zooming the PDF or by opening `thesis/figures/classes_full.svg`, which is vector and scales without loss.

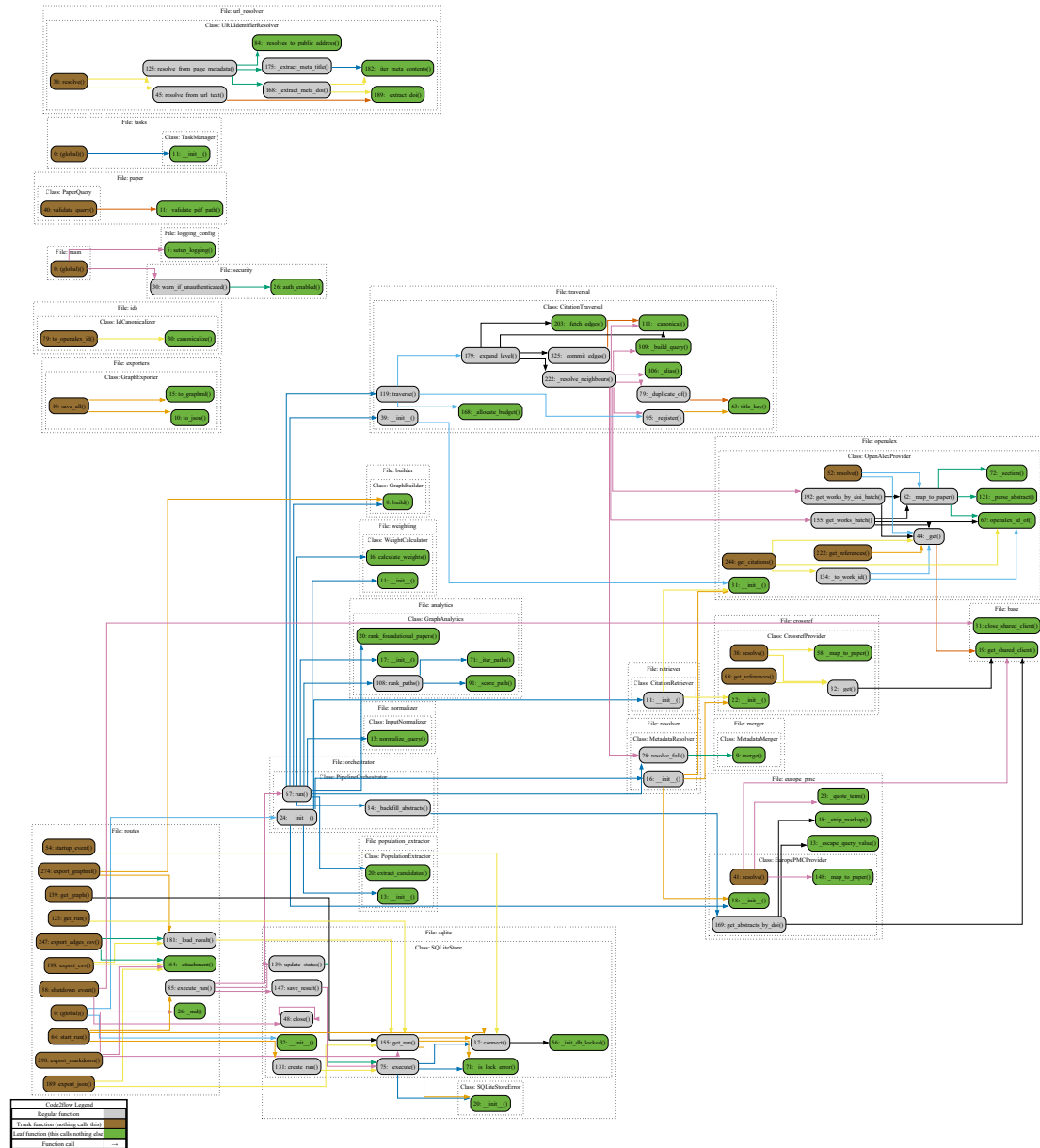


Plate H.3 The complete call graph from code2flow: 105 functions and every call between them, grouped by file and class. Figure 5.1 is the subgraph reachable from `PipelineOrchestrator.run()` within two levels.

H.4 A caveat on the call graphs

code2flow resolves calls statically by name, which has two consequences a reader should keep in mind when using Plate H.3 as a map of control flow.

Calls made through a variable whose type it cannot infer are recorded against an unknown owner and do not appear as edges. This affects the provider calls most, because the orchestrator holds providers behind the `MetadataProvider` protocol; a call to `provider.resolve()` cannot be attributed to `OpenAlexProvider`, `CrossrefProvider` or `EuropePMCProvider` without running the program. The three concrete providers therefore appear less connected in the plate than they are at runtime.

Second, the tool records a call site, not a call count or an execution order. An edge drawn once may execute once per paper in a graph of several hundred, and the left-to-right arrangement

within a rank carries no meaning. Section 6.6 gives the measured runtime distribution, which is the correct source for where time is actually spent.

Neither limitation affects the class, package or data-model figures, which are derived from declarations rather than from call sites.

References

Every entry below was resolved against Crossref (with OpenAlex as a fallback) by `scripts/verify_references.py` before being cited. Entries that failed to resolve were removed rather than cited from memory; see Section 6.7 for the three identifiers this process corrected.

All 29 entries resolve as of the verification run.

[1] `beltagy2019scibert`. Iz Beltagy, Kyle Lo, and Arman Cohan. "SciBERT: A Pretrained Language Model for Scientific Text." *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019. DOI: [10.18653/v1/D19-1371](https://doi.org/10.18653/v1/D19-1371)

Cited for: SciBERT

[2] `brin1998anatomy`. Sergey Brin, and Lawrence Page. "The anatomy of a large-scale hypertextual Web search engine." *Computer Networks and ISDN Systems*, 1998. DOI: [10.1016/S0169-7552\(98\)00110-X](https://doi.org/10.1016/S0169-7552(98)00110-X)

Cited for: Anatomy of a large-scale hypertextual search engine

[3] `button2013power`. Katherine S. Button et al.. "Power failure: why small sample size undermines the reliability of neuroscience." *Nature Reviews Neuroscience*, 2013. DOI: [10.1038/nrn3475](https://doi.org/10.1038/nrn3475)

Cited for: Small sample size undermines reliability

[4] `chen2007cocitation`. Chaomei Chen. "CiteSpace II: Detecting and visualizing emerging trends and transient patterns in scientific literature." *Journal of the American Society for Information Science and Technology*, 2005. DOI: [10.1002/asi.20317](https://doi.org/10.1002/asi.20317)

Cited for: CiteSpace / co-citation visual analytics

[5] `chen2007gems`. P. Chen, Haoxuan Xie, Sergei Maslov, and S. Redner. "Finding scientific gems with Google's PageRank algorithm." *Journal of Informetrics*, 2006. DOI: [10.1016/j.joi.2006.06.001](https://doi.org/10.1016/j.joi.2006.06.001)

Cited for: Finding scientific gems with PageRank on a citation network

[6] `europemc2015`. Anon.. "Europe PMC: a full-text literature database for the life sciences and platform for innovation." *Nucleic Acids Research*, 2014. DOI: [10.1093/nar/gku1061](https://doi.org/10.1093/nar/gku1061)

Cited for: Europe PMC full-text literature database

[7] `garfield1955`. Eugene Garfield. "Citation Indexes for Science." *Science*, 1955. DOI: [10.1126/science.122.3159.108](https://doi.org/10.1126/science.122.3159.108)

Cited for: Citation indexing as a tool for science

[8] `hagberg2008networkx`. Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. "Exploring Network Structure, Dynamics, and Function using NetworkX." *Proceedings of the Python in Science Conference*, 2008. DOI: [10.25080/TCWV9851](https://doi.org/10.25080/TCWV9851)

Cited for: NetworkX

[9] `hendricks2020crossref`. Ginny Hendricks, Dominika Tkaczyk, Jennifer Lin, and Patricia Feeney. "Crossref: The sustainable source of community-owned scholarly metadata." *Quantitative Science Studies*, 2020. DOI: [10.1162/qss_a_00022](https://doi.org/10.1162/qss_a_00022)

Cited for: Crossref as scholarly infrastructure

- [10] higgins2011cochrane. J. P. T. Higgins et al.. "The Cochrane Collaboration's tool for assessing risk of bias in randomised trials." *BMJ*, 2011. DOI: [10.1136/bmj.d5928](https://doi.org/10.1136/bmj.d5928)
Cited for: Cochrane risk of bias tool
- [11] hirsch2005hindex. J. E. Hirsch. "An index to quantify an individual's scientific research output." *Proceedings of the National Academy of Sciences*, 2005. DOI: [10.1073/pnas.0507655102](https://doi.org/10.1073/pnas.0507655102)
Cited for: h-index
- [12] ioannidis2005why. John P. A. Ioannidis. "Why Most Published Research Findings Are False." *PLoS Medicine*, 2005. DOI: [10.1371/journal.pmed.0020124](https://doi.org/10.1371/journal.pmed.0020124)
Cited for: Why most published research findings are false
- [13] jin2018pico. Di Jin, and Peter Szolovits. "PICO Element Detection in Medical Text via Long Short-Term Memory Neural Networks." *Proceedings of the BioNLP 2018 workshop*, 2018. DOI: [10.18653/v1/W18-2308](https://doi.org/10.18653/v1/W18-2308)
Cited for: PICO element detection
- [14] kim2003genia. JD Kim, Tomoko Ohta, Yuka Tateisi, and J Tsujii. "GENIA corpus—a semantically annotated corpus for bio-textmining." *Bioinformatics*, 2003. DOI: [10.1093/bioinformatics/btg1023](https://doi.org/10.1093/bioinformatics/btg1023)
Cited for: GENIA corpus for biomedical IE
- [15] lee2020biobert. Jinhyuk Lee et al.. "BioBERT: a pre-trained biomedical language representation model for biomedical text mining." *Bioinformatics*, 2019. DOI: [10.1093/bioinformatics/btz682](https://doi.org/10.1093/bioinformatics/btz682)
Cited for: BioBERT
- [16] lopez2009grobid. Patrice Lopez. "GROBID: Combining Automatic Bibliographic Data Recognition and Term Extraction for Scholarship Publications." *Lecture Notes in Computer Science*, 2009. DOI: [10.1007/978-3-642-04346-8_62](https://doi.org/10.1007/978-3-642-04346-8_62)
Cited for: GROBID
- [17] marshall2016robotreviewer. Iain J Marshall, Joël Kuiper, and Byron C Wallace. "RobotReviewer: evaluation of a system for automatically assessing bias in clinical trials." *Journal of the American Medical Informatics Association*, 2015. DOI: [10.1093/jamia/ocv044](https://doi.org/10.1093/jamia/ocv044)
Cited for: RobotReviewer: automatic risk-of-bias assessment
- [18] marshall2020trialstreamer. Iain Marshall et al.. "Trialstreamer: A living, automatically updated database of clinical trial reports." *Journal of the American Medical Informatics Association*, 2020. DOI: [10.1093/jamia/ocaa163](https://doi.org/10.1093/jamia/ocaa163)
Cited for: Trialstreamer: auto-updated RCT database
- [19] martin2021loadoi. Heather Piwowar et al.. "The state of OA: a large-scale analysis of the prevalence and impact of Open Access articles." *PeerJ*, 2018. DOI: [10.7717/peerj.4375](https://doi.org/10.7717/peerj.4375)
Cited for: Unpaywall / open access state
- [20] moher2009prisma. David Moher, Alessandro Liberati, Jennifer Tetzlaff, and Douglas G. Altman. "Preferred Reporting Items for Systematic Reviews and Meta-Analyses: The PRISMA Statement." *PLoS Medicine*, 2009. DOI: [10.1371/journal.pmed.1000097](https://doi.org/10.1371/journal.pmed.1000097)
Cited for: PRISMA reporting guideline
- [21] neumann2019scispacy. Mark Neumann, Daniel King, Iz Beltagy, and Waleed Ammar. "ScispaCy: Fast and Robust Models for Biomedical Natural Language Processing." *Proceedings*

of the 18th BioNLP Workshop and Shared Task, 2019. DOI: [10.18653/v1/W19-5034](https://doi.org/10.18653/v1/W19-5034)
Cited for: ScispaCy biomedical NLP pipeline

[22] newman2001structure. M. E. J. Newman. "The structure of scientific collaboration networks." *Proceedings of the National Academy of Sciences*, 2001. DOI: [10.1073/pnas.98.2.404](https://doi.org/10.1073/pnas.98.2.404)
Cited for: Structure of scientific collaboration networks

[23] nye2018ebmnlp. Benjamin Nye et al.. "A Corpus with Multi-Level Annotations of Patients, Interventions and Outcomes to Support Language Processing for Medical Literature." *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2018. DOI: [10.18653/v1/P18-1019](https://doi.org/10.18653/v1/P18-1019)
Cited for: EBM-NLP corpus: PICO spans in abstracts

[24] priem2022openalex. Jason R Priem et al.. "OpenAlex Snapshot." *arXiv (Cornell University)*, 2022. DOI: [10.48550/arXiv.2205.01833](https://doi.org/10.48550/arXiv.2205.01833)
Cited for: OpenAlex open scholarly catalogue

[25] radicchi2008universality. Filippo Radicchi, Santo Fortunato, and Claudio Castellano. "Universality of citation distributions: Toward an objective measure of scientific impact." *Proceedings of the National Academy of Sciences*, 2008. DOI: [10.1073/pnas.0806977105](https://doi.org/10.1073/pnas.0806977105)
Cited for: Universality of citation distributions

[26] redner1998citation. S. Redner. "How popular is your paper? An empirical study of the citation distribution." *The European Physical Journal B*, 1998. DOI: [10.1007/s100510050359](https://doi.org/10.1007/s100510050359)
Cited for: Citation distribution statistics

[27] walker2007citerank. Dylan Walker, Huafeng Xie, Koon-Kiu Yan, and Sergei Maslov. "Ranking scientific publications using a model of network traffic." *Journal of Statistical Mechanics: Theory and Experiment*, 2007. DOI: [10.1088/1742-5468/2007/06/P06010](https://doi.org/10.1088/1742-5468/2007/06/P06010)
Cited for: CiteRank: finding scientific gems

[28] waltman2016review. Ludo Waltman. "A review of the literature on citation impact indicators." *Journal of Informetrics*, 2016. DOI: [10.1016/j.joi.2016.02.007](https://doi.org/10.1016/j.joi.2016.02.007)
Cited for: Review of citation impact indicators

[29] wang2020mag. Kuansan Wang, Zhihong Shen, Chiyuan Huang, Chieh-Han Wu, Yuxiao Dong, and Anshul Kanakia. "Microsoft Academic Graph: When experts are not enough." *Quantitative Science Studies*, 2020. DOI: [10.1162/qss_a_00021](https://doi.org/10.1162/qss_a_00021)
Cited for: Microsoft Academic Graph